

CODEBASE DEEP DIVE · TECHNICAL REFERENCE

Knowledge Workspace Internals

A complete technical walkthrough of the AI-Powered Knowledge Workspace codebase — every technology, every architectural decision, and every file, from zero to full understanding.

[Next.js 15](#) · [NestJS 11](#) · [PostgreSQL + pgvector](#) · [BullMQ](#) · [Socket.IO](#) · [Grok \(xAI\)](#)

A complete technical walkthrough of the AI-Powered Knowledge Workspace codebase — from zero to full understanding.

Table of Contents

1. [Project Overview](#)
2. [Part 1 — Technology Stack - Languages - Backend Framework & Libraries - Frontend Framework & Libraries - APIs & External Services - Async Processing & Realtime - Build Tools & Package Management - Testing & Quality Assurance - Deployment & DevOps](#)
3. [Part 2 — Architecture & System Design](#)
4. [Part 3 — File Structure & Implementation](#)
5. [Part 4 — Implementation Flow \(Build Order\)](#)
6. [Part 5 — Execution Summary](#)
7. [Index of Technologies](#)
8. [Index of Files](#)

Project Overview

The **AI-Powered Knowledge Workspace** is a single-user knowledge management application. A user writes documents in a block editor, uploads PDF/DOCX/TXT/MD files, and can search everything by keyword and by meaning, and ask questions that get answered from their own content using retrieval-augmented generation (RAG).

It is a **pnpm monorepo** with two workspace packages:

- `backend/` — a NestJS 11 API (REST + Socket.IO + BullMQ workers)
- `frontend/` — a Next.js 15 App Router single-page application

Both talk to a shared set of managed services: **Supabase Postgres** (with the `pgvector` extension, holding both relational rows and embedding vectors in one database), **Upstash Redis** (backing BullMQ), **Supabase Storage** (raw file bytes), and **xAI's Grok** (the LLM). Embeddings come from either a local in-process model (`Xenova/all-MiniLM-L6-v2` , via `transformers.js`) or OpenAI's `text-embedding-3-small` , both truncated/normalized to 384 dimensions so they are interchangeable behind one interface.

The three decisions that shape almost everything else in this codebase:

1. **One Postgres, not two databases.** Vectors and rows live together, so a chunk and its parent document join in plain SQL.
2. **AIProvider and EmbeddingProvider are separate interfaces.** Grok has no embeddings endpoint; splitting the interfaces means RAG never breaks when the chat model changes.
3. **Every AI/file operation except streaming chat runs through a BullMQ queue.** Streaming chat is the one exception, because the SSE response *is* the answer — queuing it would defeat the purpose.

The rest of this document works through every technology used, how the system is put together, and what each file does — in the order you would actually want to read them to understand the project from scratch.

Part 1 — Technology Stack

Languages

TypeScript

What is it?

A statically-typed superset of JavaScript that compiles to plain JS. Every line of application code in this repository — backend and frontend — is TypeScript.

Purpose in this project

The entire monorepo (`backend/src/**/*.ts` , `frontend/**/*.ts(x)`) is TypeScript in `strict` mode. It is the one language that ties the NestJS backend, the Next.js frontend, the Prisma schema types, and the shared DTO shapes together.

Key Features Used

- Strict null checks and strict mode (`strict: true` in both `tsconfig.json` s), catching null/undefined bugs at compile time
- Decorators (`@Controller` , `@Injectable` , `@IsString` , etc.) — both NestJS and class-validator lean on TypeScript's experimental decorator support
- Discriminated unions (e.g. the SSE event union `{type: 'sources'} | {type: 'token'} | {type: 'done'} | {type: 'error'}` in `lib/api.ts` , and `JobStage` in `queue.types.ts`)
- Interfaces as the seam for swappable implementations (`AIPProvider` , `EmbeddingProvider`)
- Generated types from Prisma (`@prisma/client`) shared into service and DTO type signatures

Detailed Description

TypeScript's job here is less "catch typos" and more "make the provider-swap architecture actually safe." Because `AIPProvider` and `EmbeddingProvider` are TypeScript interfaces, swapping `GrokProvider` for a hypothetical `ClaudeProvider` is a compile-time-checked operation — the compiler will refuse to build if the new class doesn't implement every method the rest of the app calls. The same applies to the DTO layer: `class-validator` decorators double as both runtime validation and (via `class-transformer`) the TypeScript type the rest of the request-handling code sees.

On the frontend, TypeScript types in `lib/types.ts` are hand-mirrored copies of the backend's DTOs/Prisma models (there is no shared types package in this monorepo — each side maintains its own view of the wire contract). This is a deliberate simplicity trade-off: no codegen step, no shared package to version, at the cost of the two sides being able to drift silently if a field is renamed on one side and not the other.

Backend `tsconfig.json` targets ES2023 with `strictPropertyInitialization: false` — a NestJS-specific carve-out, since Nest's dependency injection assigns constructor-injected class properties without a field initializer, which the stricter setting would otherwise flag as an error.

Alternatives

1. **Plain JavaScript** — no compile step, no type errors to fight, but the swappable-provider pattern (interfaces) and the DTO validation/type inference pairing lose their compile-time guarantee; refactors become riskier as the codebase grows.
2. **Flow** — Facebook's older static type checker for JS; smaller ecosystem, no first-class NestJS/Next.js support, effectively abandoned in favor of TypeScript industry-wide.
3. **JSDoc + `checkJs`** — type-checking plain JS via comments; workable for small utility scripts but doesn't scale to decorator-heavy frameworks like NestJS.
4. **Deno with its native TS support** — would remove some build tooling, but neither NestJS's nor Next.js's ecosystem targets Deno as a first-class runtime, so this would fight the frameworks rather than the language.

Pros

- Compile-time safety for the two most fragile seams in the app: provider interfaces and DTO validation
- First-class support in both NestJS and Next.js — no bolted-on tooling
- Decorator metadata reflection (`reflect-metadata`) that NestJS's DI container depends on
- Autocomplete/refactor safety across a codebase with ~85 source files and several cross-module dependencies (e.g. `SearchModule` importing `AiModule`)
- Prisma generates types directly from `schema.prisma` , so database shape and application types can't drift on the backend
- Shared mental model for anyone moving between `backend/` and `frontend/`

Cons

- Two independently-maintained type definitions for the same wire contract (backend DTOs vs. `frontend/lib/types.ts`) — no single source of truth, so a backend field rename won't produce a frontend compile error
- Build step adds latency to iteration compared to plain JS
- Decorator-based metaprogramming (NestJS, class-validator) is powerful but adds a layer of "magic" a newcomer has to learn before the DI/validation flow makes sense
- `strictPropertyInitialization: false` on the backend slightly weakens one of `strict` 's guarantees, traded for NestJS ergonomics

Resources

- <https://www.typescriptlang.org/docs/>
- <https://www.typescriptlang.org/docs/handbook/decorators.html>
- <https://docs.nestjs.com/> (TypeScript-first framework docs)

SQL (PostgreSQL dialect, incl. pgvector & full-text search extensions)

What is it?

The query language and dialect used directly (via `Prisma.sql` / `$queryRaw` / `$executeRaw`) wherever Prisma's query builder can't express what's needed — chiefly vector similarity search and full-text search.

Purpose in this project

Two hand-written raw-SQL code paths exist specifically because Prisma has no native support for `vector` or `tsvector` columns: `indexing.service.ts` (embedding writes + cosine similarity search) and `search.service.ts` (keyword search via `websearch_to_tsquery` / `ts_rank` / `ts_headline`). The HNSW vector index and the generated `tsvector` column are both defined in hand-written migration SQL rather than `schema.prisma`.

Key Features Used

- `CREATE EXTENSION IF NOT EXISTS "vector"` — enables pgvector
- Generated column: `search_vector tsvector GENERATED ALWAYS AS (...) STORED`, weighted `setweight(to_tsvector(...), 'A'/'B')` combining title + body
- GIN index on the generated tsvector column
- HNSW index (`USING hnsw (embedding vector_cosine_ops)`) on `document_chunks.embedding` — sub-linear approximate nearest-neighbor search with no training step
- `websearch_to_tsquery` (accepts natural human search syntax, never throws on malformed input) + `ts_rank` + `ts_headline` (server-generated `<mark>` snippets)
- The `<=>` cosine-distance operator from pgvector, used as `1 - (embedding <=> query) AS similarity`

Detailed Description

Raw SQL shows up in exactly two files, both deliberately: Prisma's schema DSL cannot express a `vector(384)` column type or an HNSW index, so both are declared with `Unsupported("vector(384)")` / `Unsupported("tsvector")` in `schema.prisma` (so Prisma at least knows the column exists and won't try to drop it) and the actual DDL lives in the hand-written init migration.

The two search code paths deliberately don't try to merge their scores: full-text rank and cosine similarity are, in the codebase's own words, "incomparable scales" — so `search.service.ts` runs both in parallel and returns them side by side rather than attempting a weighted fusion.

Every raw query is built with parameterized tagged templates (`Prisma.sql` / `$queryRaw` template literals), never string concatenation — this is what keeps `websearch_to_tsquery(${trimmed})`-style queries safe from SQL injection despite being hand-written.

Alternatives

1. **Elasticsearch/OpenSearch for full-text + a vector DB (Pinecone/Weaviate/Qdrant) for embeddings** — better search-specific tooling and horizontal scale, at the cost of running and paying for two more services and reconciling two stores whenever a document changes. Rejected per the README's explicit "one Postgres" design decision.
2. **MongoDB Atlas Search + Atlas Vector Search** — single-database convenience like this project's approach, but would mean abandoning the relational/Prisma stack entirely.
3. **SQLite + sqlite-vec** — viable for a genuinely single-user local-first app, but loses managed backups/pooling and Supabase's hosted Postgres conveniences this project relies on for auth-adjacent and storage integration.
4. **Prisma's `$queryRawUnsafe`** — technically an alternative to tagged-template `$queryRaw`, but it accepts raw strings without parameterization and was correctly avoided here for injection safety.

Pros

- No second database/service to provision, back up, or keep consistent with the relational data
- HNSW needs no pre-training step, unlike IVFFlat — works immediately on a small or growing table
- `websearch_to_tsquery` degrades gracefully on malformed user input instead of raising a syntax error, so an FTS request can never crash on bad input

- Parameterized raw SQL keeps the injection-safety guarantees of an ORM while accessing Postgres-specific features
- Server-side `ts_headline` snippet generation means no client-side highlighting logic is needed

Cons

- Raw SQL bypasses Prisma's type safety — return values from `$queryRaw` are `any` /manually typed, so a schema drift here won't be caught at compile time
- `prisma migrate diff` will always flag the HNSW index as missing drift (documented explicitly in the README) since Prisma doesn't know it exists — anyone unfamiliar with this will be tempted to "fix" it and break vector search
- Keyword and semantic scores can't be merged into one ranked list without an app-level fusion step that doesn't exist yet
- Requires the specific `pgvector` Postgres extension to be installed/enabled — not portable to a vanilla Postgres instance without that step

Resources

- <https://github.com/pgvector/pgvector>
- <https://www.postgresql.org/docs/current/textsearch.html>
- <https://www.prisma.io/docs/orm/prisma-client/queries/raw-database-access>

Backend Framework & Libraries

NestJS 11

What is it?

An opinionated Node.js framework built on top of Express (or Fastify) that structures a backend into modules, controllers, providers/services, guards, filters, interceptors and pipes — heavily inspired by Angular's dependency-injection architecture.

Purpose in this project

NestJS is the entire `backend/` application: every module in `backend/src/` (`auth`, `users`, `documents`, `files`, `ai`, `search`, `queue`, `events`, `common`, `prisma`) is a Nest module wired together in `app.module.ts`.

Key Features Used

- Modules (`@Module`) for every feature area, plus a `@Global()` `PrismaModule` and `EventsModule` so `PrismaService` and the Socket.IO gateway don't need re-importing everywhere
- Guards (`JwtAuthGuard` globally via `APP_GUARD`, `GithubAuthGuard`, `ThrottlerGuard`) — `JwtAuthGuard` is default-deny, opted out per-route with a custom `@Public()` decorator
- Global exception filter (`AllExceptionsFilter`, `APP_FILTER`) normalizing every thrown error (HTTP exceptions, known Prisma errors, unexpected errors) into one JSON shape
- Global interceptor (`LoggingInterceptor`) logging method/URL/duration for every request
- `ValidationPipe` applied globally with `whitelist`, `forbidNonWhitelisted`, and `transform: true` — DTOs are the only way data enters a controller method

- `@nestjs/websockets` + `@nestjs/platform-socket.io` for the realtime gateway; `@nestjs/bullmq` for typed queue/processor decorators; `@nestjs/throttler` for rate limiting; `@nestjs/passport` + `@nestjs/jwt` for auth

Detailed Description

NestJS gives this backend its module boundaries "for free" through the DI container: `SearchModule` can `import: [AiModule]` and inject `IndexingService` without either module needing to know about the other's internals beyond the exported provider. This is what makes the AI-provider swap (README's "one class + one `useClass` line") actually a one-line change — `AiModule`'s `useClass: GrokProvider` binding is the only place a provider is chosen; every consumer only ever sees the `AIProvider` interface token.

The framework's guard/interceptor/filter pipeline is applied globally rather than per-controller, which is why `JwtAuthGuard` had to invent an escape hatch (`@Public()` + `Reflector.getAllAndOverride`) instead of simply not applying the guard to public routes — a deliberate default-deny security posture: a newly added controller is authenticated by default unless someone explicitly opts it out.

Nest's module system also cleanly separates *producing* work from *consuming* it: `QueueModule` (global, producer-only) is imported by feature modules that need to enqueue jobs, while `WorkerModule` (consumer-only, imports `AiModule` + `FilesModule`) holds the actual `@Processor` classes — avoiding a circular-import problem that would otherwise occur if every feature module needed to know about every processor.

Alternatives

1. **Express (bare)** — what Nest is built on; far less structure, no DI, no decorator-based routing — would require hand-rolling everything Nest's guards/pipes/filters give for free, at the benefit of a much smaller mental model and dependency footprint.
2. **Fastify (bare, or via `@nestjs/platform-fastify`)** — faster raw request throughput than Express; Nest supports it as a swappable HTTP adapter, but this project uses the default Express adapter, likely because Passport/GitHub OAuth middleware compatibility with Express is more battle-tested.
3. **Koa** — smaller, middleware-centric alternative to Express; no official Nest adapter, would mean abandoning the framework entirely.
4. **trRPC** (paired with a lighter HTTP layer) — end-to-end type safety without hand-mirrored DTOs on the frontend, but no natural fit for the file-upload/Socket.IO/BullMQ-worker architecture this project needs, and no equivalent to Nest's guard/module system.

Pros

- DI container makes the "swap the AI provider in one line" and "swap the embedding provider via config" design decisions actually enforceable, not just aspirational
- Global guards/filters/interceptors mean every one of the ~10 feature modules gets consistent auth, error shape, and logging without repeating boilerplate per controller
- First-class, well-maintained integrations for exactly the pieces this project needs (BullMQ, WebSockets, Passport, class-validator via `ValidationPipe`)
- Structure scales predictably as modules are added — a newcomer who understands one module (e.g. `files`) can predict the shape of another (e.g. `documents`)
- Built-in testing utilities (`@nestjs/testing`'s `Test.createTestingModule`) make mocking Prisma/Queue trivial in `.spec.ts` files
- Decorator-based routing (`@Get()` , `@Post('/:id')`) keeps controller code declarative and short

Cons

- Heavier learning curve than a bare Express app — modules, providers, DI tokens, decorators are all new concepts for someone coming from simpler Node backends

- Decorator/reflection-based DI adds a layer of implicit "magic" (e.g. `@Global()` , `Reflector.getAllAndOverride`) that isn't visible just from reading a class in isolation
- More boilerplate for very small features (a trivial module still needs its own `*.module.ts` file)
- Express as the default underlying adapter is not the fastest option available (Fastify is faster), though this project doesn't appear to be throughput-bound

Resources

- <https://docs.nestjs.com/>
- <https://docs.nestjs.com/fundamentals/custom-providers>
- <https://docs.nestjs.com/security/authentication>

Prisma 6 (ORM)

What is it?

A type-safe database toolkit for Node/TypeScript: a schema DSL (`schema.prisma`) that generates a fully-typed client, plus a migration system.

Purpose in this project

Prisma is the entire data-access layer for relational reads/writes across every module (`User` , `Document` , `File` , `DocumentChunk` , `AIRequest` , `RefreshToken`). It also carries the two `Unsupported(...)` column types (`vector(384)` , `tsvector`) so raw SQL can coexist with the typed client in the same schema file.

Key Features Used

- `$transaction` for atomic multi-query operations (e.g. `documents.findAll` 's `findMany+count`, `users.dashboard()` 's six-query aggregate)
- `$queryRaw` / `$executeRaw` tagged templates for pgvector/tsvector operations Prisma's query builder can't express
- Generated enums (`AIRequestType` , `AIRequestStatus` , `FileStatus` , `ChunkSource`) shared as TypeScript types across services
- `onDelete: Cascade` / `onDelete: SetNull` relational actions declared in-schema (e.g. deleting a `Document` cascades to its `DocumentChunk` rows automatically — no app-level cleanup code needed)
- `directUrl` vs. pooled `url` split specifically for Supabase's transaction-mode pooler (6543) vs. direct connection (5432) migrations
- `postgresExtensions` preview feature to declare the `vector` extension in-schema

Detailed Description

Prisma is deliberately used for everything it's good at (typed CRUD, relations, transactions, migrations) and deliberately bypassed for everything it can't express (vector similarity, full-text search) rather than forcing a workaround. The schema still declares those two special columns via `Unsupported(...)` purely so Prisma's own drift-detection knows they exist and won't try to drop them — the actual index/generated-column DDL lives in the raw init migration SQL, which the README explicitly warns not to "fix" when `prisma migrate diff` reports it as missing.

`PrismaService` extends `PrismaClient` directly and is registered in a `@Global()` module, so it's injectable everywhere without every feature module re-importing `PrismaModule`. It also exposes a `truncateAll()` helper (hard-guarded to throw outside test/dev) used by the e2e suite to reset state between runs, and wires `enableShutdownHooks` so the Nest app closes cleanly on Prisma's `beforeExit`.

Alternatives

1. **Drizzle ORM** — lighter-weight, SQL-closer alternative with a TypeScript-first schema; would express the raw SQL parts (vector ops) more natively via its SQL template helper, but has a smaller migration/tooling ecosystem than Prisma's.
2. **TypeORM** — Nest's other commonly-paired ORM; more "magic" active-record-style patterns, historically weaker TypeScript type inference than Prisma's generated client.
3. **Kysely** — a purely type-safe SQL query builder (no ORM abstraction); would fit this project's already-frequent raw-SQL needs well, at the cost of writing every relation/model by hand instead of Prisma's schema-first generation.
4. **Raw pg / node-postgres** — maximum control, matches the vector/FTS code paths this project already needs, but loses migrations, typed models, and relation cascades for the 90% of the app that's plain CRUD.

Pros

- Generated TypeScript types keep model shape and application code in sync automatically after every `prisma generate`
- Migration history is explicit and versioned (`prisma/migrations/*`), while still allowing hand-written raw SQL migrations for pgvector-specific DDL
- `$transaction` gives atomic multi-query operations without manually managing a client/connection
- Relational cascade rules live in the schema, not scattered across service code
- Works cleanly with Supabase's connection pooler via the `directUrl` / `url` split
- Strong local dev ergonomics (`prisma studio` , `prisma migrate dev`)

Cons

- No native vector or full-text-search column types — anything using them needs raw SQL and loses type safety on those queries
- `prisma migrate diff` permanently reports the hand-written HNSW index as drift, which is confusing without the README's explicit warning
- Query builder abstraction can hide N+1 query patterns if used carelessly in more complex relation loading (not observed as a problem in this codebase, but a known Prisma pitfall)
- Client generation is a build step (`postinstall: prisma generate`) that must run before TypeScript compilation succeeds

Resources

- <https://www.prisma.io/docs>
- <https://www.prisma.io/docs/orm/prisma-client/using-raw-sql>
- <https://github.com/pgvector/pgvector#prisma>

Passport.js (passport , passport-github2 , passport-jwt)

What is it?

A modular authentication middleware for Node.js built around pluggable "strategies" — each strategy implements one authentication mechanism (OAuth providers, JWT bearer tokens, local username/password, etc).

Purpose in this project

Two strategies are used: `passport-github2` for the GitHub OAuth login flow, and `passport-jwt` for validating the short-lived access token on every protected API request. Both are wired through `@nestjs/passport`'s `AuthGuard()` factory.

Key Features Used

- `passport-github2`'s `user:email` scope request (needed because GitHub hides private emails from the bare profile response)
- `passport-jwt`'s Bearer-token extraction + signature verification against the configured JWT secret
- Nest's `AuthGuard('github')` / `AuthGuard('jwt')` guard subclasses (`GithubAuthGuard` , `JwtAuthGuard`)
- Strategy `validate()` hooks doing a **live database lookup** on every JWT-authenticated request (not just trusting the token's claims), so a deleted/banned account can't ride a still-valid access token for its remaining lifetime

Detailed Description

The GitHub strategy's `validate()` method is the single place where a GitHub profile becomes an application `User` row — it maps the OAuth profile into `GithubProfileData` and calls `AuthService.validateGithubUser`, which upserts on `githubId` and only overwrites `email` when GitHub actually supplied one (subsequent OAuth logins often omit the email field if the user has kept it private).

The JWT strategy's decision to hit the database on every request is a deliberate trade against pure-stateless-JWT performance: a 15-minute access token is short-lived enough that this cost is bounded, and it buys the ability to instantly revoke a compromised account instead of waiting up to 15 minutes for a stale token to expire naturally.

Alternatives

1. `next-auth` / `Auth.js` — a much higher-level, batteries-included auth library popular in Next.js projects; would need to run server-side in Next.js though, which doesn't fit this project's split-backend architecture (NestJS owns all auth state).
2. **Lucia** (or a hand-rolled OAuth2 client) — lighter-weight, no Passport strategy ecosystem dependency; would mean hand-implementing the GitHub OAuth handshake instead of using a maintained strategy package.
3. **Auth0 / Clerk / WorkOS (hosted auth)** — offloads the entire OAuth+session flow to a managed service; adds a paid third-party dependency and less control over the specific refresh-token-rotation/reuse-detection scheme this project implements itself.
4. `jsonwebtoken` **directly (no Passport)** — Passport's JWT strategy here is a thin wrapper around signature verification; using the `jsonwebtoken` library directly inside a custom guard would remove one dependency layer at the cost of reimplementing Passport's guard integration.

Pros

- Well-established, widely audited strategy ecosystem — GitHub OAuth and JWT bearer auth are both "solved problems" here, not custom crypto
- Clean separation between *how* a user authenticates (strategy) and *what happens after* (guard + `validate()` callback)

- Nest's `@nestjs/passport` integration means strategies plug directly into the guard pipeline with no extra glue code
- The live-DB-lookup pattern in `JwtStrategy.validate()` is a simple, auditable way to get near-instant revocation without a token blacklist/Redis lookup

Cons

- Passport's `req.user` -mutation pattern is a slightly dated, side-effecting style compared to more modern typed-context approaches
- A DB round trip on every authenticated request adds latency compared to trusting the JWT claims outright (an explicit, documented trade-off here, not an oversight)
- `passport-github2` is a community-maintained package (not first-party GitHub or Passport-core), so its update cadence depends on volunteer maintenance
- Session-less (`session: false`) Passport usage sidesteps most of Passport's session-management features — only a thin slice of the library is actually used

Resources

- <https://www.passportjs.org/>
- <https://docs.nestjs.com/recipes/passport>
- <https://github.com/jaredhanson/passport-github2>

class-validator & class-transformer

What is it?

`class-validator` adds decorator-based runtime validation (`@IsString()` , `@IsIn()` , `@MaxLength()` , etc.) to plain TypeScript classes. `class-transformer` handles converting plain JSON objects into instances of those classes (and back), including type coercion.

Purpose in this project

Every DTO in the backend (`CreateDocumentDto` , `SummarizedDto` , `CreateUploadDto` , `EnvironmentVariables` , etc.) is validated this way, wired in globally via Nest's `ValidationPipe({whitelist, forbidNonWhitelisted, transform})` . It's also used once outside the HTTP layer — `env.validation.ts` validates `process.env` itself at boot via `validateSync` .

Key Features Used

- Field-level decorators (`@IsString` , `@IsObject` , `@IsIn` , `@Min / @Max` , `@MaxLength` , `@IsOptional`)
- `whitelist: true` — strips any request field not declared on the DTO, closing off mass-assignment style bugs
- `forbidNonWhitelisted: true` — actively rejects (400) requests containing undeclared fields, rather than silently dropping them
- `transform: true` — coerces incoming JSON into real class instances with correct primitive types (e.g. query-string numbers become actual `number` s)
- Cross-file reuse of a single validation source of truth (e.g. `CreateUploadDto` 's `type` field validates via `@IsIn(Object.keys(SUPPORTED_MIME_TYPES))` , importing the whitelist straight from `text-extractor.service.ts`)

instead of duplicating it)

- Boot-time environment validation (`validateSync` in `env.validation.ts`) — the app refuses to start with a misconfigured `.env` instead of failing on the first real request

Detailed Description

The combination of `whitelist` + `forbidNonWhitelisted` + `transform` is what makes DTOs the *only* way data can enter a controller method in this codebase — any field a client sends that isn't declared on the DTO class causes an immediate 400, and any field that is declared gets both validated and type-coerced before the controller method body ever runs. This removes an entire category of "did I forget to check this field" bugs.

The environment-variable validation is the same tool applied to a different boundary: `EnvironmentVariables` is a class with the same decorators, and `validateSync` runs it once at process boot (`ConfigModule.forRoot({validate: validateEnv})`), so a missing `JWT_SECRET` or a `JWT_SECRET` under 32 characters kills the process immediately with a clear error, instead of only surfacing on the first auth request in production.

Alternatives

1. **Zod** — schema-first (rather than decorator/class-first) validation with excellent TypeScript type inference; increasingly popular in newer Nest and Next.js projects, and would remove the "decorator magic" but requires restructuring DTOs as schemas rather than classes.
2. **Joi** — mature, schema-based validation library; not decorator-based, so it doesn't integrate as natively with Nest's DTO-as-class pattern without extra wiring.
3. **Yup** — similar schema-based approach to Zod, more common historically in frontend form validation than backend API validation.
4. **Manual validation (hand-written if/throw checks)** — no dependency, full control, but no declarative single source of truth and every DTO needs its own boilerplate checks repeated by hand.

Pros

- Declarative, colocated validation rules right on the DTO class — no separate schema file to keep in sync
- `whitelist` / `forbidNonWhitelisted` close off mass-assignment and unexpected-field bugs by default, project-wide, from one global pipe registration
- Reusable across two very different boundaries in this codebase: HTTP request bodies and process environment variables
- Integrates natively with Nest's `ValidationPipe` — no custom glue code per controller
- Supports importing shared constants into decorator arguments (`@IsIn(Object.keys(SUPPORTED_MIME_TYPES))`), keeping a single source of truth across files

Cons

- Decorator-based validation errors can be less immediately readable than a schema library's structured error paths
- Two separate libraries (`class-validator` + `class-transformer`) working together adds conceptual overhead versus a single unified library like Zod
- Class-based DTOs carry more boilerplate (constructor-less classes, one file per shape) than schema objects would
- Runtime validation logic is invisible to TypeScript's compiler — a decorator typo silently does nothing rather than failing to compile

Resources

- <https://github.com/typestack/class-validator>

- <https://github.com/typestack/class-transformer>
- <https://docs.nestjs.com/techniques/validation>

Async Processing & Realtime

BullMQ + ioredis

What is it?

BullMQ is a Redis-backed job queue library for Node.js — producers push jobs onto named queues, workers consume them with configurable concurrency, retries, backoff and delays. `ioredis` is the underlying Redis client BullMQ (and this project's own Redis config helper) is built on.

Purpose in this project

Every AI operation except streaming chat, every uploaded-file processing job, and every document re-indexing run goes through one of three BullMQ queues (`files` , `ai` , `indexing`), defined once in `queue.constants.ts` and consumed by three dedicated `@Processor` classes in `worker.module.ts` .

Key Features Used

- Three named queues with `defaultJobOptions` : `attempts: 3` , exponential backoff starting at 2s, `removeOnComplete` / `removeOnFail` retention windows
- **Fixed, deterministic job IDs** used as idempotency/dedup keys instead of a separate dedup table: `file-${fileId}` , `ai-${aiRequestId}` , `index-${documentId}`
- Delayed jobs with **explicit cancel-and-reschedule debouncing**: `enqueueIndexing` removes any existing `index-${documentId}` job before re-adding one with a 15-second delay, so a burst of autosaves collapses into a single indexing run
- `@Processor` / `WorkerHost` / `@OnWorkerEvent('failed')` from `@nestjs/bullmq` , checking `job.attemptsMade >= job.opts.attempts` before writing a terminal `FAILED` status — so a DB row isn't marked failed until every retry is actually exhausted
- Per-processor `concurrency` tuning (`AiProcessor : 4` , `FileProcessor` / `IndexingProcessor : 2` — the latter two are CPU-bound, tuned down for a single-core deployment target)
- `ioredis` 's `RedisOptions` built manually from `REDIS_URL` in `redis.config.ts` , detecting `rediss://` for TLS and setting `maxRetriesPerRequest: null` / `enableReadyCheck: false` — both required by BullMQ's blocking-command usage, the latter also needed because Upstash restricts the `INFO` command BullMQ would otherwise call

Detailed Description

The queue/worker split across two Nest modules (`QueueModule` , global, producer-only vs. `WorkerModule` , consumer-only) exists specifically to avoid a circular dependency: any feature module can inject `QueueService` and enqueue work without importing the processor classes that actually execute it, while `WorkerModule` alone imports `AiModule` and `FilesModule` to get the services its processors need to do real work.

The debounced-indexing pattern is arguably the single cleverest piece of async design in the codebase: rather than a separate debounce timer living in application memory (which would break across multiple server instances or process restarts), the debounce

is implemented *as a queue operation itself* — removing then re-adding a job with the same fixed ID and a fresh delay. This means the debounce state lives in Redis, not in a Node process, so it survives restarts and works correctly even if the backend eventually runs as multiple replicas.

Failure handling checks `attemptsMade` against the configured `attempts` count before writing a terminal failure state to the database — this avoids a subtle bug where the first of three retry attempts failing would prematurely tell the user (via the DB row and the Socket.IO event derived from it) that the whole job failed, when BullMQ was actually about to retry it automatically.

Alternatives

1. **AWS SQS + Lambda** — fully managed queue and workers, but this project's workers need long-lived state (the local embedding model stays loaded in memory across jobs) and Socket.IO needs a persistent process — a poor fit for Lambda's stateless, short-lived execution model.
2. **RabbitMQ (with `amqp-lib`)** — more general-purpose message broker with richer routing (exchanges, topics); heavier operationally than a Redis-backed queue for a project that already needs Redis for nothing else.
3. **Agenda / Bree** (MongoDB or in-process cron-style job runners) — simpler for basic scheduled jobs, but lack BullMQ's built-in retry/backoff/concurrency/rate-limiting primitives this project actively uses.
4. **pg-boss** (Postgres-backed queue) — would avoid needing Redis at all, reusing the already-present Postgres instance; a real alternative, but BullMQ's maturity and the project's separate need for Redis-backed Socket.IO scaling (not currently used, but architecturally available) likely tipped the choice toward Redis-based queuing.

Pros

- Retry/backoff/concurrency/delay are all built-in primitives, not hand-rolled
- Fixed job IDs give free idempotency — re-enqueuing the same logical job (e.g. reprocessing a file) can't create duplicate work
- Delayed-job debouncing solves the "autosave burst" problem without any in-process timers or extra database bookkeeping
- Clean separation of producer/consumer concerns across two Nest modules avoids circular imports
- Mature, actively maintained, TypeScript-native

Cons

- Requires Redis as an additional infrastructure dependency (Upstash in production) beyond Postgres
- Workers currently run in the same process as the API server (explicitly called out as a known limitation in a code comment) — a CPU-heavy embedding job can, in principle, compete with the API for the same core
- BullMQ's Redis connection requirements (`maxRetriesPerRequest: null`, blocking commands) are easy to misconfigure against a managed Redis provider with restricted commands, as the `enableReadyCheck: false` workaround for Upstash demonstrates
- No built-in dead-letter-queue UI out of the box (would need Bull Board or similar for operational visibility, not present in this codebase)

Resources

- <https://docs.bullmq.io/>
- <https://github.com/luin/ioredis>
- <https://docs.nestjs.com/techniques/queues>

Socket.IO (`socket.io` + `socket.io-client`)

What is it?

A realtime, bidirectional event-based communication library over WebSocket (falling back to HTTP long-polling), with server and client packages that share a protocol.

Purpose in this project

`events.gateway.ts` runs a single Socket.IO server inside the NestJS process, used exclusively to push job-progress and resource-update notifications to the browser — never for request/response data, which stays REST/SSE.

Key Features Used

- Custom JWT handshake authentication (`handshake.auth.token` or `Authorization` header), verified with the same `JwtService` /secret as REST auth, disconnecting unauthenticated sockets immediately
- **Per-user rooms** (`user:${userId}`) — every emit targets `to(userRoom(userId))` , never a broadcast, so one user's job progress can never leak to another connected client
- Two event types: `job:progress` (`JobProgressEvent` : jobId, resourceId, resource type, stage, percent, message/error) and `resource:updated` (tells the client to refetch a resource)
- `transports: ['websocket', 'polling']` on the server, but the client forces `transports: ['websocket']` only — skipping the long-polling fallback intentionally on the client side
- CORS origin resolved as a **function evaluated per-handshake** rather than at decorator-eval time, reading `process.env.FRONTEND_URL` directly — necessary because `ConfigModule` hasn't finished loading `.env` yet when Nest evaluates gateway decorators

Detailed Description

The gateway is deliberately a thin, one-directional notification channel: it never receives meaningful data from the client (beyond the initial auth handshake) and never carries the actual answer to any operation — that job is split between REST polling (for queued AI operations) and the socket purely providing a progress percentage/stage in between. The three BullMQ workers each call into the same injected gateway singleton (made possible by `EventsModule` being `@Global()`) to emit progress at each stage of their pipeline (e.g. `FileProcessor` 's `extracting` → `chunking` → `embedding` → `completed` sequence).

On the frontend, the socket connection is opened once a valid access token exists (inside `RealtimeBridge` in `providers.tsx`) and is a module-level singleton rather than a per-component connection — reused across the whole app and explicitly torn down on logout.

Alternatives

1. **Server-Sent Events (SSE) for everything** — the project already uses raw SSE for one thing (streaming chat), and could in principle push job-progress the same way, but SSE is one-directional-per-connection and would need one open connection per in-flight job rather than one shared connection carrying many jobs' progress.
2. **Polling only (no realtime channel)** — simpler infrastructure (no WebSocket server to run/scale), and the codebase actually keeps polling as a fallback anyway (React Query's `refetchInterval` for AI jobs, a 4-second poll for pending files) — but a pure-polling design would feel noticeably less responsive for progress bars.
3. **Pusher / Ably / Supabase Realtime (hosted pub/sub)** — offloads running a WebSocket server entirely; would remove Socket.IO from the backend but add a paid third-party dependency and a second place auth has to be re-verified.

4. **Raw ws library** — lower-level, no automatic reconnection/fallback/room abstraction; would mean re-implementing per-user targeting and reconnection logic Socket.IO provides out of the box.

Pros

- Room-based targeting (`user:${userId}`) makes tenant isolation for realtime events trivial and hard to get wrong
- Automatic reconnection and transport fallback handled by the library, not hand-rolled
- One shared connection can carry progress events for arbitrarily many concurrent background jobs
- JWT handshake auth reuses the exact same secret/verification logic as REST, so there's only one auth scheme to reason about
- The codebase treats it as a "nice to have" progress channel with polling fallbacks everywhere it matters, so a dropped socket connection degrades gracefully rather than breaking functionality

Cons

- Running a stateful WebSocket server means the backend can't be deployed as a serverless function — this is explicitly why Render (a long-lived process host) was chosen over a serverless platform for the backend
- Horizontal scaling of the backend would need a Redis adapter for Socket.IO to synchronize rooms across instances — not currently configured, since the project runs a single backend instance
- The CORS-origin-as-a-function workaround (needed due to `ConfigModule` load ordering) is a subtle gotcha that would confuse anyone refactoring gateway bootstrapping
- Real-time is entirely progress/notification metadata, not primary data — so it adds an entire subsystem for what is, functionally, a UX nicety layered on top of REST polling

Resources

- <https://socket.io/docs/v4/>
- <https://docs.nestjs.com/websockets/gateways>
- <https://socket.io/docs/v4/client-api/>

APIs & External Services

xAI Grok (via the `openai` SDK)

What is it?

Grok is xAI's family of large language models. Its API is wire-compatible with OpenAI's Chat Completions API, so this project talks to it using the official `openai` npm SDK pointed at a different `baseUrl` .

Purpose in this project

Grok is the sole `AIProvider` implementation (`GrokProvider`) — every text-generation feature (summarize, rewrite, generate, chat, streaming chat) ultimately calls Grok's `grok-4-fast` model.

Key Features Used

- OpenAI-SDK-compatible Chat Completions endpoint reached via a custom `GROK_BASE_URL` (`https://api.x.ai/v1`)
- `response_format: {type: 'json_object'}` for the summarize call, paired with a system prompt that explicitly contains the word "JSON" (a documented requirement — the SDK/API 400s on structured-output requests otherwise)
- Native streaming (`stream: true`) consumed as an async iterable, re-exposed as `chatStream()` — an `AsyncIterable<string>` yielding `delta.content` chunks
- SDK-level `maxRetries: 2` and `timeout: 120_000` configured once at client construction
- A local `truncate()` guard capping prompt input at ~100k characters (a `chars/4` token-count heuristic, no real tokenizer dependency) before every call

Detailed Description

Every method on `GrokProvider` funnels through one private `complete()` helper, so error handling (wrapping any SDK/network failure into a `ServiceUnavailableException`) is centralized in one place rather than repeated per public method. The summarize path additionally defines `parseSummary()` , a small parser that strips markdown code fences the model sometimes wraps JSON in and falls back to treating the raw text as the summary if JSON parsing still fails — a defensive measure against an LLM not perfectly honoring the requested format.

Because `AIPProvider` is an interface and `GrokProvider` its only current implementation, adding a second text-generation provider (Claude, Gemini, OpenAI's own chat models) is, per the README, "writing one class and changing one `useClass` line in `ai.module.ts` " — nothing else in the codebase references `GrokProvider` by name.

Alternatives

1. **OpenAI GPT models directly** — the SDK is already a dependency (used for embeddings); using it for chat too would mean paying OpenAI's pricing instead of xAI's, and losing whatever model-specific behavior/cost profile motivated choosing Grok.
2. **Anthropic Claude (via `@anthropic-ai/sdk`)** — would require its own SDK (different from the OpenAI-compatible wire format Grok/Groq share) and a new provider class, but is explicitly the kind of swap this architecture was built to support cheaply.
3. **Groq (the inference-speed company, not xAI's Grok)** — the `.env.example` file itself documents Groq as a drop-in alternative base URL/model (`https://api.groq.com/openai/v1` , `llama-3.3-70b-versatile`) since it's also OpenAI-wire-compatible — effectively a one-env-var swap without even a new provider class.
4. **Self-hosted open models (via Ollama/vLLM, also usually OpenAI-wire-compatible)** — would remove per-token API costs and external dependency entirely, at the cost of hosting/serving infrastructure and likely quality trade-offs versus a frontier hosted model.

Pros

- OpenAI SDK compatibility means no bespoke HTTP client was needed — retries, streaming, and typed responses come from a mature, widely-used library
- Cheap to swap for a same-shaped provider (Groq) via env vars alone, or a differently-shaped provider (Claude) via one new class
- Streaming support is native to the SDK, not hand-rolled over raw fetch
- Centralized error wrapping (`complete()`) gives one consistent failure mode (`ServiceUnavailableException`) across every AI feature

Cons

- The whole app's text-generation quality and behavior is currently tied to one model family's quirks (e.g. the JSON-mode prompt requirement, occasional markdown-fenced JSON needing defensive parsing)

- No token-accurate truncation — the `chars/4` heuristic can under- or over-estimate real token counts depending on content (code vs. prose vs. non-English text)
- Streaming chat is the one AI code path that runs synchronously inside an HTTP request rather than through the BullMQ queue, so a slow/stuck Grok stream ties up a request-handling connection for its duration
- Dependent on a third-party API's uptime and rate limits for every AI feature in the product

Resources

- <https://docs.x.ai/>
- <https://platform.openai.com/docs/api-reference/chat> (the wire protocol both APIs share)
- <https://github.com/openai/openai-node>

Embeddings: `@xenova/transformers` (local) & OpenAI `text-embedding-3-small`

What is it?

`@xenova/transformers` (`transformers.js`) runs Hugging Face-format ML models directly in a Node.js process via ONNX Runtime — no external API call, no GPU required for small models. It's paired here with `Xenova/all-MiniLM-L6-v2`, a compact sentence-embedding model. The alternative provider calls OpenAI's hosted `text-embedding-3-small` embeddings endpoint via the same `openai` SDK already used for Grok.

Purpose in this project

Both are implementations of the same `EmbeddingProvider` interface, chosen at boot via the `EMBEDDING_PROVIDER` env var (`local` default, or `openai`). Whichever is active turns document/chunk text into the 384-dimension vectors stored in `document_chunks.embedding` and used for semantic search and RAG retrieval.

Key Features Used

- **Local:** lazy `import()` of the model (only loaded on first real use, not at boot), a single-flight promise so concurrent embedding jobs share one loaded pipeline instance instead of racing to load the model twice; `pipeline('feature-extraction', modelId)` configured with `pooling: 'mean', normalize: true` — L2-normalizing output vectors so cosine similarity reduces to a plain dot product, matching pgvector's `vector_cosine_ops` index type exactly
- **OpenAI:** `dimensions: 384` parameter on the embeddings call — a Matryoshka-style truncation from the model's native 1536 dimensions down to 384, chosen specifically so both providers emit identically-shaped vectors and the `pgvector` column never needs a dimension-changing migration when swapping providers
- Batch embedding (`embedMany`) used by `indexing.service.ts` in groups of 32 chunks at a time
- OpenAI response re-sorted by `item.index` before use, since the API doesn't guarantee response ordering matches request ordering

Detailed Description

The local provider is the default specifically because it has no per-call cost and no API key requirement — the trade-off is a one-time ~25MB model download and a 2-4 second cold-start on first use, which is why embedding *only* ever happens inside a BullMQ worker

(`FileProcessor` / `IndexingProcessor`), never inline in an HTTP request — a multi-second stall would be unacceptable in a request/response cycle but is invisible inside an already-asynchronous background job.

The decision that both providers emit exactly 384 dimensions is what keeps the `pgvector` column's fixed width stable across a provider swap — this is called out explicitly as a deliberate design decision in the README, not an accidental convergence, and is why OpenAI's naturally-1536-dimension model is truncated via its own `dimensions` parameter rather than accepted at full size.

Alternatives

1. **OpenAI `text-embedding-3-large`** — higher quality/dimensionality embeddings at higher cost; would still need the same truncation trick to fit the 384-dim column, or a schema migration to widen it.
2. **Cohere Embed / Voyage AI** — dedicated embeddings-as-a-service providers, sometimes tuned specifically for retrieval tasks; would be a third `EmbeddingProvider` implementation, same swap pattern.
3. **A different local model** (e.g. `bge-small-en` , `gte-small`) — similarly-sized alternatives to MiniLM runnable through the same `@xenova/transformers` pipeline, differing mainly in benchmark recall/precision trade-offs.
4. **Running a dedicated embeddings microservice** (e.g. a Python FastAPI service wrapping `sentence-transformers`) — would move the CPU cost off the Node process entirely, at the cost of an extra service to deploy/scale/secure.

Pros

- Free, private, and zero-latency-to-external-network for the default local option — meaningful for a project positioned as privacy-conscious "your own content" search
- Fixed 384-dimension contract across both providers means switching is a config change, not a schema migration
- Normalizing local embeddings at generation time (`normalize: true`) avoids a separate normalization step before the cosine-distance query
- Batched embedding calls (groups of 32) reduce per-call overhead versus embedding one chunk at a time
- Swappable without touching any calling code — `IndexingService` only depends on the `EmbeddingProvider` interface

Cons

- Local model quality (MiniLM, a small/older sentence-embedding model) trails larger hosted embedding models on recall-heavy benchmarks — the README itself notes "set it to openai... for better recall"
- Local embedding is CPU-bound and competes for the same core the API server runs on in a single-instance deployment, motivating the workers' reduced concurrency settings
- Truncating OpenAI's native 1536-dim embeddings to 384 via the `dimensions` parameter necessarily discards information relative to using the full-size vector
- No tokenizer-level truncation guard visible on embedding input size beyond whatever the chunking step already bounds it to

Resources

- <https://huggingface.co/docs/transformers.js/index>
- <https://huggingface.co/Xenova/all-MiniLM-L6-v2>
- <https://platform.openai.com/docs/guides/embeddings>

Supabase (Postgres, Storage & JS client)

What is it?

Supabase is a managed backend platform built on Postgres, bundling a hosted database (with extensions like `pgvector` enabled), an S3-compatible object storage service, auth, and realtime — this project uses only the **Postgres hosting** and **Storage** pieces, via the `@supabase/supabase-js` client.

Purpose in this project

Supabase Postgres is the production database (`pgvector` included) behind Prisma. Supabase Storage holds every uploaded file's raw bytes in a private bucket (`knowledge-files`), accessed through signed URLs rather than Supabase's own row-level-security-based client auth.

Key Features Used

- Managed Postgres with `pgvector` pre-installable, reachable via both a transaction-mode connection pooler (port 6543, used by the running app) and a direct connection (port 5432, used only for Prisma migrations)
- `createSignedUploadUrl` — mints a one-time-use upload URL + token the browser can PUT bytes to directly, without ever proxying through the NestJS API
- `createSignedDownloadUrl` — time-boxed (`expiresInSeconds` , default 3600) read URLs for previews/downloads
- Service-role key used **server-side only** (never sent to the browser) — bypasses Supabase's row-level security entirely, since authorization is enforced by this app's own JWT/ownership checks instead
- `NEXT_PUBLIC_SUPABASE_ANON_KEY` used client-side only for the one call that needs it (`uploadToSignedUrl`), which is safe to expose because the anon key alone grants no storage access without a valid signed token

Detailed Description

The two-key model here (service-role key on the backend, anon key on the frontend) is a deliberate security boundary: the backend uses its elevated key only to *mint* narrowly-scoped signed URLs, never to hand out broad access, while the browser's anon key is functionally inert without one of those signed tokens. This means a compromised frontend build could leak the anon key with no meaningful impact, while the service-role key never leaves the server.

Bypassing the API server for the actual file bytes (the "uploads bypass the API server" decision from the README) is what lets a 20MB PDF cost the NestJS process only two small JSON round trips (mint ticket, confirm completion) instead of buffering 20MB of multipart request body — a real scalability/cost consideration on a resource-constrained single-instance backend host.

Alternatives

1. **AWS S3 (+ RDS Postgres separately)** — the "build it yourself from primitives" version of the same architecture; more configuration surface (IAM policies, bucket policies, presigned URL generation via the AWS SDK) for the same signed-URL pattern, and a separate managed Postgres service to provision.
2. **Cloudflare R2 + Neon/PlanetScale Postgres** — cheaper egress (R2) and a different Postgres-hosting model (Neon's branching); R2 doesn't natively support `pgvector` -style Postgres extensions, so the "one database for rows and vectors" decision would need Neon or similar instead.
3. **Firebase Storage + Firestore** — a fully different (NoSQL) data model that would conflict with this project's relational-plus-pgvector design entirely; not a realistic drop-in.
4. **Self-hosted MinIO (S3-compatible) + self-hosted Postgres** — full control and no vendor lock-in, at the cost of operating both services yourself instead of using a managed platform.

Pros

- One provider for both the vector-capable Postgres database and object storage, reducing the number of external services/vendors to manage
- Signed-URL upload pattern keeps large file bytes off the API server entirely
- Service-role vs. anon key separation gives a clean, auditable security boundary between server and browser capabilities
- Storage API is S3-compatible enough to be familiar to anyone who's used AWS S3 presigned URLs before

Cons

- Adds a vendor dependency beyond "just Postgres" — migrating off Supabase later means migrating both the database and the storage bucket
- The service-role key is extremely powerful (bypasses all RLS) — a leak of that specific key would be a serious incident, even though the anon key alone is harmless
- Supabase's connection pooler (transaction mode) has known compatibility caveats with certain Postgres features/prepared statements, which is part of why a separate `directUrl` is needed just for migrations
- Storage bucket privacy/policy configuration is a manual setup step (the README calls out creating a **private** bucket explicitly) that's easy to get wrong

Resources

- <https://supabase.com/docs>
- <https://supabase.com/docs/guides/storage>
- <https://supabase.com/docs/guides/database/extensions/pgvector>

File Text Extraction: `pdf-parse` & `mammoth`

What is it?

Two focused, single-purpose Node libraries: `pdf-parse` extracts plain text from PDF file buffers; `mammoth` extracts raw text (or HTML) from `.docx` (Word) buffers.

Purpose in this project

`text-extractor.service.ts` uses both as the two non-trivial branches of its MIME-type dispatch — PDF and DOCX are the two upload formats that need real parsing (TXT/MD are just decoded as UTF-8 directly).

Key Features Used

- `pdfParse(buffer).text` — single-call text extraction from a `Buffer`, no intermediate file needed
- `mammoth.extractRawText({buffer})` — plain-text extraction (not the HTML-conversion mode mammoth also offers), with conversion warnings logged at debug level rather than surfaced to the user
- Both wrapped by a shared `normalise()` post-processing step (CRLF→LF, whitespace collapsing) since PDFs in particular are noted in a code comment to produce "ragged whitespace that wrecks chunk boundaries" if left unprocessed

Detailed Description

Both libraries are used at exactly one seam — turning uploaded bytes into the plain text that everything downstream (chunking, embedding, full-text search, AI prompts) actually operates on. This is also why `text-extractor.service.ts`'s `SUPPORTED_MIME_TYPES` map is imported directly by the upload DTO (`CreateUploadDto`) for validation — the set of formats the app can actually parse is defined once, in the extraction service, not duplicated in a separate validation list.

Alternatives

1. `pdfjs-dist` (**Mozilla's PDF.js**) — a much larger, more capable PDF library (rendering, not just text extraction); would be overkill for plain-text-only extraction needs.
2. `docx` **npm package** (structured DOCX parsing, not just raw text) — useful if the app needed to preserve DOCX formatting/structure; this project only needs flattened plain text, so `mammoth`'s simpler raw-text mode is a better fit.
3. **A hosted document-extraction API (e.g. Unstructured.io, AWS Textract)** — more robust extraction (tables, layout, OCR for scanned PDFs) at the cost of an external network dependency and per-document cost for a feature currently free and local.
4. `textextract` (**npm, wraps multiple CLI tools**) — broader format support via external binaries (e.g. `antiword` , `pdftotext`), but adds native/binary dependencies this project's pure-JS approach avoids.

Pros

- Both are pure-JS(ish), lightweight, single-purpose libraries with no external service dependency or per-document cost
- Buffer-in-text-out APIs integrate cleanly with the Supabase Storage download flow (bytes never touch disk)
- A single `normalise()` step shared across all supported formats keeps downstream chunking behavior consistent regardless of source format

Cons

- Neither handles scanned/image-only PDFs (no OCR) — such a file would extract as empty or garbled text
- `mammoth`'s raw-text mode discards all DOCX formatting/structure (tables, headers) — acceptable here since the target is a flat plain-text document, but a real limitation if richer conversion were ever needed
- `pdf-parse` has known rough edges on complex, multi-column, or unusually-encoded PDFs
- Both are relatively unmaintained/low-velocity packages compared to more actively developed alternatives

Resources

- <https://www.npmjs.com/package/pdf-parse>
- <https://www.npmjs.com/package/mammoth>

Sentry (`@sentry/node` , `@sentry/nextjs`)

What is it?

An error-tracking and performance-monitoring platform; the Node and Next.js SDKs capture unhandled exceptions, stack traces, and (optionally) performance traces, shipping them to a Sentry project dashboard.

Purpose in this project

Both the backend (`instrument.ts` , imported first in `main.ts`) and the frontend (`instrumentation.ts` + `instrumentation-client.ts`) initialize Sentry — but only if a DSN environment variable is actually set, making it a fully optional, no-op-by-default piece of the stack for local development.

Key Features Used

- Backend: `Sentry.init` guarded by `SENTRY_DSN` presence, `tracesSampleRate` set to 0.1 in production and 0 in development; the global `AllExceptionsFilter` explicitly calls `Sentry.captureException` for every 5xx error
- Frontend: dual instrumentation entry points split by Next.js runtime (`register()` for server/nodejs and edge runtimes, a separate browser-side init in `instrumentation-client.ts`) since each runs in a different JS environment and needs its own `Sentry.init` call
- Session-replay/error-replay sampling explicitly disabled on the frontend client init

Detailed Description

The "no-op without a DSN" design means Sentry adds zero friction to local development or running the e2e test suite — nobody needs a Sentry account to clone and run this project. In production, it's wired into exactly one place on the backend (the global exception filter), so every unhandled error anywhere in the app funnels through one `Sentry.captureException` call rather than needing per-module instrumentation.

Alternatives

1. **Rollbar / Bugsnag** — comparable hosted error-tracking services with similar SDK shapes; would be a drop-in-ish swap at the SDK-integration points.
2. **Self-hosted GlitchTip** (open-source, Sentry-protocol-compatible) — same client SDKs, no third-party hosted service or its cost, at the cost of operating the collector yourself.
3. **OpenTelemetry + a generic APM backend** (Honeycomb, Datadog, Grafana) — a more vendor-neutral, broader observability approach (traces+metrics+logs unified), heavier to set up for a project whose current need is just "tell me when something crashes."
4. **No error tracking (rely on hosting-platform logs only)** — the zero-dependency option; loses structured error grouping, stack traces, and alerting that a dedicated tool provides.

Pros

- Fully optional/no-op without configuration — doesn't burden local dev or CI
- One integration point on the backend (the global filter) captures every unhandled error project-wide
- Separate server/edge/browser instrumentation on the frontend correctly matches Next.js's actual multi-runtime execution model
- Mature product with strong stack-trace source-mapping support for both Node and Next.js

Cons

- A third-party service dependency for production observability — outages or misconfiguration there mean silent blind spots
- Sampling (`tracesSampleRate: 0.1`) means only a fraction of transactions get full performance tracing in production, by design, trading completeness for cost
- Adds SDK weight to both the backend and frontend bundles even when unused (DSN unset) — though the runtime cost of the no-op path is negligible

Resources

- <https://docs.sentry.io/platforms/node/>
- <https://docs.sentry.io/platforms/javascript/guides/nextjs/>

Frontend Framework & Libraries

Next.js 15 (App Router)

What is it?

A React meta-framework providing file-system-based routing, server/client component rendering, build tooling, and deployment conventions — version 15 uses the "App Router" (the `app/` directory convention) rather than the older Pages Router.

Purpose in this project

The entire `frontend/` package is a Next.js 15 App Router application. Every route lives under `app/`, split into a public area (`login`, `auth/callback`) and an authenticated route group (`(workspace)`).

Key Features Used

- Route groups — `app/(workspace)/` groups the six authenticated pages (dashboard, documents, documents/[id], files, search, assistant, settings) under one shared `layout.tsx` without adding a `/workspace` URL segment
- Dynamic routes (`app/(workspace)/documents/[id]/page.tsx`)
- Client components (`'use client'`) used pervasively — this app is authenticated, interactive, and state-heavy enough that almost nothing benefits from server rendering
- `output: 'standalone'` build mode (conditionally, only for Docker builds) producing a minimal self-contained server bundle
- Next's Image remote-pattern whitelist (GitHub avatars, Supabase) for `next/image`
- Dual instrumentation hooks (`instrumentation.ts` for server/edge, `instrumentation-client.ts` for the browser) — a Next.js 15-specific convention for runtime-aware SDK initialization (used here for Sentry)

Detailed Description

This is a deliberately client-heavy use of Next.js: there is no server-side auth check anywhere in the App Router tree (a code comment in the workspace layout explicitly justifies this — the session lives in an `httpOnly` cookie the Next.js server has no cheap way to validate, and every API call is independently authorized by the NestJS backend anyway). The workspace `layout.tsx` guard is pure client-side: it watches a Zustand auth store and renders only a loading spinner until authentication status resolves, so protected pages never mount before auth is known one way or the other.

The route-group pattern (`(workspace)`) is used purely for layout-sharing and code organization — it has no effect on the URL structure — letting six otherwise-unrelated pages share one sidebar/topbar/command-palette shell.

Alternatives

1. **Next.js Pages Router** — the older, file-based routing convention; would lose React Server Component support and the newer layout-nesting model, though this app barely uses server rendering anyway, so the practical difference here is smaller than it

would be for a content-heavy site.

2. **Remix** — a comparable full-stack React framework with a different data-loading philosophy (loaders/actions tied to routes); would fit a more server-rendering-centric app better than this client-heavy SPA-style one.
3. **Vite + React Router (a pure SPA, no meta-framework)** — arguably a more honest fit for how this app is actually used (almost entirely client-rendered, no SEO need behind an auth wall) — Next.js's SSR/RSC machinery is largely unused here beyond routing conventions and the build pipeline.
4. **Astro** — optimized for content-heavy, mostly-static sites with islands of interactivity; a poor fit for an interactive, always-authenticated dashboard app like this one.

Pros

- File-system routing keeps the six workspace pages' structure self-evident from the directory tree alone
- Route groups cleanly separate "needs auth shell" from "public" pages without polluting the URL
- Built-in image optimization, font handling, and a mature deployment story (Vercel, or standalone Docker builds elsewhere)
- Dual instrumentation hooks correctly acknowledge Next's multiple JS runtimes (Node, Edge, browser) instead of assuming one global environment

Cons

- Much of the App Router's headline feature set (Server Components, streaming SSR, server actions) goes essentially unused in an app this client-rendering-heavy — arguably more framework than the use case strictly needs
- No server-side route protection means an unauthenticated user's browser does briefly load the workspace shell's JS bundle before the client-side redirect fires (a minor UX/security-theater gap, not a real data exposure since the API still enforces auth independently)
- `output: 'standalone'` vs. default build mode is a config branch (`DOCKER_BUILD` env flag) that has to be remembered/kept consistent between Vercel and Docker deployment targets
- Next.js's opinionated conventions (file naming, special files like `layout.tsx`) require learning the framework's specific vocabulary before the directory structure is self-explanatory

Resources

- <https://nextjs.org/docs>
- <https://nextjs.org/docs/app/building-your-application/routing/route-groups>
- <https://nextjs.org/docs/app/building-your-application/upgrading/instrumentation>

TanStack Query (React Query) v5

What is it?

A server-state management library for React — handles fetching, caching, background refetching, and mutation/optimistic-update patterns, so components don't manually manage loading/error/cache state for API data.

Purpose in this project

Every piece of data that comes from the backend REST API (documents, files, dashboard stats, search results, AI job status) flows through a React Query hook (`hooks/use-*.ts`). Zustand (see below) is reserved for genuinely client-only state; React Query owns everything that originates on the server.

Key Features Used

- Query keys namespaced per domain (`documents.list(q)` , `documents.detail(id)` , `documents.recent` , `aiKeys.history` , etc.) for precise cache invalidation
- **Optimistic updates:** `useUpdateDocument` and `useDeleteDocument` both use `onMutate` to snapshot and patch the cache immediately, with `onError` rollback — this is what makes the editor's autosave and document deletion feel instant rather than waiting on a round trip
- `refetchInterval` -based polling for queued AI jobs (`useAiJob` , polling every 1.5s until a job leaves `PENDING` / `PROCESSING`) and for pending file uploads (`useFiles` , 4s poll) — explicitly kept as a fallback alongside Socket.IO events "to stay correct across reconnects"
- Query gating via `enabled` (e.g. semantic search only fires once the query string is longer than 1 character)
- Global defaults set once in `providers.tsx` : 30s `staleTime` , no refetch-on-window-focus, and retries skipped entirely for 401/403/404 responses (since those represent auth/not-found states, not transient failures worth retrying)

Detailed Description

The polling-as-fallback pattern is a deliberate resilience choice documented directly in the hook code: Socket.IO gives near-instant progress updates when connected, but a dropped/reconnecting socket shouldn't mean a job's final status is lost — React Query's interval polling guarantees the UI eventually reflects the true server state regardless of socket health, at the cost of a small amount of redundant network traffic while the socket is healthy.

Optimistic updates are scoped carefully to operations where the outcome is virtually certain to succeed (renaming/editing your own document, deleting your own document) — a case where the UX benefit of zero perceived latency outweighs the small risk of a visible rollback on a rare failure.

Alternatives

1. **SWR** (Vercel's own data-fetching library) — simpler API surface, similar stale-while-revalidate caching model; lacks React Query's more elaborate mutation/optimistic-update tooling and devtools.
2. **Apollo Client** — the natural choice if the backend were GraphQL; this backend is REST, so Apollo's cache-normalization strengths wouldn't be leveraged and its GraphQL-specific tooling would be dead weight.
3. **RTK Query (Redux Toolkit)** — comparable feature set, but would require adopting Redux as the surrounding state architecture, which this project deliberately avoids in favor of the much lighter Zustand.
4. **Manual `useEffect` + `useState` data fetching** — no dependency, but every one of the optimistic-update, cache-invalidation, and polling-with-socket-fallback patterns used here would need hand-rolling per hook, a substantial amount of repeated logic.

Pros

- Optimistic updates + automatic rollback make the editor and document-list UX feel instantaneous without hand-written reconciliation logic
- Query-key-based cache invalidation gives fine-grained control over what refetches after a mutation
- Built-in polling (`refetchInterval`) is exactly the right tool for the queued-AI-job status pattern, and composes cleanly with the socket-based fast path
- DevTools (`@tanstack/react-query-devtools`) included, giving visibility into the cache during development

- Global `retry/staleTime` defaults set once, so individual hooks don't need to repeat configuration

Cons

- A second state-management concept (alongside Zustand) that a newcomer needs to learn where the line is drawn — "is this server state or client state?"
- Polling as a fallback means some redundant network requests happen even when the socket connection is perfectly healthy
- Cache key structure (`documents.list(q)` , etc.) is a hand-maintained convention, not enforced by the library — a typo'd key silently creates a cache miss rather than an error
- Optimistic updates add complexity (snapshot/rollback logic) to mutations that use them, versus a simpler wait-for-server-response approach

Resources

- <https://tanstack.com/query/latest/docs/framework/react/overview>
- <https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

Zustand

What is it?

A minimal, hook-based client-state management library for React — a store is just a function returning state and actions, with no providers, reducers, or boilerplate action-type constants required.

Purpose in this project

Exactly two stores exist: `auth-store.ts` (access token, current user, auth status) and `ui-store.ts` (command palette open/closed, AI panel open/closed, and the live job-progress map fed by Socket.IO). Everything else — anything that originates from the API — lives in React Query instead.

Key Features Used

- Plain `create()` store definition with typed state + actions in one object, no separate reducer/action-type files
- No persistence middleware on `auth-store` — deliberately: `accessToken` and `user` live only in memory, never `localStorage` , so an XSS vulnerability can't exfiltrate a long-lived credential (the durable credential is the `httpOnly` refresh cookie, which JS can't read at all)
- A `status: 'loading' | 'authenticated' | 'anonymous'` state machine that every route guard in the app reads (the workspace layout, the login page, the callback page)
- `ui-store` 's `jobs: Record<resourceId, JobProgressEvent>` map, written to by the Socket.IO event bridge and read by any component showing a progress bar

Detailed Description

The deliberate absence of persistence middleware on the auth store is the single most security-relevant frontend decision in the codebase: on every hard page refresh, `accessToken` / `user` are wiped from memory, and `SessionBootstrap` (in `providers.tsx`) immediately calls the refresh endpoint using the `httpOnly` cookie to silently re-authenticate. This means the only

long-lived credential that ever touches persistent browser storage is a cookie JavaScript cannot read — a meaningfully smaller attack surface than storing a JWT in `localStorage`, at the cost of one extra network round trip on every page load.

The two-store split cleanly maps to the "server state vs. client state" boundary React Query and Zustand divide between them: `ui-store`'s job-progress map is genuinely ephemeral, session-local UI state that has no meaning to persist or refetch, which is exactly what Zustand (not React Query) is for.

Alternatives

1. **Redux Toolkit** — far more structure (slices, reducers, middleware, devtools) than two small stores need; would be a significant boilerplate increase for no functional gain here.
2. **React Context** + `useReducer` — no extra dependency at all, but loses Zustand's selector-based re-render optimization (components subscribing to one slice of state don't re-render when an unrelated slice changes) without manually splitting into multiple contexts.
3. **Jotai / Recoil** (atom-based state) — a genuinely different mental model (many small atoms vs. a few larger stores); would fit a more fine-grained-state app, but this project's needs (two small, cohesive stores) don't obviously benefit from atomization.
4. **Valtio** (proxy-based mutable state) — a different ergonomics trade-off (mutate state directly rather than via setter functions); a reasonable alternative, mostly a stylistic preference versus Zustand's explicit-setter approach.

Pros

- Near-zero boilerplate — a store is a plain function, no providers to wrap the app in
- No persistence middleware on the auth store is a deliberate, meaningful security choice, not an oversight
- Selector-based subscriptions avoid unnecessary re-renders without manual `React.memo` /context-splitting
- Small enough (two stores) that the client/server-state boundary with React Query stays easy to reason about
- TypeScript-friendly with no extra typing ceremony beyond the state interface itself

Cons

- No official devtools as polished as Redux's time-travel debugging (a browser extension exists but is less commonly reached for)
- Losing all auth state on every hard refresh means a refresh always costs one network round trip before the app is usable — a real (small) latency cost paid for the security benefit
- With only two stores, the project hasn't needed to establish conventions for a much larger app with many more Zustand stores — that scaling question is untested here
- No built-in undo/redo or time-travel, if either were ever needed for the editor

Resources

- <https://docs.pmnd.rs/zustand/getting-started/introduction>

Tiptap (`@tiptap/*`) + ProseMirror

What is it?

Tiptap is a headless, extensible rich-text editor framework built on top of ProseMirror (`@tiptap/pm`), the underlying document-model/schema/transaction engine. Tiptap supplies React bindings and a large ecosystem of pluggable "extensions" (one per block type

or behavior).

Purpose in this project

Tiptap is the block editor for documents (`features/editor/editor.tsx`) and the engine behind the `/` slash-command menu (`features/editor/slash-command.tsx`). Its JSON document format is also what's stored directly in the `Document.content` column and converted to/from plain text by `tiptap.util.ts` on the backend.

Key Features Used

- `StarterKit` (bundling paragraphs, headings limited to levels 1-3, lists, blockquotes, code blocks, etc.) as the base extension set
- `Placeholder` , `TaskList` + `TaskItem` (nested checklist support), `Image` , `Link` (with autolink enabled, click-to-navigate disabled)
- A custom `SlashCommand` extension built on `@tiptap/suggestion` , Tiptap's official primitive for "type a trigger character, show a filtered popup, insert on selection" interactions — used here instead of hand-building caret-position tracking
- `immediatelyRender: false` — a Next.js 15-specific flag avoiding SSR/hydration mismatches, since Tiptap's DOM-dependent editor view can't meaningfully render on the server
- Debounced autosave (`onUpdate` resets a 900ms timer before calling the update mutation) plus explicit `Ctrl/Cmd+S` to force-flush and save immediately
- `editor.commands.setContent(...)` used when switching between documents, preserving the single editor instance rather than remounting it per document

Detailed Description

Tiptap's JSON document schema is the same shape stored in Postgres (`Document.content`) and walked by the backend's own `tiptapToPlainText` / `plainTextToTiptap` functions — meaning the frontend editor and the backend's text-flattening logic both operate on the exact same document format, with no separate "rich text" vs. "plain text" storage model to keep in sync. When a file is uploaded and its extracted text needs to become an editable document, the backend's `plainTextToTiptap` produces exactly the JSON shape Tiptap expects, so uploaded files open in the same editor as hand-written notes.

The custom slash-menu is one of the more intricate pieces of frontend code in the project: it uses `tippy.js` for popup positioning (manually created/destroyed on `onStart` / `onExit` rather than a persistent component), renders the dropdown itself as a React component via Tiptap's `ReactRenderer` , and forwards raw keyboard events (arrow keys, Enter) from Tiptap's `onKeyDown` into the popup component via `useImperativeHandle` — necessary because the popup isn't a normal focused DOM element that would naturally receive keyboard events on its own.

Alternatives

1. **Slate.js** — another popular headless React rich-text framework; more low-level/DIY than Tiptap (you build more of the schema/plugin system yourself), trading Tiptap's larger ready-made extension ecosystem for more control.
2. **Lexical** (Meta's editor framework) — newer, performance-focused, plugin-based; smaller extension ecosystem than Tiptap/ProseMirror's more mature one at the time this project was likely built.
3. **Editor.js** — a block-based (not rich-text/inline-focused) editor with a JSON output format; conceptually similar to Tiptap's block approach but a different underlying architecture (blocks as discrete tools, not a ProseMirror document tree).
4. **Quill** — a long-established rich-text editor; less extensible/composable than Tiptap's extension model, and its own document format (Delta) would need its own flatten-to-plain-text logic instead of reusing Tiptap's schema.

Pros

- Extension-based architecture matches this project's exact needs (headings, lists, tasks, images, links, code, plus a fully custom slash-menu extension) without forking the library

- One JSON document schema shared identically between "rich document" storage, plain-text flattening, and file-upload-to-document conversion — no format translation layer needed
- `@tiptap/suggestion` gives a maintained, correct implementation of the notoriously fiddly "trigger character + filtered popup + keyboard navigation" UX pattern
- React bindings (`@tiptap/react`) integrate cleanly with hooks and component lifecycle, including the Next.js SSR-safety flag

Cons

- ProseMirror's underlying document/transaction model has a real learning curve beneath Tiptap's friendlier surface API — debugging schema or transaction issues eventually requires ProseMirror-level understanding
- The custom slash-command extension's imperative keyboard-forwarding (`useImperativeHandle`) is one of the least "React-idiomatic" pieces of the frontend, needed specifically because Tiptap's popup isn't a naturally-focused DOM element
- A relatively large dependency footprint (nine separate `@tiptap/*` packages) for what is, feature-wise, a single editor component
- `tippy.js` is an additional, separate positioning library rather than something built into Tiptap/React itself

Resources

- <https://tiptap.dev/docs>
- <https://tiptap.dev/docs/editor/api/utilities/suggestion>
- <https://prosemirror.net/docs/>

UI Layer: Tailwind CSS + Radix UI (shadcn-style primitives) + cmdk + Framer Motion

What is it?

Four pieces that together form the visual/interaction layer: **Tailwind CSS** is a utility-class CSS framework; **Radix UI** (`@radix-ui/react-*`) supplies unstyled, fully-accessible component primitives (dialog, dropdown, tabs, avatar, scroll-area, separator, slot); **cmdk** is a specific headless primitive for command-palette/fuzzy-search UIs; **Framer Motion** handles animation.

Purpose in this project

`components/ui/*.tsx` are thin, hand-written wrappers around Radix primitives styled with Tailwind — the "shadcn/ui" pattern (copy-owned component code, not an installed component library) rather than importing a pre-built component kit. `cmdk` powers `command-palette.tsx`. Framer Motion animates the sidebar's active-route highlight, dashboard stat-card stagger-in, and the AI panel's collapse/expand width transition.

Key Features Used

- `class-variance-authority` (cva) for variant-based className generation (e.g. `Button` 's default/destructive/outline/secondary/ghost/link × size variants)
- `clsx` + `tailwind-merge`, combined into one `cn()` helper, resolving conflicting Tailwind classes predictably when variants and one-off overrides combine

- CSS custom properties (`hsl(var(--x))`) for every color token, switched by `next-themes` 's class-based dark-mode strategy — the same component code renders correctly in both themes without conditional logic
- Radix's `asChild` / `Slot` pattern for polymorphic components (e.g. a `Button` that renders as an `<a>` when needed, without wrapper-element duplication)
- `cmdk` 's `Command.Dialog` as the entire structural/keyboard-navigation engine behind the `Ctrl/Cmd+K` command palette, combining static navigation links with live query results (recent documents, semantic search hits)
- Framer Motion's `layoutId` shared-element technique for the sidebar's animated active-route highlight pill, and simple stagger/width-transition animations elsewhere

Detailed Description

The "own the component code" (shadcn-style) approach means `components/ui/*.tsx` are ordinary project files, not an opaque installed dependency — `card.tsx` 's factory function generating `Card` / `CardHeader` / `CardTitle` /etc. from one shared `make()` helper, or `misc.tsx` 's hand-rolled `Progress` bar (built without a Radix progress primitive, since Radix doesn't ship one used here) are exactly the kind of small customization this pattern makes trivial — there's no library API to fight, just a local file to edit.

Radix's actual contribution is accessibility and interaction correctness that's easy to get subtly wrong by hand: focus trapping in dialogs, correct ARIA attributes on dropdown menus, keyboard navigation in tabs — all "solved" by the underlying primitive, with only visual styling layered on top via Tailwind classes.

Alternatives

1. **A pre-built component kit (Chakra UI, Mantine, MUI)** — faster initial setup (import and use, no local component files to write), but far less customizable without fighting the library's own theming system, and a much heavier bundle/API surface than this project needs.
2. **Headless UI** (Tailwind Labs' own primitive library) — a close alternative to Radix, similarly unstyled and accessible; the choice between the two is largely which primitives each ships (Radix has a notably broader set, including the dropdown/dialog/tabs combination used here).
3. **Plain hand-rolled components with no primitive library** — maximum control, but reimplementing correct focus management and ARIA behavior for a dialog or dropdown from scratch is a well-known source of accessibility bugs.
4. **A different CSS approach** (CSS Modules, vanilla-extract, styled-components) instead of Tailwind — would trade utility-class terseness for more traditional component-scoped stylesheets; a stylistic choice rather than a functional one for this app's needs.

Pros

- Component code lives in the project, fully readable and editable — no fighting a third-party library's theming API to make a small visual change
- Radix guarantees accessibility correctness (focus management, ARIA, keyboard nav) for the genuinely tricky interactive primitives (dialog, dropdown, tabs)
- Tailwind's utility classes plus `cn()` keep styling colocated with markup, with predictable override resolution via `tailwind-merge`
- CSS-custom-property-based theming makes dark/light mode a variable swap, not conditional component logic
- `cmdk` specifically solves fuzzy-search-plus-keyboard-navigation for the command palette — a nontrivial UX pattern to get right by hand

Cons

- The shadcn-style "copy the code in" pattern means bug fixes/improvements upstream don't arrive automatically — each component file has to be manually updated if ever needed

- Utility-class-heavy JSX can be visually dense/harder to scan than named CSS classes, especially on components with many conditional variants
- A meaningful number of small dependencies (seven separate `@radix-ui/react-*` packages, plus `cva` , `clsx` , `tailwind-merge` , `cmdk`) for what functions as one cohesive design system
- Framer Motion adds bundle weight for what are, in this app, fairly modest animation needs (a highlight pill, a stagger, a width transition)

Resources

- <https://tailwindcss.com/docs>
- <https://www.radix-ui.com/primitives/docs/overview/introduction>
- <https://ui.shadcn.com/> (the pattern this project's `components/ui` follows, without using the shadcn CLI itself)
- <https://cmdk.paco.me/>
- <https://www.framer.com/motion/>

Build Tools & Package Management

pnpm 10 (workspaces)

What is it?

A fast, disk-space-efficient Node.js package manager using a content-addressable global store and strict, non-flattened `node_modules` linking — and, via `pnpm-workspace.yaml` , native monorepo support.

Purpose in this project

`pnpm-workspace.yaml` declares `backend` and `frontend` as the two workspace packages. Root `package.json` scripts use `pnpm --filter` to run commands scoped to one or both packages (`pnpm --filter "./backend" build` , `pnpm --parallel --filter "./backend" --filter "./frontend" dev`).

Key Features Used

- Workspace `--filter` flags for per-package scripts (`pnpm --filter @kw/backend prisma:deploy` , `pnpm --filter @kw/frontend build`)
- `onlyBuiltDependencies` allowlist in `pnpm-workspace.yaml` — pnpm 10 blocks lifecycle/postinstall scripts by default for security, so packages that genuinely need theirs (`@prisma/client` , `@prisma/engines` , `prisma` , `msgpackr-extract` , `protobufjs` , `sharp`) are explicitly allowlisted rather than globally re-enabling all postinstall scripts
- `--frozen-lockfile` installs in CI and Docker builds — refuses to proceed if `pnpm-lock.yaml` is out of sync with `package.json` , catching an un-committed dependency change before it reaches a deployed build
- `packageManager: "pnpm@10.30.3"` pinned in root `package.json` , enforcing a single pnpm version across all contributors/CI via Corepack

Detailed Description

The `onlyBuiltDependencies` allowlist is worth calling out specifically: pnpm 10 made blocking arbitrary postinstall scripts the default (a supply-chain-security hardening measure, since a malicious/compromised package can otherwise run arbitrary code at install time), and this project's workspace config represents someone having actually audited which of its dependencies need that capability (Prisma's own engine download, and native-binary-build packages like `sharp` / `msgpackr-extract` / `protobufjs`) rather than reflexively disabling the protection.

Alternatives

1. **npm workspaces** — built into npm itself, no extra tool, but historically slower installs and a flat `node_modules` structure with less strict dependency isolation than pnpm's symlinked store.
2. **Yarn (Berry / v2+, workspaces + PnP)** — comparable monorepo tooling and a similarly strict dependency-resolution mode (Plug'nPlay); a real alternative, differing mainly in ecosystem tooling maturity and specific edge-case compatibility with packages expecting a real `node_modules`.
3. **Turborepo / Nx** (build-orchestration layered on top of a package manager) — would add task caching and dependency-graph-aware build ordering across the two packages; this project's two-package monorepo is small enough that plain `pnpm --filter` scripts suffice without needing a dedicated build-orchestration tool yet.
4. **Two entirely separate repositories** (no monorepo) — simpler tooling, but loses the ability to `pnpm install` once for both packages and to run coordinated scripts (`pnpm dev` starting both frontend and backend together) from one root.

Pros

- Disk-efficient content-addressable storage — shared dependencies across `backend` / `frontend` aren't duplicated on disk
- Strict dependency resolution surfaces "phantom dependency" bugs (using a package that isn't actually declared) that npm/Yarn's flatter `node_modules` can silently hide
- Native workspace filtering makes per-package scripts (`pnpm --filter @kw/backend ...`) simple and explicit
- The default-blocked-postinstall-scripts security posture, correctly configured here via an explicit allowlist, meaningfully reduces supply-chain attack surface
- `--frozen-lockfile` in CI/Docker guarantees reproducible installs matching the committed lockfile

Cons

- Smaller ecosystem familiarity than npm — some tooling/docs still assume npm or Yarn by default
- The symlinked `node_modules` structure occasionally trips up tools that make naive assumptions about package resolution
- The `onlyBuiltDependencies` allowlist requires active maintenance — a new dependency that genuinely needs a postinstall script silently fails to build until someone notices and adds it to the list
- Two-package monorepo tooling here is genuinely simple (`--filter` flags), but would need a real build-orchestration tool (Turborepo/Nx) if the monorepo grew to many more packages

Resources

- <https://pnpm.io/workspaces>
- <https://pnpm.io/npmrc#onlybuiltdependencies>

Docker & Docker Compose

What is it?

Docker packages an application and its runtime into a portable container image; Docker Compose defines and runs multi-container setups from one YAML file.

Purpose in this project

`docker-compose.yml` provides the local development environment (pgvector-enabled Postgres on host port 5433, Redis on 6379, plus optional full backend/frontend containers). Separate `backend/Dockerfile` and `frontend/Dockerfile` build the actual production images used by Render and (optionally) any Docker-based deployment.

Key Features Used

- `pgvector/pgvector:pg16` image — a Postgres 16 image with the `pgvector` extension pre-installed, matching production (Supabase) exactly so pgvector behavior is identical between local dev and prod
- Healthchecks (`pg_isready` , `redis-cli ping`) gating `depends_on: condition: service_healthy` , so the backend container only starts once Postgres/Redis are actually ready to accept connections, not merely "started"
- Host port remapping (`5433:5432`) specifically to avoid colliding with any other local Postgres instance a developer might already be running, while the backend inside the Compose network still connects to the standard `postgres:5432`
- Multi-stage builds in both Dockerfiles (`deps` → `build` → `runtime`) — final images contain only production `node_modules` /build output, not devDependencies or source TypeScript
- Backend Dockerfile copies `backend/prisma` before `pnpm install` specifically because the `postinstall` script (`prisma generate`) needs the schema file present at install time
- Backend runtime entrypoint runs `prisma migrate deploy && node dist/main.js` — migrations apply automatically on every container boot, before the server starts serving traffic
- Frontend Dockerfile bakes `NEXT_PUBLIC_*` variables as build **arguments** (not runtime environment variables) — a Next.js-specific gotcha since `NEXT_PUBLIC_*` values are inlined into the client bundle at build time and can't be changed by setting an env var on the running container afterward

Detailed Description

Both Dockerfiles use the repository root as their build context (not their own subdirectory) specifically to preserve pnpm workspace linking — a package inside `backend/` or `frontend/` that depends on workspace-internal linking needs the whole monorepo's lockfile and structure visible during the build, not just its own folder in isolation.

The local Postgres port choice (5433 instead of the default 5432) is a small but genuinely useful piece of developer-experience design: it means this project's Compose stack can run alongside another local Postgres-based project without a port conflict, while the container-to-container connection inside Compose's own network is unaffected (containers address each other by service name, not host-mapped port).

Alternatives

1. **Podman + Podman Compose** — a daemonless, largely Docker-CLI-compatible alternative; would work as a drop-in for local dev without changing the Compose file meaningfully.
2. **Nix / Nix flakes for reproducible dev environments** — a fundamentally different reproducibility approach (declarative package pinning rather than containerization); steeper learning curve, no container image as an output artifact.

3. **Running Postgres/Redis natively on the host** (no containers) — removes Docker as a prerequisite entirely, at the cost of every developer needing to install and version-match Postgres 16 + pgvector + Redis themselves instead of `docker compose up`.
4. **Kubernetes (even for local dev, e.g. via Kind/Minikube)** — massive overkill for a two-service local dev dependency set; appropriate only if the project's actual production deployment target were Kubernetes, which it isn't (Render + Vercel instead).

Pros

- `pgvector/pgvector:pg16` guarantees local dev and Supabase production run the identical Postgres+extension combination — no "works on my machine, breaks on pgvector version mismatch" class of bug
- Healthcheck-gated startup ordering eliminates a whole class of "backend crashed because Postgres wasn't ready yet" flakiness
- Multi-stage Docker builds keep final image size and attack surface down (no devDependencies, no source TypeScript in the runtime image)
- The `NEXT_PUBLIC_*` -as-build-arg pattern is correctly handled here, avoiding a very common Next.js/Docker misconfiguration where developers expect runtime env vars to affect an already-built client bundle
- Auto-running `prisma migrate deploy` on every boot means deploying a new image always leaves the database schema in sync, with no separate manual migration step to forget

Cons

- Running migrations automatically on every container boot is convenient but means a bad migration ships and applies itself the moment a new image starts — no manual gate before schema changes hit production
- Local development requires Docker to be installed and running at all — a real prerequisite (explicitly listed in the README) that's not needed for Vercel's own frontend-only local `next dev` workflow
- Baking `NEXT_PUBLIC_*` values at build time means changing one of those values requires a full image rebuild, not just a container restart with new env vars
- Two separate Dockerfiles (backend, frontend) plus the Compose file for local dev is three places container/build configuration lives, though each has a distinct, non-overlapping purpose

Resources

- <https://docs.docker.com/>
- <https://docs.docker.com/compose/>
- <https://hub.docker.com/r/pgvector/pgvector>

Testing & Quality Assurance

Jest, ts-jest & Supertest

What is it?

Jest is a JavaScript/TypeScript test runner with built-in assertions, mocking, and coverage reporting; `ts-jest` lets Jest run TypeScript test files directly; Supertest provides a fluent API for issuing HTTP requests against an in-process server and asserting on the response.

Purpose in this project

The backend has two distinct test suites, both Jest-based but configured separately: **unit tests** (`*.spec.ts` files colocated next to the code they test — `auth.service.spec.ts` , `documents.service.spec.ts` , `ai.service.spec.ts` , `search.service.spec.ts`) with all external dependencies (Prisma, Queue, providers) mocked; and one **end-to-end test** (`test/app.e2e-spec.ts`) that boots the real `AppModule` against real Postgres and Redis, using Supertest to drive actual HTTP requests through the full Nest pipeline.

Key Features Used

- `@nestjs/testing` 's `Test.createTestingModule` + `overrideProvider` — used in the e2e suite to swap only the three genuinely external integrations (`AI_PROVIDER` , `EMBEDDING_PROVIDER` , `StorageService`) for deterministic fakes, while everything else (Postgres via Prisma, Redis via BullMQ, the real workers) runs for real
- A deterministic fake embedding provider in the e2e test (`Math.sin` -based, L2-normalized) — chosen specifically so pgvector still receives valid, comparable 384-dimension vectors rather than random noise, letting the semantic-search assertion be meaningful
- `supertest(app.getHttpServer())` request-chaining for every e2e assertion (`.post('/auth/refresh')` , `.get('/documents/:id')` , etc.)
- A hand-written `waitFor()` polling helper (500ms interval, 60s timeout) in the e2e test — necessary because file processing and AI jobs run asynchronously through real BullMQ workers, so the test has to poll for eventual completion rather than asserting on an immediate response
- Two separate Jest configs: the unit config embedded in `backend/package.json` (`rootDir: src` , `testRegex: /\.spec\.ts$/`) and `test/jest-e2e.json` (`testRegex: .e2e-spec.ts$` , a generous 120-second `testTimeout` to accommodate real async worker completion)
- `PrismaService.truncateAll()` — a test-only reset helper (hard-guarded to throw if `NODE_ENV === 'production'`) used to clear database state between e2e runs

Detailed Description

The e2e test's design is the most instructive artifact in the whole test suite: it proves the queue, the worker, the pgvector write, and the semantic read-back path all genuinely work together end to end, by stubbing only the three integrations that are either slow, costly, or nondeterministic to call for real in CI (external object storage, a real LLM, real embeddings) while leaving the actually-interesting infrastructure (Postgres, Redis, BullMQ, the Nest DI graph) completely real. This is a deliberately narrow stubbing boundary — it would have been easier to mock Prisma too, but doing so would defeat the entire purpose of an e2e test, which is to catch exactly the kind of integration bug that unit tests (with everything mocked) cannot.

Unit tests take the opposite, narrower approach appropriately: `documents.service.spec.ts` mocks Prisma and Queue entirely to isolate and assert on pure business logic (e.g. "renaming a document must not enqueue a reindex job, but editing its content must") — a fact that would be expensive and slow to verify against a real database on every test run, and doesn't need to be.

Alternatives

1. **Vitest** — a newer, Vite-native test runner with a largely Jest-compatible API and generally faster execution; would be a plausible modern swap, though NestJS's official testing utilities and most existing Nest tutorials still assume Jest by default.
2. **Playwright / Cypress** (for true browser-driven e2e, as opposed to this project's API-level e2e) — neither is used here; the e2e suite tests the backend's HTTP+worker pipeline directly via Supertest, not the frontend UI in a real browser. A frontend e2e layer using one of these tools is notably absent from this codebase.
3. **Mocha + Chai + Sinon** — an older, more manually-assembled combination (separate test runner, assertion library, and mocking library) versus Jest's all-in-one package; would work but adds configuration surface Jest bundles by default.
4. **testcontainers** (spinning up ephemeral Postgres/Redis containers per test run programmatically) — an alternative to this project's approach of relying on `docker compose up` /CI service containers being already running; would make the e2e suite more self-

contained at the cost of slower/heavier test startup.

Pros

- The unit/e2e split correctly matches "fast, isolated business-logic checks" against "slow, real, cross-system integration checks" to the tool suited for each
- Stubbing only the three genuinely external integrations in the e2e test maximizes real coverage (Postgres, Redis, BullMQ, Prisma, the full Nest DI graph) without needing network access to xAI/OpenAI/Supabase in CI
- `Test.createTestingModule`'s `overrideProvider` makes swapping just those three dependencies clean, without needing a parallel "test version" of the entire `AppModule`
- The e2e suite genuinely proves the hardest-to-get-wrong part of the system (async queue → worker → pgvector write → semantic read-back) actually functions, not just that each piece compiles in isolation
- Deterministic fake embeddings (rather than random) keep the semantic-search e2e assertion meaningful instead of trivially always-passing

Cons

- The e2e suite needs real Postgres and Redis available (via Docker Compose locally, or CI service containers) — it cannot run in complete isolation the way the mocked unit tests can
- A 120-second test timeout, while justified by real async worker completion, makes the e2e suite meaningfully slower to run than the unit suite — a real cost during fast local iteration
- No frontend end-to-end test layer exists in this codebase (no Playwright/Cypress) — UI regressions have no automated test coverage, only the manual "test in a browser" step
- The hand-written `waitFor()` polling helper is a bespoke pattern rather than a library-provided one — reasonable for one test file, but would need generalizing if more async-completion assertions were added elsewhere

Resources

- <https://jestjs.io/docs/getting-started>
- <https://docs.nestjs.com/fundamentals/testing>
- <https://github.com/ladjs/supertest>

Deployment & DevOps

Vercel (frontend hosting)

What is it?

A hosting platform built by the creators of Next.js, offering Git-integrated deployments, edge/serverless functions, and first-class Next.js build support.

Purpose in this project

`vercel.json` configures Vercel to build and deploy only the `@kw/frontend` workspace package out of the monorepo, using pnpm's workspace filtering (`pnpm install --frozen-lockfile --filter @kw/frontend...`, `pnpm --filter @kw/frontend`

build).

Key Features Used

- Monorepo-aware build/install commands scoped via `pnpm --filter` , so Vercel doesn't need to build the unrelated `backend` package
- `outputDirectory: "frontend/.next"` — pointing Vercel at the correct build output location inside the monorepo structure
- `framework: "nextjs"` — opts into Vercel's Next.js-specific build optimizations and conventions automatically
- Git-integrated automatic deployments on push (implied by Vercel's standard model; `"github": {"silent": true}` suppresses Vercel's own PR-comment bot noise)

Detailed Description

Vercel was chosen specifically because it only needs to serve the frontend — a static-ish, client-rendering-heavy Next.js app with no long-lived server-side process requirement (no Socket.IO server, no BullMQ workers to host). This cleanly matches the README's explicit reasoning for why the *backend* is hosted elsewhere: Vercel's serverless/edge function model is a poor fit for a service that needs a persistent WebSocket server and background job workers, but is an excellent fit for a Next.js frontend that doesn't need either.

Alternatives

1. **Netlify** — a comparable Git-integrated static/serverless host with reasonable Next.js support, though historically slightly behind Vercel in Next.js-specific feature parity (since Vercel builds Next.js itself).
2. **Cloudflare Pages** — fast edge-network deployment, increasingly good Next.js support via its adapter, though with some historical rough edges around newer App Router features.
3. **Self-hosted (the frontend's own Dockerfile, on the same host as the backend)** — the frontend's Dockerfile already exists and works standalone (as used for local Docker Compose), so self-hosting is a real, already-supported alternative to Vercel — it would just mean giving up Vercel's zero-config CDN/edge network and Git-integration conveniences.
4. **AWS Amplify** — another Git-integrated frontend hosting platform with Next.js support, generally requiring more manual configuration than Vercel's near-zero-config Next.js experience.

Pros

- Zero-configuration Next.js optimizations (image optimization, edge caching, ISR support) since Vercel builds the framework it also hosts
- Monorepo-scoped build/install commands mean Vercel never wastes time building the unrelated backend package
- Git-integrated deploy previews for every pull request, with the noisy PR-comment bot explicitly silenced in this project's config
- Global CDN distribution for static assets with no separate CDN configuration needed

Cons

- A platform-specific dependency for the frontend — moving off Vercel later means relying on the frontend's own Dockerfile (already present, but a different deployment/ops model) instead
- Serverless/edge function cold starts can matter for API routes hosted on Vercel itself, though this project's frontend doesn't appear to lean on Vercel API routes (all real API calls go to the separately-hosted NestJS backend)
- Free/hobby tier limitations (build minutes, bandwidth) could become a real constraint at higher production traffic, requiring a paid plan
- Vercel's pricing model is usage-based and can become expensive at scale compared to a flat-rate self-hosted container

Resources

- <https://vercel.com/docs>
- <https://vercel.com/docs/monorepos>

Render (backend hosting) & Upstash (managed Redis)

What is it?

Render is a cloud platform hosting long-lived web services (including Docker-based ones) with a declarative `render.yaml` "blueprint" format. Upstash is a serverless-friendly managed Redis provider, notable for offering both a REST API and a real TCP `redis://` endpoint.

Purpose in this project

`render.yaml` declares the backend as a `runtime: docker` web service (not a serverless function), built from `backend/Dockerfile`, with a health check at `/api/health` and `autoDeploy: true`. Upstash provides the `REDIS_URL` (the `redis://` TCP endpoint specifically) that BullMQ connects to in production.

Key Features Used

- Render's Docker-runtime web service type — required because the backend hosts a persistent Socket.IO server and in-process BullMQ workers, neither of which fits a serverless/functions-as-a-service execution model
- `healthCheckPath: /api/health` — Render polls this endpoint (the same `HealthController` route that itself round-trips a `SELECT 1` through Prisma) to determine whether a deployment is healthy before routing traffic to it
- `generateValue: true` for `JWT_SECRET` / `JWT_REFRESH_SECRET` — Render auto-generates cryptographically random values for these at first deploy, rather than requiring a human to invent and paste in secrets
- `sync: false` for every genuinely sensitive/environment-specific variable (database URLs, API keys, GitHub OAuth credentials) — meaning Render prompts for these to be set manually in its dashboard rather than committing them anywhere
- Upstash's `redis://` TCP endpoint specifically (not its REST API) — explicitly called out in both the README and `.env.example` because BullMQ needs Redis's blocking commands (`BLPOP` etc.), which only the TCP protocol endpoint supports; the REST endpoint cannot serve BullMQ's needs at all

Detailed Description

The Render/Vercel split is a direct consequence of the backend's own architecture: because the Socket.IO gateway and BullMQ workers both require a long-lived, stateful Node process (an in-memory Socket.IO room table, an in-process-loaded embedding model, active Redis blocking-command connections), a serverless deployment model is fundamentally incompatible with this backend — hence a platform like Render (or an EC2 instance, or a VPS) is required, rather than trying to force the same serverless-first hosting model used for the frontend.

The Upstash TCP-vs-REST distinction is a genuinely easy mistake to make (Upstash prominently offers both, and REST is often the more heavily marketed option for serverless use cases) — this project's explicit documentation of the correct choice in two separate places (README and `.env.example`) suggests it was a real point of confusion worth guarding against for future maintainers.

Alternatives

1. **Fly.io** — another long-lived-container-friendly platform with a comparable Docker-deploy model; a reasonable direct substitute for Render's role here.
2. **Railway** — similar developer-experience-focused container hosting; comparable trade-offs to Render for this use case.
3. **A raw VPS (DigitalOcean, Linode, a bare EC2 instance) + Docker Compose** — full control, the same Compose file already used for local dev could plausibly be adapted, at the cost of self-managing OS updates, TLS certificates, and process supervision that Render handles automatically.
4. **AWS ElastiCache (managed Redis) instead of Upstash** — a comparable managed-Redis alternative, likely chosen against here in favor of Upstash's simpler serverless-friendly pricing/setup, though ElastiCache's TCP endpoint would satisfy BullMQ's requirements equally well.

Pros

- Render's Docker-runtime web service correctly matches the backend's actual requirement (a persistent process), avoiding the mismatch a serverless platform would create
- Auto-generated JWT secrets on first deploy remove a manual "did I use a strong enough secret" step from the deployment process
- Health-check-gated deploys mean a broken new deployment (failing its `/api/health` check) won't get traffic routed to it
- Upstash's TCP endpoint gives BullMQ everything it needs (blocking commands) while still being a fully managed, low-maintenance Redis instance

Cons

- Render's Docker-based deploys are typically slower to build/deploy than a purpose-built serverless platform's near-instant function deploys
- A single Render web service instance (per the `plan: starter` tier) is a single point of failure for the Socket.IO server and all three BullMQ queues unless upgraded to a higher, multi-instance plan (which would then need a Socket.IO Redis adapter to coordinate rooms across instances — not currently configured)
- Upstash's serverless-Redis pricing model (often request-count-based) can behave differently cost-wise than a traditional fixed-size Redis instance under BullMQ's continuous polling/blocking connection pattern
- Two separate hosting dashboards (Render for backend, Vercel for frontend, Upstash for Redis, Supabase for database/storage) means deployment/ops knowledge is spread across four different platforms' consoles and conventions

Resources

- <https://render.com/docs/blueprint-spec>
- <https://render.com/docs/deploy-an-image>
- <https://upstash.com/docs/redis/overall/getstarted>

GitHub Actions (CI/CD)

What is it?

GitHub's built-in CI/CD platform — workflows defined as YAML files in `.github/workflows/`, running on GitHub-hosted (or self-hosted) runners triggered by repository events.

Purpose in this project

`.github/workflows/ci.yml` runs on every push/PR to `main`, executing four sequential jobs: **backend** (lint, unit test, build, e2e test against real Postgres/Redis service containers), **frontend** (typecheck, build), **docker** (build both Dockerfiles, cache-only, no push — a build-correctness check, not a publish step), and **deploy** (a single `curl` to Render's deploy webhook, gated to only run on a direct push to `main`, not on pull requests).

Key Features Used

- **Service containers** (`postgres:` using the `pgvector/pgvector:pg16` image, `redis:`) attached directly to the `backend` job — GitHub Actions spins these up as sibling containers with health checks before the job's own steps run, giving the e2e test suite real infrastructure without needing Docker Compose in CI
- `concurrency: {group, cancel-in-progress: true}` — automatically cancels a previous in-progress run for the same branch/PR when a new commit supersedes it, saving CI minutes
- Job-level `needs:` dependencies (`docker` needs `[backend, frontend]` ; `deploy` needs `[docker]`) — enforcing that a broken lint/test/build never reaches the Docker-build-verification or deploy stages
- Environment-scoped secrets/CI-only fake credentials (`JWT_SECRET: ci-access-secret-value-...` , `GITHUB_CLIENT_ID: ci-client-id`) — values that satisfy the boot-time `env.validation.ts` checks without being real secrets, since the e2e suite stubs the actual external integrations anyway
- `docker/build-push-action@v6` with `push: false` and GitHub Actions cache (`cache-from/to: type=gha`) — verifies both Dockerfiles actually build successfully on every PR without publishing an image anywhere, catching a broken Dockerfile before it would fail in an actual deploy
- A conditional final deploy step (`if: github.ref == 'refs/heads/main' && github.event_name == 'push'`) that only fires the Render deploy webhook on a real merge to `main`, and further guards on the webhook secret actually being configured (`if: ${ secrets.RENDER_DEPLOY_HOOK_URL != '' }`)

Detailed Description

This pipeline embodies a "test the real thing" philosophy consistent with the backend's own e2e test design: rather than mocking Postgres/Redis in CI too, it spins up actual service containers (using the very same `pgvector/pgvector:pg16` image as local Docker Compose) so the e2e suite's assertions about real database/queue behavior are actually meaningful in CI, not just locally. The Docker-build-verification job similarly exists purely to catch a broken Dockerfile as an explicit, fast-failing CI check — long before a bad Dockerfile would otherwise be discovered mid-deploy on Render.

The frontend deploy is conspicuously **not** triggered from this workflow at all — a comment in the workflow file makes this explicit: Vercel's own Git integration handles frontend deployment independently, so GitHub Actions' only deployment responsibility is the one piece Vercel doesn't own, the backend.

Alternatives

1. **GitLab CI / CircleCI / Jenkins** — comparable general-purpose CI platforms; the specific patterns here (service containers, job dependencies, conditional deploy steps) all have direct equivalents in each, making this a largely interchangeable choice tied mostly to where the code is hosted (GitHub, in this case).
2. **A single Render/Vercel-native CI step** (skip GitHub Actions, rely purely on each platform's own build pipeline) — would lose the unified lint/test/e2e/docker-build gate that currently runs *before* either platform's own deploy trigger fires, and would lose the cross-cutting `needs:` dependency graph across both frontend and backend checks.
3. **Buildkite / self-hosted runners** — useful if CI needed more compute/customization than GitHub-hosted runners provide; this project's CI needs (a couple of service containers, a straightforward test/build/deploy pipeline) don't obviously require it.
4. **Skipping CI entirely, deploying straight from local machines** — the simplest possible approach, but loses every safety net this pipeline provides (no e2e run gating a bad merge, no build verification before deploy).

Pros

- Real Postgres/Redis service containers in CI mean the e2e suite's guarantees hold in the actual CI environment, not just on a developer's machine
- The `needs:` dependency graph ensures a failing lint/test/build never reaches the (comparatively expensive) Docker build or deploy stages
- `concurrency: cancel-in-progress` avoids wasting CI minutes on superseded commits
- The Docker-build-verification job catches a broken Dockerfile as an explicit CI failure rather than discovering it during an actual Render deploy
- Deploy is conditionally gated (branch + event type + secret presence) so it can never accidentally fire from a fork's pull request or an unconfigured environment

Cons

- CI-only fake secrets satisfying `env.validation.ts` (rather than genuinely testing what happens with a truly missing/invalid env var) means the boot-time validation logic itself isn't exercised for its actual failure path in CI
- The deploy job only handles the backend — frontend deploy correctness is entirely dependent on Vercel's own separate pipeline being configured correctly, invisible to this workflow file
- No rollback automation is visible in this pipeline — a bad deploy to Render would need to be handled manually (e.g. via Render's dashboard) rather than an automated rollback step
- GitHub Actions minutes/service-container usage adds up on a busy repository, though this project's CI footprint (two short jobs plus a Docker build) is modest

Resources

- <https://docs.github.com/actions>
- <https://docs.github.com/actions/using-containerized-services/about-service-containers>

Other Utilities & Dev Tooling (quick reference)

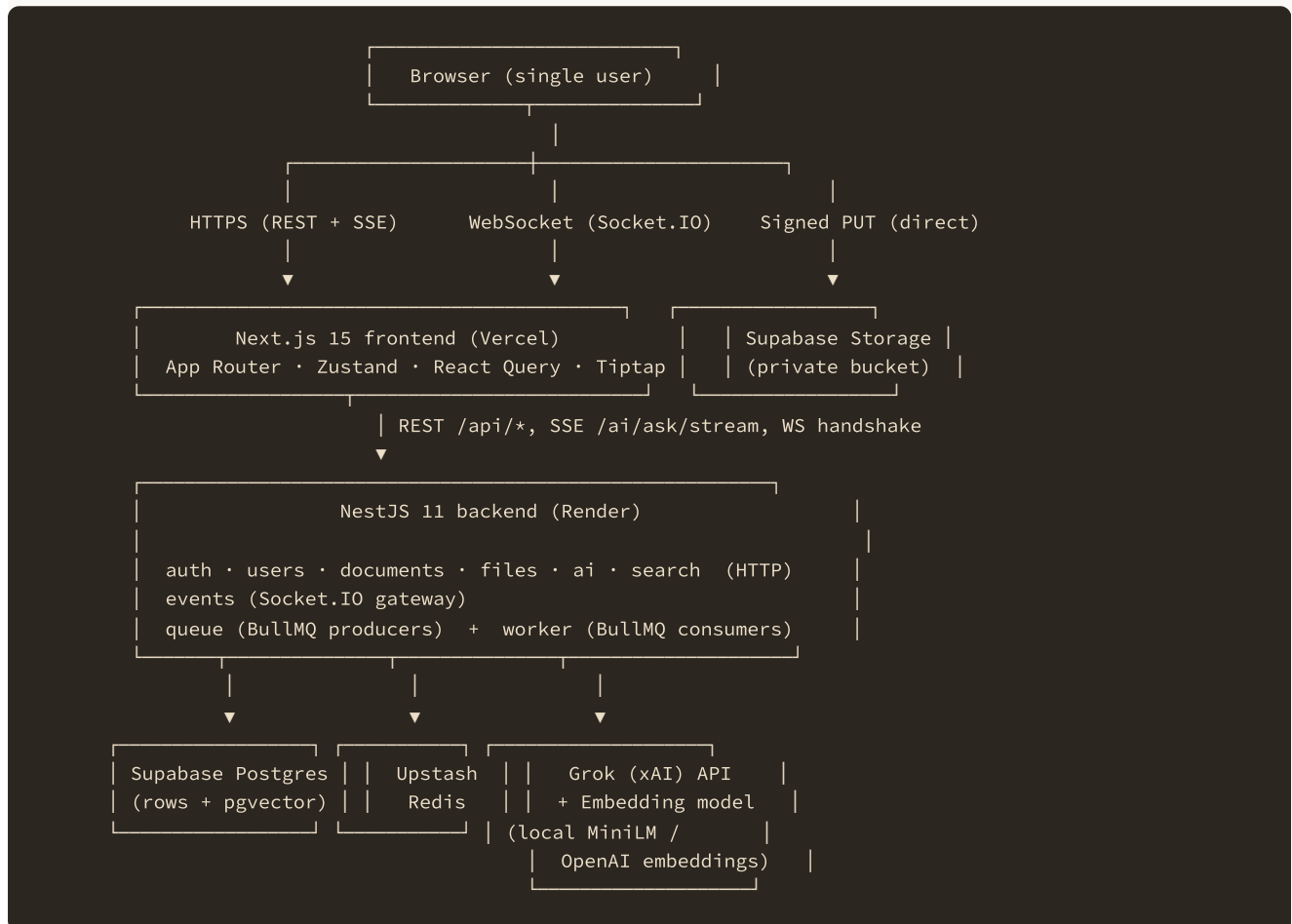
Smaller, single-purpose dependencies that don't need a full dossier but are worth knowing what they do and why they're there:

PACKAGE	SIDE	PURPOSE	WHY THIS ONE
<code>helmet</code>	Backend	Sets security-related HTTP headers (CSP, X-Frame-Options, etc.)	Standard, minimal-config Express hardening; CSP/COEP explicitly disabled here since the API is JSON-only on a separate origin from the frontend
<code>cookie-parser</code>	Backend	Parses the <code>Cookie</code> header into <code>req.cookies</code>	Needed to read the <code>httpOnly</code> refresh-token cookie in <code>AuthController</code>
<code>reflect-metadata</code>	Backend	Polyfills decorator metadata reflection	Hard requirement for NestJS's decorator-based DI to function at all
<code>rxjs</code>	Backend	Reactive-stream primitives (<code>Observable</code> , <code>operators</code>)	Pulled in as a NestJS peer dependency; used directly in <code>LoggingInterceptor</code> 's <code>tap()</code> operator
<code>helmet</code> -adjacent CORS config	Backend	<code>enableCors</code> in <code>main.ts</code>	Locked to <code>frontendUrl</code> with <code>credentials: true</code> , matching the cookie-based refresh-token flow
<code>date-fns</code>	Frontend	Lightweight date formatting/relative-time utilities	Used for "3 minutes ago"-style relative timestamps; chosen over Moment.js for its smaller, tree-shakeable, immutable-function design

PACKAGE	SIDE	PURPOSE	WHY THIS ONE
<code>lucide-react</code>	Frontend	Icon component set	Consistent, tree-shakeable SVG icon set matching the shadcn-style component aesthetic
<code>sonner</code>	Frontend	Toast notification component	Used by every mutation hook (<code>toast.success</code> / <code>toast.error</code>) for user-facing feedback on save/delete/upload actions
<code>next-themes</code>	Frontend	Dark/light/system theme switching for Next.js	Handles the class-based theme toggle and SSR-safe hydration (topbar waits for client mount before rendering the sun/moon icon to avoid a hydration mismatch)
<code>react-markdown</code> + <code>remark-gfm</code>	Frontend	Renders markdown strings as React elements, with GitHub-Flavored-Markdown extensions (tables, strikethrough, task lists)	Used to render the AI assistant's markdown-formatted chat/generate output safely, without <code>dangerouslySetInnerHTML</code>
<code>ESLint</code> + <code>typescript-eslint</code>	Both	Static linting	Backend: flat config, <code>typescript-eslint</code> recommended rules + Prettier-conflict-disabling. Frontend: <code>eslint-config-next</code> . Both run with <code>--max-warnings 0</code> in CI/lint scripts, so warnings fail the build, not just errors
<code>Prettier</code>	Backend	Code formatting	Single quotes, trailing commas, 100-character line width, configured in <code>.prettierrc</code>

Part 2 — Architecture & System Design

High-Level Architecture Overview



Main components and their relationships

- The **frontend** never talks to Postgres, Redis, or the AI providers directly — every one of those goes through the NestJS backend. The one deliberate exception is **file bytes**, which the browser PUTs straight to Supabase Storage using a signed URL the backend minted.
- The **backend** is really three cooperating processes-in-one: an HTTP API (controllers/services), a Socket.IO server (the `events` module), and a set of BullMQ workers (the `worker` module) — all three currently run inside the same Node process, sharing one Nest DI container, but are logically independent enough that the worker portion could be split into its own deployment later (a comment in `worker.module.ts` says as much).
- Postgres** is the single source of truth for both relational data (users, documents, files, AI requests) and vector embeddings (`document_chunks.embedding`) — there is no separate vector database.
- Redis** exists solely to back BullMQ; nothing else in the app touches it directly.
- Grok** and the **embedding provider** are both reached only from inside the `ai` module, behind the `AIProvider` / `EmbeddingProvider` interfaces — no other module calls an LLM or embedding API directly.

External integrations

INTEGRATION	REACHED FROM	PURPOSE
GitHub OAuth	<code>auth</code> module (Passport strategy)	Login/identity

INTEGRATION	REACHED FROM	PURPOSE
xAI Grok API	<code>ai</code> module (<code>GrokProvider</code>)	Summarize, rewrite, generate, chat
Local MiniLM model (in-process) or OpenAI embeddings API	<code>ai</code> module (<code>LocalEmbeddingProvider</code> / <code>OpenAIEmbeddingProvider</code>)	Turning text into vectors for RAG/semantic search
Supabase Storage	<code>files</code> module (<code>StorageService</code>) — mints signed URLs; browser uses them directly	Raw uploaded file bytes
Supabase Postgres	<code>prisma</code> module, every feature module	All relational + vector data
Upstash Redis	<code>queue</code> / <code>worker</code> modules	BullMQ job queue backend
Sentry	Global exception filter (backend), instrumentation hooks (frontend)	Error tracking

Data Flow

Entry points — data enters the system through exactly four doors:

1. **REST requests** from the frontend (create/update a document, request a summary, search)
2. **A direct signed PUT** from the browser to Supabase Storage (uploaded file bytes — never touch the backend)
3. **The GitHub OAuth callback** (identity data)
4. **BullMQ job payloads** — internal, produced by the backend itself, consumed by its own workers

Transformation pipeline (the RAG path, the most complex flow in the system):

```

raw bytes (PDF/DOCX/TXT/MD)
| (FileProcessor: download from Supabase Storage)
▼
extracted plain text (text-extractor.service.ts: pdf-parse / mammoth / utf8 decode + normalise())
|
▼
Document row created/updated (plainText column) + Tiptap JSON (plainTextToTiptap, so it's editable)
|
▼
chunks (chunking.util.ts: paragraph-aware sliding window, ~1200 chars, 200-char overlap)
|
▼
embeddings (EmbeddingProvider: 384-dim vectors, batched 32 at a time)
|
▼
document_chunks rows (raw SQL INSERT, embedding cast to ::vector)
|
▼
available for: semantic search (cosine similarity, HNSW index) AND RAG retrieval (same query, feeding chat context)

```

Storage mechanisms — three distinct stores, each holding a different shape of data:

- **Postgres relational tables** — `User` , `Document` , `File` , `AIRequest` , `RefreshToken` (structured, queried via Prisma)
- **Postgres** `document_chunks.embedding` (**pgvector**) and `documents.search_vector` (**tsvector**) — the same database, but accessed only via raw SQL
- **Supabase Storage** — the actual file bytes, referenced from Postgres only by a `storagePath` string, never stored in the database itself

Exit points — what leaves the system, and in what shape:

- JSON REST responses (documents, search results, dashboard stats)
- Server-Sent Events (`text/event-stream`) for streaming chat answers — a discriminated-union frame protocol (`sources` → `token` * → `done` / `error`)
- Socket.IO events (`job:progress` , `resource:updated`) — realtime, best-effort, never carrying the actual answer/result, only status
- Signed download URLs (time-boxed, pointing directly at Supabase Storage, so the actual bytes never round-trip through the backend on download either)

Control Flow

How different parts communicate

- Frontend ⇌ Backend: REST/JSON (React Query), raw fetch + hand-parsed SSE (streaming chat only), Socket.IO (progress notifications only)
- Feature module ⇌ Feature module (backend): plain Nest dependency injection — e.g. `SearchModule` imports `AiModule` to inject `IndexingService` ; `DocumentsService` injects `QueueService` to enqueue reindexing
- Backend ⇌ Workers: never a direct function call — always a BullMQ job. A controller/service that wants background work done calls `QueueService.enqueueX(...)` ; it has no reference to the processor that will eventually run it
- Workers ⇌ Frontend: via the Socket.IO gateway, injected into every processor through the `@Global()` `EventsModule`

Request/response patterns

- Standard synchronous REST for anything that completes quickly (CRUD, auth, search)
- **Queued async** for anything that's slow or costs an LLM/embedding call except streaming chat: the HTTP handler creates a DB row (`status: PENDING`) and an idempotent BullMQ job, then returns immediately with the row's ID — the client polls `GET /ai/requests/:id` (or a files-list poll) until the status changes, with Socket.IO providing a faster, best-effort progress signal in between
- **Streaming-in-request** for chat only: the one deliberate exception, because the SSE response *is* the answer, so queuing would defeat the purpose (an `AIRequest` history row is still written afterward, just not used to gate the response)

Event-driven flows

- BullMQ job completion/failure events (`@OnWorkerEvent('failed')`) drive both the database status update and the Socket.IO emission — one event, two downstream effects
- The frontend's `resource:updated` socket event doesn't carry data itself — it's a cache-invalidation signal, telling React Query (via `router.refresh()` /query invalidation) to refetch the affected resource from the REST API, keeping the socket payload minimal and the REST API the single source of truth for actual data

Error handling flows

- Backend: every thrown error funnels through the global `AllExceptionsFilter` , which normalizes HTTP exceptions, known Prisma error codes, and unexpected errors into one JSON shape, and forwards 5xx errors to Sentry
- AI provider errors are always wrapped into `ServiceUnavailableException` (503) before reaching the filter, so callers never see a raw provider-specific error shape
- BullMQ job failures retry automatically (3 attempts, exponential backoff); only after `attemptsMade` reaches the configured limit does a processor mark the underlying DB row `FAILED`
- Frontend: `ApiError` carries the backend's normalized `{status, details}` shape; `lib/api.ts` 's single-flight refresh logic handles the specific 401-then-retry-once case; every mutation surfaces failures via `sonner` toasts

Async vs. sync

OPERATION	SYNC OR ASYNC	WHY
CRUD (documents, files metadata, users, search)	Sync REST	Fast, no external API call
Summarize / Rewrite / Generate	Async (BullMQ)	Calls Grok, can take seconds
File processing (extract → chunk → embed)	Async (BullMQ)	CPU-bound + I/O-bound, potentially large files
Document re-indexing on edit	Async (BullMQ), debounced 15s	Same cost as file processing, plus needs burst-collapsing for autosave
Chat (<code>/ai/ask/stream</code>)	Sync , streamed	The SSE response is the answer itself
File upload (bytes)	Sync from the browser's perspective, but direct-to-Supabase, bypassing the backend entirely	Avoids buffering large payloads through the API server

Deployment Architecture

- **Frontend:** deployed to **Vercel**, Git-integrated, building only the `@kw/frontend` workspace (`vercel.json`)
- **Backend:** deployed to **Render** as a Docker-runtime long-lived web service (`render.yaml`) — chosen specifically because it hosts the Socket.IO server and BullMQ workers, which need a persistent process, not a serverless function
- **Database:** **Supabase Postgres** with `pgvector` enabled; a pooled connection (port 6543, transaction mode) for the running app, a direct connection (port 5432) reserved for Prisma migrations
- **Redis:** **Upstash**, accessed via its `rediss://` TCP endpoint (not its REST API), because BullMQ needs Redis's blocking commands
- **File storage:** **Supabase Storage**, a private bucket, accessed only via signed URLs
- **Error tracking:** **Sentry**, optional (no-op without a DSN) on both sides
- **Configuration management:** environment variables everywhere, validated at boot on the backend (`env.validation.ts` , fails fast on missing/malformed values) and inlined at build time on the frontend (`NEXT_PUBLIC_*` variables, baked into the Docker image as build args, or provided directly as Vercel project environment variables)
- **Local development:** Docker Compose runs just Postgres (pgvector image) and Redis, matching production's engine/extension versions exactly; the actual Next.js and NestJS processes run natively via `pnpm dev` for fast iteration, with full container versions of both also defined in Compose for a closer-to-prod local test if needed

Scaling considerations — not heavily built out yet, but the architecture leaves room:

- The backend currently runs as a single Render instance; scaling to multiple instances would require a Socket.IO Redis adapter (to synchronize per-user rooms across instances) that isn't currently configured
- BullMQ workers currently share the same process as the API server; `worker.module.ts` is already a separate Nest module specifically so it *could* be extracted into its own deployment later without restructuring the rest of the app
- The embedding provider choice (local vs. OpenAI) is itself a scaling lever — local embedding is CPU-bound and competes with the API server for a single core in the current deployment; switching to the OpenAI provider offloads that work entirely, at a per-call cost

Integration Points

- **External APIs called:** GitHub OAuth (`github.com` / `api.github.com` , via Passport), xAI Grok's chat completions API, optionally OpenAI's embeddings API, Supabase's Storage REST API
- **Third-party services:** Sentry (error tracking, optional), Upstash (managed Redis)
- **Database connections:** two distinct Postgres connection strings (`DATABASE_URL` pooled, `DIRECT_URL` direct) managed entirely through Prisma; no other module connects to Postgres independently

- **File system interactions:** essentially none in application logic — uploaded files never touch the backend's local disk; text extraction operates on in-memory buffers downloaded straight from Supabase Storage and discarded after processing

Part 3 — File Structure & Implementation

Files are grouped the way you'd actually want to learn them — starting from the backend's bootstrap and data layer (nothing else makes sense without these), through each feature module in the order the request pipeline touches them, then the frontend, then root-level infrastructure. Within each group, straightforward files (trivial modules, DTOs, UI primitives) are covered in a compact table rather than a full dossier — the "why" is usually one line for these, and repeating the full template five times for five near-identical DTOs would only dilute the substantive material.

Group 1 — Backend Bootstrap & Global Wiring

Why these belong together: nothing in the backend runs without these files. They define what "the app" even is before any feature module does anything.

Order to understand them: `instrument.ts` → `main.ts` → `app.module.ts` → `config/*` → `common/*` → `health.controller.ts`.

File: `backend/src/instrument.ts`

Type: Bootstrap / observability setup **Purpose:** Initializes Sentry error tracking before anything else in the process runs. **Why Created:** Sentry needs to patch Node's runtime (HTTP, error handling) before the rest of the app constructs any objects it would need to instrument — hence it's imported as the very first line of `main.ts`. **Dependencies:** `@sentry/node`; reads `SENTRY_DSN` from `process.env` directly (not via `ConfigService`, since `ConfigModule` hasn't initialized yet at this point in the boot sequence). **Key Concepts:** No-op guard (`if (!dsn) return`) — the entire file does nothing if `SENTRY_DSN` is unset, which is what makes Sentry a fully optional dependency for local development. **Detailed Explanation:** `Sentry.init` is called with `tracesSampleRate` set to 0.1 in production and 0 in development, so performance tracing only samples a fraction of production transactions and is entirely disabled locally. Because this runs before `main.ts`'s own imports resolve, it can capture errors that occur even during the rest of the bootstrap process. **How To Use:** Never called directly elsewhere — its only job is to execute its side effects on import. **Important Notes:** The direct `process.env` read (rather than `ConfigService`) is a necessary exception to the rest of the codebase's config-access convention, purely due to load-order constraints.

File: `backend/src/main.ts`

Type: Application endpoint **Purpose:** Boots the Nest application, applies global middleware/pipes, and starts the HTTP server. **Why Created:** Every Nest app needs exactly one bootstrap function; this is it. **Dependencies:** `./instrument` (imported first, for its side effect), `@nestjs/core` (`NestFactory`), `helmet`, `cookie-parser`, `AppModule`, `PrismaService` (for shutdown hooks). **Key Concepts:** Global API prefix (`api`), CORS locked to `frontendUrl` with `credentials: true`, a global `ValidationPipe`, Prisma shutdown-hook wiring. **Detailed Explanation:** The function creates the Nest app from `AppModule`, sets the `api` global prefix (so every route is `/api/*` without each controller repeating it), applies `cookie-parser()` (needed to read the refresh-token cookie), `helmet()` (with CSP/COEP disabled — deliberately, since this is a pure JSON API on a separate origin from the frontend, and a restrictive CSP is a browser-page concern, not an API-server one), and `enableCors` scoped exactly to the configured frontend origin with credentials allowed (required for the `httpOnly` cookie to be sent cross-origin between Vercel and Render). A global `ValidationPipe` is installed with `whitelist`, `forbidNonWhitelisted`, and `transform` — the single point where every DTO's validation rules actually take effect. Finally it calls `app.get(PrismaService).enableShutdownHooks(app)` so the process shuts down cleanly, and listens on `0.0.0.0:PORT` (not `localhost`, which matters for Docker container networking). **Key Code Sections Explained:** The ordering matters — `instrument.ts` must be the first import; `cookie-parser` / `helmet` / CORS must be applied

before any route can be hit; the `ValidationPipe` must be global rather than per-controller for the "DTOs are the only way in" guarantee to hold everywhere. **How To Use:** Not imported anywhere — it's the process entrypoint (`node dist/main.js` in production, `nest start --watch` in development). **Important Notes:** Listening on `0.0.0.0` rather than the default is essential for the app to be reachable at all inside a Docker container.

File: `backend/src/app.module.ts`

Type: Root module **Purpose:** Declares every feature module the backend has, and registers the four global cross-cutting providers (guard, filter, interceptor, throttler). **Why Created:** Nest requires exactly one root module as the entry point of its dependency-injection graph. **Dependencies:** Every feature module (`AuthModule` , `UsersModule` , `DocumentsModule` , `FilesModule` , `AIModule` , `SearchModule` , `QueueModule` , `WorkerModule` , `EventsModule` , `PrismaModule`), `@nestjs/config` , `@nestjs/throttler` . **Key Concepts:** `ConfigModule.forRoot({isGlobal: true, load: [configuration], validate: validateEnv})` ; `ThrottlerModule.forRoot` with a 120 req/min global baseline; `APP_GUARD: JwtAuthGuard` , `APP_GUARD: ThrottlerGuard` , `APP_FILTER: AllExceptionsFilter` , `APP_INTERCEPTOR: LoggingInterceptor` . **Detailed Explanation:** This is the one file that makes the "default-deny auth" design decision real: `JwtAuthGuard` is registered as a global `APP_GUARD` , meaning any newly-added controller in the future is authenticated by default unless it explicitly opts out with `@Public()` . The module also directly registers `HealthController` , since a health-check endpoint doesn't belong to any particular feature domain. **Key Code Sections Explained:** The order of provider registration (guard before filter before interceptor, as Nest applies them) determines the actual request-processing pipeline: auth check → rate-limit check → route handler (wrapped by the logging interceptor) → any thrown error normalized by the filter. **How To Use:** Passed once to `NestFactory.create(AppModule)` in `main.ts` ; never imported elsewhere. **Important Notes:** Every new backend feature module must be added to this file's `imports` array to actually take effect — a module that exists as a file but isn't imported here is dead code.

File: `backend/src/health.controller.ts`

Type: Controller (root-level, not a feature module) **Purpose:** Exposes `GET /api/health` , proving the process is up and can actually reach the database. **Why Created:** Render's deployment health check needs a real endpoint to poll — an endpoint that merely returns `200 OK` without touching the database wouldn't catch a "server is up but database is unreachable" failure mode. **Dependencies:** `PrismaService` , the `@Public()` decorator. **Key Concepts:** `@Public()` — this route must be reachable without a valid JWT, since Render's health checker has no access token. **Detailed Explanation:** The handler runs `SELECT 1` via Prisma and returns `{status, uptime, timestamp}` , so a failing database connection surfaces as a failing health check (and Render withholds traffic from the new deployment) rather than the deployment silently succeeding while the app is actually broken. **How To Use:** Called only by Render's health-check poller and, incidentally, anyone manually checking `GET /api/health` . **Important Notes:** Must stay public — if `@Public()` were ever accidentally removed, Render's health checker (which sends no auth token) would start failing deploys.

Config & Common files (table):

FILE	TYPE	PURPOSE
<code>config/configuration.ts</code>	Config	Typed namespaced config factory (<code>jwt</code> , <code>github</code> , <code>cookie</code> , <code>redis</code> , <code>ai</code> , <code>embedding</code> , <code>supabase</code> , <code>sentry</code>) reading <code>process.env</code> exactly once at boot, injected everywhere via <code>ConfigService</code> — no service ever reads <code>process.env</code> directly except <code>instrument.ts</code> / <code>events.gateway.ts</code> (documented load-order exceptions)
<code>config/env.validation.ts</code>	Config	<code>class-validator</code> -based <code>EnvironmentVariables</code> class, run via <code>validateSync</code> in <code>ConfigModule.forRoot</code> ; fails the process at boot (not on first request) if required vars (<code>DATABASE_URL</code> , both JWT secrets ≥32 chars, GitHub credentials, <code>REDIS_URL</code>) are missing or malformed
<code>common/decorators/current-user.decorator.ts</code>	Decorator	<code>createParamDecorator</code> reading <code>request.user</code> (set by <code>JwtStrategy.validate</code>); optional key param projects a single field, e.g. <code>@CurrentUser('id')</code>

FILE	TYPE	PURPOSE
<code>common/decorators/public.decorator.ts</code>	Decorator	<code>SetMetadata(IS_PUBLIC_KEY, true)</code> — the escape hatch from the global <code>JwtAuthGuard</code> 's default-deny posture
<code>common/dto/pagination.dto.ts</code>	DTO	<code>skip / take</code> (default 20, max 100) + optional <code>q</code> ; exports the generic <code>Paginated<T></code> response shape used by every list endpoint
<code>common/filters/all-exceptions.filter.ts</code>	Filter	Global <code>@Catch()</code> normalizing every error (HTTP exceptions, known Prisma codes <code>P2002</code> / <code>P2025</code> / <code>P2003</code> , unexpected errors) into one JSON shape; forwards 5xx to Sentry
<code>common/interceptors/logging.interceptor.ts</code>	Interceptor	Logs <code>METHOD url duration_ms</code> for every HTTP request via an RxJS <code>tap()</code>
<code>common/types/jwt-payload.ts</code>	Types	<code>JwtPayload</code> , <code>AuthenticatedUser</code> , <code>TokenPair</code> — shared types used across the auth flow

Group 2 — Data Layer (Prisma)

Why these belong together: the schema is the shape every other module's data ultimately conforms to; nothing else can be fully understood without it.

Order to understand them: `schema.prisma` → `migration.sql` → `prisma.service.ts` → `prisma.module.ts` .

File: `backend/prisma/schema.prisma`

Type: Schema definition **Purpose:** Declares every table, column, relation, and index Prisma manages — plus two columns (`vector(384)` , `tsvector`) it can only acknowledge, not create. **Why Created:** The single source of truth for the relational data model; `prisma generate` reads this to produce the typed client every service imports. **Dependencies:** None (it's the root of the data model); everything else depends on it. **Key Concepts:** `Unsupported("vector(384)")` and `Unsupported("tsvector")` column types — Prisma's escape hatch for column types its own DSL can't express, letting the schema stay aware of columns whose actual DDL lives in raw migration SQL instead.

Full data model:

MODEL	KEY FIELDS	RELATIONS	NOTABLE INDEXES
User	<code>id</code> (cuid), <code>githubId</code> (unique), <code>email</code> (unique, nullable), <code>name</code> , <code>avatar</code> , <code>preferences</code> (Json, default <code>{}</code>), timestamps	1:N → documents, files, aiRequests, refreshTokens	—
RefreshToken	<code>id</code> , <code>userId</code> , <code>tokenHash</code> (unique, SHA-256 of the actual token — the raw token is never persisted), <code>expiresAt</code> , <code>revokedAt?</code> , <code>replacedById?</code> (chains rotations), <code>userAgent?</code> , <code>ip?</code>	N:1 → User (cascade delete)	<code>[userId]</code> , <code>[expiresAt]</code>
Document	<code>id</code> , <code>title</code> (default "Untitled"), <code>content</code> (Json, Tiptap document), <code>plainText</code> (flattened text, feeds FTS/chunking/AI), <code>summary</code> (Json?: {summary, keyPoints[], actionItems[] }), <code>icon?</code> , <code>isArchived</code> , <code>searchVector</code> (<code>Unsupported("tsvector")</code> , DB-generated)	N:1 → User (cascade); 1:N → chunks; 1:1 → sourceFile; 1:N → aiRequests	<code>[userId, updatedAt]</code> , GIN on <code>searchVector</code>

MODEL	KEY FIELDS	RELATIONS	NOTABLE INDEXES
File	<code>id</code> , <code>filename</code> , <code>storagePath</code> , <code>url</code> (default <code>""</code>), <code>type</code> , <code>size</code> , <code>status</code> (enum, default <code>PENDING</code>), <code>error?</code> , <code>documentId?</code> (unique, nullable)	N:1 → User (cascade); 1:1 → Document (<code>onDelete: SetNull</code>); 1:N → chunks	<code>[userId, createdAt]</code>
DocumentChunk	<code>id</code> , <code>documentId</code> , <code>fileId?</code> , <code>content</code> , <code>chunkIndex</code> , <code>embedding</code> (<code>Unsupported("vector(384)")</code>), <code>source</code> (enum <code>DOCUMENT / FILE</code>), <code>metadata</code> (Json), <code>tokenCount?</code>	N:1 → Document (cascade); N:1 → File (cascade)	unique <code>[documentId, chunkIndex]</code> , <code>[documentId]</code> , <code>[fileId]</code> ; HNSW vector index exists only in raw SQL
AIRequest	<code>id</code> , <code>userId</code> , <code>documentId?</code> , <code>type</code> (enum <code>SUMMARIZE / REWRITE / GENERATE / CHAT</code>), <code>status</code> (enum <code>PENDING / PROCESSING / COMPLETED / FAILED</code>), <code>input</code> , <code>output?</code> , <code>result</code> (Json?), <code>error?</code> , <code>jobId?</code> , <code>tokensUsed?</code> , <code>model?</code>	N:1 → User (cascade); N:1 → Document (<code>SetNull</code>)	<code>[userId, createdAt]</code>

Enums: `AIRequestType`, `AIRequestStatus`, `FileStatus` (`PENDING / UPLOADING / PROCESSING / COMPLETED / FAILED`), `ChunkSource` (`DOCUMENT / FILE`).

Detailed Explanation: Cascade rules do real work here — deleting a `Document` cascades to its `DocumentChunk` rows automatically at the database level, so `DocumentsService.remove()` doesn't need to manually clean up chunks. `File.documentId` being nullable with `onDelete: SetNull` reflects that a `File` 's parent `Document` can be deleted independently without orphaning the file record into an invalid state — it just becomes a file no longer linked to any document.

How To Use: Edited directly to change the data model, then `prisma migrate dev` (locally) or `prisma migrate deploy` (in Docker/production) applies the change; `prisma generate` (run automatically via `postinstall`) regenerates the typed client every service imports from `@prisma/client` . **Important Notes:** `directUrl` (port 5432) is used only for running migrations against Supabase's pooler-incompatible direct connection; the pooled `url` (port 6543) is what the running application actually connects through.

File: `backend/prisma/migrations/20250101000000_init/migration.sql`

Type: Raw SQL migration **Purpose:** Creates the actual database schema, including the two things `schema.prisma` cannot express: the generated `tsvector` column with weighted full-text search, and the HNSW vector index. **Why Created:** Hand-written specifically because Prisma's schema DSL has no syntax for `GENERATED ALWAYS AS (...) STORED` columns or `USING hnsw` index types.

Dependencies: Requires the `vector` Postgres extension, created as the very first statement (`CREATE EXTENSION IF NOT EXISTS "vector"`) since it must exist before any `vector` -typed column can be declared. **Key Concepts:** Weighted generated `tsvector` (`setweight(to_tsvector('english', title), 'A') || setweight(to_tsvector('english', plain_text), 'B')`) — title matches rank higher than body matches; GIN index on that column; HNSW index (`vector_cosine_ops`) on `document_chunks.embedding` . **Detailed Explanation:** HNSW was chosen over the alternative `pgvector` index type, `IVFFlat`, specifically because HNSW requires no "train on existing rows" step and offers sub-linear approximate-nearest-neighbor search that works correctly even on a table with very few rows — a real concern for a fresh single-user installation that might start with zero or a handful of documents. **Important Notes:** The README explicitly warns that `prisma migrate diff` will forever report the HNSW index as "missing" drift, since Prisma has no way to know it exists — this is expected and must not be "fixed" by dropping it.

File: `backend/src/prisma/prisma.service.ts`

Type: Injectable service **Purpose:** Wraps `PrismaClient` as a Nest-injectable service with lifecycle hooks and a test-only reset helper. **Dependencies:** `@prisma/client` . **Key Concepts:** `onModuleInit` connects the client; `enableShutdownHooks` closes the Nest app cleanly on Prisma's `beforeExit` event; `truncateAll()` (hard-guarded against `NODE_ENV === 'production'`) resets all tables,

used by the e2e test suite between runs. **Detailed Explanation:** Logging verbosity is environment-aware — warn/error in development, error-only in production — avoiding noisy query logs in a production environment while keeping useful debugging output locally. **How To Use:** Injected via constructor (`constructor(private prisma: PrismaService) {}`) into every service that needs database access.

File: `backend/src/prisma/prisma.module.ts`

Type: Module **Purpose:** Registers `PrismaService` as a provider and marks the module `@Global()` . **Why Created:** Being `@Global()` means every other module can inject `PrismaService` without individually importing `PrismaModule` — a deliberate convenience for what is, functionally, infrastructure every feature needs.

Group 3 — Auth Module

Why these belong together: GitHub OAuth login, JWT issuance/rotation, and every request-time auth check all live here — the one module every other authenticated feature implicitly depends on via the global `JwtAuthGuard` .

Order to understand them: `strategies/github.strategy.ts` → `auth.service.ts` → `strategies/jwt.strategy.ts` → `guards/*.ts` → `auth.controller.ts` → `auth.module.ts` .

File: `backend/src/auth/strategies/github.strategy.ts`

Type: Passport strategy **Purpose:** Handles the GitHub OAuth handshake and turns a GitHub profile into an application user.

Dependencies: `passport-github2` , `AuthService` . **Key Concepts:** `scope: ['user:email']` — requested specifically because GitHub omits email from the bare profile response when a user has kept it private. **Detailed Explanation:** The `validate()` method maps the raw GitHub profile into a `GithubProfileData` shape and calls `authService.validateGithubUser(...)` , which does the actual upsert. This strategy has no knowledge of JWTs or sessions at all — its only job is "given a successful GitHub OAuth handshake, produce (or find) an application user." **How To Use:** Invoked automatically by `GithubAuthGuard` on the `GET /auth/github` and `GET /auth/github/callback` routes; never called directly.

File: `backend/src/auth/auth.service.ts`

Type: Injectable service **Purpose:** Owns every piece of actual auth logic: upserting users from GitHub profiles, issuing token pairs, and — the most intricate part of the whole codebase — rotating and revoking refresh tokens with theft detection. **Dependencies:** `PrismaService` , `JwtService` , `ConfigService` . **Key Concepts:** `issueTokens` , `rotateRefreshToken` , `revokeRefreshToken` / `revokeAllForUser` , `hashToken` (SHA-256).

Detailed Explanation — the refresh-rotation-with-theft-detection scheme:

1. `issueTokens(user)` signs an access token (15-minute TTL, payload `{sub, email}`) and a refresh token (7-day TTL, payload adds a `jti: randomUUID()` — without this, two tokens issued in the same second for the same user could collide on the `tokenHash` unique constraint). The refresh token's **hash** (not the raw token) is persisted to `RefreshToken.tokenHash` , along with its expiry, a truncated user-agent string, and an IP address.
2. `rotateRefreshToken(rawToken)` verifies the token's signature and expiry, looks up the matching row by hash, and branches on its state: - If the row is **already revoked**, this is treated as a signal that the token has been stolen and replayed by an attacker after the legitimate rotation already happened — `revokeAllForUser` is called, killing every active session for that user, and an `UnauthorizedException` is thrown. - Otherwise, a new token pair is issued, the old row is marked `revokedAt` + `replacedById` (pointing at the new row, forming an auditable rotation chain), and the new pair is returned.
3. `expiryOf` decodes the *freshly-signed* JWT's own `exp` claim rather than recomputing the TTL independently — a small but meaningful decision that keeps expiry computation as a single source of truth (the signed token itself) rather than two places that could drift apart.

Why raw tokens are never persisted: only `hashToken()` 's SHA-256 output goes into the database. A comment in the code explains the choice of SHA-256 over bcrypt specifically: the refresh token itself already carries 256+ bits of cryptographically signed entropy from JWT signing, so bcrypt's deliberately-slow hashing would only add latency without adding meaningful security — bcrypt earns its cost defending against low-entropy human passwords, which this isn't.

Key Code Sections Explained:

- `issueTokens` : the `jti` claim is the detail most likely to be missed on a naive reimplement — without it, rapid successive logins/rotations for the same user could produce hash collisions.
- `rotateRefreshToken` : the "already revoked → nuke every session" branch is the actual theft-detection mechanism; it's what makes token rotation meaningfully more secure than a long-lived static refresh token, not just cosmetically different.
- `hashToken` : exported so other files (`auth.controller.ts` , for reading incoming cookies) can compute the same hash to look up a token's DB row without ever storing the raw value.

How To Use: Injected into `AuthController` ; `hashToken()` is also imported directly where a raw token needs to be looked up by its hash. **Important Notes:** `JwtModule.register({})` in `auth.module.ts` is deliberately empty — secrets are passed per-`sign / verify` call, not module-wide, specifically so the access-token secret and refresh-token secret are cryptographically independent (a leaked access-token secret alone cannot be used to forge a refresh token). `auth.service.spec.ts` has a dedicated test asserting exactly this: a refresh token signed with the `access` secret is rejected.

File: `backend/src/auth/strategies/jwt.strategy.ts`

Type: Passport strategy **Purpose:** Validates the `Authorization: Bearer <token>` header on every protected request.

Dependencies: `passport-jwt` , `PrismaService` , `ConfigService` . **Key Concepts:** A **live database lookup** inside `validate()` on every single authenticated request — not merely trusting the JWT's own claims. **Detailed Explanation:** After `passport-jwt` verifies the token's signature and expiry, `validate()` still performs an indexed primary-key lookup against the `User` table before considering the request authenticated. This is a deliberate trade: it costs one extra (fast, indexed) database round trip per request, in exchange for near-instant revocation — if an account is deleted or banned, its still-cryptographically-valid access token stops working immediately, rather than remaining usable for up to its full 15-minute lifetime. **How To Use:** Invoked automatically by `JwtAuthGuard` (the global guard) on every request that isn't `@Public()` .

Guards & controller (table + narrative for the controller):

FILE	TYPE	PURPOSE
<code>guards/github-auth.guard.ts</code>	Guard	Trivial <code>AuthGuard('github')</code> subclass — triggers the GitHub strategy
<code>guards/jwt-auth.guard.ts</code>	Guard	<code>AuthGuard('jwt')</code> subclass, registered globally as <code>APP_GUARD</code> ; overrides <code>canActivate</code> to check <code>@Public()</code> metadata (<code>Reflector.getAllAndOverride</code> , checking both the handler and its class) before deciding whether to actually enforce auth

File: `backend/src/auth/auth.controller.ts`

Type: Controller **Purpose:** Exposes the five HTTP endpoints that make up the entire auth surface: `GET github` , `GET github/callback` , `POST refresh` , `POST logout` , `GET me` . **Dependencies:** `AuthService` , `GithubAuthGuard` , `@Public()` , `@CurrentUser()` . **Detailed Explanation:** `GET github` and `GET github/callback` are both `@Public()` (a user isn't authenticated yet when starting or completing OAuth). The callback handler is the one place the `httpOnly` refresh cookie gets set — via a shared `cookieOptions()` helper (`httpOnly: true` , `secure` from config, `sameSite: 'none'` in production since Vercel and Render are different origins, `'lax'` otherwise, scoped to `path: '/api/auth'` , 7-day `maxAge`) — and then redirects to the frontend. Critically, **the access token is never included in this redirect** — only the refresh cookie is set; the frontend's `/auth/callback` page separately calls `POST /auth/refresh` to trade that cookie for an access token. This is what the README means by "the access token is never in a URL": nothing sensitive ends up in browser history, `Referer` headers, or server access logs. `POST refresh` (also `@Public()` , since a caller doesn't have a valid access token at this point by definition) is throttled at 30/min

and calls `rotateRefreshToken`. `POST logout` revokes the current refresh token and clears the cookie. `GET me` is the only protected route in this controller, simply projecting `@CurrentUser()`. **How To Use:** `github / github/callback` are hit via full browser navigation (not `fetch`, since OAuth requires a real page redirect); `refresh / logout / me` are called via the frontend's typed API client. **Important Notes:** `cookieOptions()`'s `sameSite: 'none'` in production requires `secure: true` to be simultaneously true — a `SameSite=None` cookie without `Secure` is rejected by modern browsers, so these two settings must always change together.

File: `backend/src/auth/auth.module.ts`

Type: Module **Purpose:** Wires `PassportModule` (session-less), a deliberately-empty `JwtModule`, `AuthController`, `AuthService`, both strategies, and both guards together. **Key Concepts:** `PassportModule.register({session: false})` — this app is fully stateless/JWT-based, no server-side session store.

File: `backend/src/auth/auth.service.spec.ts`

Type: Unit test **Purpose:** Verifies the four properties that matter most about the token-rotation scheme, with Prisma fully mocked. **Detailed Explanation:** Tests assert: (1) issuing a token pair never persists the raw refresh token, only its hash; (2) rotating a valid refresh token revokes the old row and returns a genuinely new pair; (3) replaying an already-revoked refresh token throws `UnauthorizedException` and revokes every session for that user — the theft-detection path; (4) a refresh token forged using the access-token secret is rejected, proving the two secrets are cryptographically independent in practice, not just in configuration.

Group 4 — Users Module

Why these belong together: the user's own profile and the dashboard aggregate — small, self-contained, no complex logic beyond one clever query-batching trick.

File: `backend/src/users/users.service.ts`

Type: Injectable service **Purpose:** Profile read/update, plus the dashboard aggregate query. **Dependencies:** `PrismaService`. **Key Concepts:** `dashboard()` runs six Prisma queries inside one `$transaction`: documents count, AI-requests count, a file-size aggregate (`_sum.size`), the 5 most recent documents, the 5 most recent files, and 8 recent AI-activity entries (each truncated to 140 characters for display). **Detailed Explanation:** The `update()` method conditionally spreads only the fields actually provided on the DTO — avoiding accidentally overwriting a field with `undefined` if a partial update omits it. The dashboard transaction's design rationale is explicit in a code comment: one round trip to the database beats six sequential HTTP-triggered queries from the frontend, meaningfully reducing dashboard load latency. **How To Use:** `GET /users/me`, `PATCH /users/me`, `GET /users/me/dashboard` all go through `UserController`, which projects `@CurrentUser('id')` before calling into this service.

Remaining files (table):

FILE	TYPE	PURPOSE
<code>users.controller.ts</code>	Controller	<code>GET me</code> , <code>PATCH me</code> , <code>GET me/dashboard</code> — all scoped to the authenticated user via <code>@CurrentUser('id')</code>
<code>users.module.ts</code>	Module	Trivial registration, exports <code>UserService</code>
<code>dto/update-user.dto.ts</code>	DTO	<code>name</code> (optional, ≤120 chars), <code>preferences</code> (optional free-form object for theme/UI settings)

Group 5 — Documents Module

Why these belong together: CRUD for the core content object of the app, plus the Tiptap→plain-text conversion that everything downstream (search, AI, chunking) depends on.

Order to understand them: `tiptap.util.ts` → `documents.service.ts` → `documents.controller.ts` → `dto/document.dto.ts` → `documents.module.ts`.

File: `backend/src/documents/tiptap.util.ts`

Type: Pure utility functions **Purpose:** Converts between Tiptap's rich-text JSON document format and flat plain text, in both directions. **Why Created:** Everything downstream of a document's content — full-text search (`plainText` feeds the generated `tsvector`), chunking, embedding, and AI prompts — needs plain text, not a nested JSON tree; but the editor and the database's `content` column need the rich JSON. Rather than storing two independently-edited copies, this file derives one from the other automatically. **Dependencies:** None (pure, dependency-free functions) — deliberately so it can run in both the request path (on every content update) and the worker path (`plainTextToTiptap` , converting extracted file text into an editable document) without pulling in any framework code. **Key Concepts:** `tiptapToPlainText` (a recursive tree walk), `plainTextToTiptap` (its inverse). **Detailed Explanation:** `tiptapToPlainText` recursively walks the Tiptap JSON tree: any node with a `text` string contributes that text; any node with nested `content` recurses into it; block-level node types (paragraph, heading, listItem, blockquote, codeBlock, taskItem, horizontalRule) get a trailing newline appended after being walked, so the output reads as sensible paragraphs rather than one run-on string. An unrecognized node type is simply recursed into rather than throwing — making the function total (it never fails on well-formed but unexpected Tiptap JSON). The final output collapses three-or-more consecutive newlines down to two and trims. `plainTextToTiptap` runs the reverse operation: it splits raw text on blank-line boundaries and wraps each resulting block in a Tiptap paragraph node — this is exactly what lets extracted file text (from `FileProcessor`) become a real editable Tiptap document rather than a read-only text dump. **Key Code Sections Explained:** The "total function, never throws on unknown node types" property is the detail most worth noticing — a naive recursive walker that pattern-matches on a fixed set of node types and throws on anything else would break the moment Tiptap's schema gained a new block type this file didn't know about yet. **How To Use:** `tiptapToPlainText` is called from `DocumentsService` on every content update; `plainTextToTiptap` is called from `FileProcessor` after text extraction. **Important Notes:** This file has no test file of its own directly, but its correctness is exercised indirectly through `documents.service.spec.ts` 's plain-text-derivation assertions.

File: `backend/src/documents/documents.service.ts`

Type: Injectable service **Purpose:** All document CRUD, rename, and the logic deciding when a document needs re-indexing. **Dependencies:** `PrismaService` , `QueueService` , `tiptap.util.ts` . **Key Concepts:** `assertOwned` (a reused ownership-check helper called before every mutation), the `LIST_SELECT` projection (excludes `content` / `plainText` from list responses — those fields can be large and aren't needed for a document list view), and the **rename-vs-content-update fork**. **Detailed Explanation:** `create()` builds an empty Tiptap document by default, derives `plainText` via `tiptapToPlainText` , and falls back the title to "Untitled" if left blank. `findAll()` runs a paginated, ownership- and archive-status-filtered query with an optional case-insensitive title search, using `$transaction` to run the `findMany` and its `count` atomically (so the returned total always matches the page of results, even under concurrent writes). `findOne()` and every mutation call `assertOwned` first — a minimal `findFirst({select: {id}})` check that throws `NotFoundException` if the document doesn't exist *or* belongs to a different user (deliberately the same error for both cases, so a user can't distinguish "doesn't exist" from "exists but isn't yours" by response shape — a small but real security-through-ambiguity choice). The most important logic lives in `update()` : it only rebuilds `plainText` and enqueues a reindex job **if** `dto.content !== undefined` — a title-only rename, an icon change, or an archive toggle is a cheap database write with zero reindexing cost, because none of those fields affect search or AI context. `remove()` relies entirely on the schema's cascade-delete rule to clean up chunks — no application-level cleanup code exists or is needed. **Key Code Sections Explained:** The rename/content-update fork inside `update()` is the single most important piece of business logic in this file — it's directly responsible for why editing a document's title doesn't trigger an expensive embedding job, while editing its body does. **How To Use:** Called exclusively from `DocumentsController` ; every method takes a `userId` as its first real argument, enforcing tenant isolation at the service layer rather than trusting a caller to filter correctly. **Important Notes:** `recent()` returns the 5 most recently updated non-archived documents using the same `LIST_SELECT` projection as `findAll()` .

Remaining files (table):

FILE	TYPE	PURPOSE
<code>documents.controller.ts</code>	Controller	POST <code>/documents</code> , GET <code>/documents</code> , GET <code>/documents/recent</code> , GET <code>/documents/:id</code> , PATCH <code>/documents/:id</code> , PATCH <code>/documents/:id/rename</code> , DELETE <code>/documents/:id</code> — all scoped via <code>@CurrentUser('id')</code>
<code>documents.module.ts</code>	Module	Registers controller + service; exports <code>DocumentsService</code> so other modules (e.g. the AI/queue workers) can depend on it
<code>dto/document.dto.ts</code>	DTO	<code>CreateDocumentDto</code> (title ≤200, content object, icon ≤16), <code>UpdateDocumentDto</code> (adds optional <code>isArchived</code>), <code>RenameDocumentDto</code> (title required, ≤200)
<code>documents.service.spec.ts</code>	Unit test	Verifies: <code>plainText</code> correctly derived on create; blank title defaults to "Untitled"; <code>reindex</code> is enqueued only on content changes, never on a title-only update; ownership violations throw <code>NotFoundException</code> and block deletion

Group 6 — Files Module

Why these belong together: the entire signed-upload-to-Supabase pipeline and text extraction, from ticket creation through completion.

Order to understand them: `storage.service.ts` → `text-extractor.service.ts` → `files.service.ts` → `files.controller.ts` → `dto/file.dto.ts` → `files.module.ts` .

File: `backend/src/files/storage.service.ts`

Type: Injectable service **Purpose:** All communication with Supabase Storage — minting signed upload/download URLs, downloading bytes for server-side processing, and best-effort deletion. **Dependencies:** `@supabase/supabase-js` , `ConfigService` . **Key Concepts:** Lazy client initialization (`OnModuleInit`), a soft-fail mode when Supabase credentials are unset, the service-role key (bypasses row-level security, server-only, never exposed to the browser), and `buildPath(userId, filename)` . **Detailed Explanation:** If `SUPABASE_URL` / `SUPABASE_SERVICE_KEY` aren't configured, the service logs a warning and disables uploads rather than throwing — a deliberate choice so the app still boots and is usable (minus file uploads) in a local dev environment without a Supabase project configured. `buildPath` generates `userId/uuid.ext` — namespacing every object under its owner (both for organization and as a natural access boundary) and using `randomUUID()` to guarantee no filename collision regardless of what the user named their file. `createSignedUpload(path)` is what the frontend's upload flow actually calls into indirectly — it returns `{path, signedUrl, token}` , which the browser then PUTs bytes to directly, meaning the backend process never buffers the file body. `createSignedDownloadUrl(path, expiresInSeconds = 3600)` mints a time-boxed read URL for previews/downloads. `download(path)` pulls the full file as a `Buffer` — used only by `FileProcessor` for text extraction, never in the HTTP request path. `remove()` logs a warning rather than throwing on failure — a deliberate fire-and-forget approach to cleanup, contrasted explicitly with `FilesService.remove()` 's DB-delete-first ordering (see below). **Key Code Sections Explained:** Every Supabase SDK error is wrapped into `InternalServerErrorException` — callers of this service never need to know or handle Supabase-specific error shapes. **How To Use:** Injected into `FilesService` (upload/download ticket minting) and into `FileProcessor` (downloading bytes for extraction). **Important Notes:** The service-role key must never reach the browser — this is why the frontend uses a *different*, much less privileged Supabase client (the anon key) for the one call it makes directly (`uploadToSignedUrl`).

File: `backend/src/files/text-extractor.service.ts`

Type: Injectable service **Purpose:** Turns raw uploaded bytes into normalized plain text — the single point where every supported file format converges into one text representation. **Dependencies:** `pdf-parse` , `mammoth` . **Key Concepts:** `SUPPORTED_MIME_TYPES` (a map of MIME type → format, covering PDF, DOCX, TXT, and two markdown MIME variants), `MAX_FILE_BYTES` (20MB),

`extract()` , `normalise()` . **Detailed Explanation:** `extract(buffer, mimeType, filename)` dispatches on MIME type, falling back to inspecting the filename's extension if the MIME type is missing or unrecognized (browsers are known to report an empty MIME type for `.md` files, for instance). PDF text comes from `pdf-parse` ; DOCX from `mammoth.extractRawText` (conversion warnings are logged at debug level, not surfaced — they're informational, not actionable for the user); TXT/MD are decoded directly as UTF-8. An unsupported format throws `BadRequestException` . Every extracted string is passed through `normalise()` : CRLF line endings become LF, runs of spaces/tabs collapse to one, three-or-more consecutive newlines collapse to two, and the whole string is trimmed — a code comment explains PDFs specifically tend to produce "ragged whitespace that wrecks chunk boundaries" if this step is skipped, since the chunking algorithm relies on clean paragraph breaks (`\n\n`) to find natural split points. **Key Code Sections Explained:** `SUPPORTED_MIME_TYPES` being exported and imported directly by `CreateUploadDto` 's validation decorator is the detail worth noting — the set of formats the app can actually parse is defined in exactly one place, and the upload validation layer reuses it rather than maintaining a second, potentially-drifting list. **How To Use:** Called from `FileProcessor` immediately after downloading a file's bytes from Supabase Storage. **Important Notes:** Neither `pdf-parse` nor `mammoth` performs OCR — a scanned image-only PDF will extract as empty or garbled text, a known limitation rather than a bug.

File: `backend/src/files/files.service.ts`

Type: Injectable service **Purpose:** Orchestrates the two-phase signed-upload handshake and the rest of file CRUD. **Dependencies:** `PrismaService` , `QueueService` , `StorageService` . **Key Concepts:** The upload is a deliberate **two-phase commit**, not a trust-the-client-blindly flow. **Detailed Explanation:**

- **Phase 1 — `createUploadTicket`** : builds a storage path, asks `StorageService` for a signed upload URL, and creates a `File` row with `status: UPLOADING` *before* any bytes have moved anywhere. This row is the idempotency anchor — if the browser's PUT to Supabase never happens or fails, the abandoned row simply sits at `UPLOADING / PENDING` with no further consequence (no cleanup job currently exists for this case, a known minor gap).
- **Phase 2 — `completeUpload`** : called by the client only *after* the direct-to-Supabase PUT has actually succeeded; flips the row's status to `PENDING` , records the final URL, and only now enqueues the file-processing job. Text extraction never starts until the client explicitly confirms the upload finished — the database row, not Supabase's own state, is the source of truth the rest of the system trusts.

`remove()` deletes the database row **before** attempting to delete the underlying Supabase object, with the ordering explained directly in a code comment: an orphaned storage object left behind by a failed cleanup call is a cheap, low-consequence problem (some wasted storage), while a failed delete request that leaves a dangling, now-unreferenced database row would be a more confusing state to detect and recover from later — so the riskier failure mode is pushed toward the cheaper side. **How To Use:** Called from `FilesController` ; the two-phase flow is driven entirely by the frontend's `useUploadFile` hook, which calls `createUploadTicket` , then PUTs to Supabase directly, then calls `completeUpload` . **Important Notes:** `getDownloadUrl` mints a fresh time-boxed signed URL on every call rather than caching one — download URLs are cheap to generate and this avoids ever serving an expired link.

Remaining files (table):

FILE	TYPE	PURPOSE
<code>files.controller.ts</code>	Controller	<code>POST /files/upload-url</code> (tightly throttled at 20/min — tighter than the global limit, since it mints external signed URLs), <code>POST /files/:id/complete</code> , <code>GET /files</code> , <code>GET /files/:id</code> , <code>GET /files/:id/download-url</code> , <code>DELETE /files/:id</code>
<code>files.module.ts</code>	Module	Registers <code>FilesController</code> , <code>FilesService</code> , <code>StorageService</code> , <code>TextExtractorService</code> ; exports all three so <code>WorkerModule</code> can reuse them in <code>FileProcessor</code>
<code>dto/file.dto.ts</code>	DTO	<code>CreateUploadDto</code> : <code>filename</code> (≤255), <code>type</code> (validated via <code>@IsIn(Object.keys(SUPPORTED_MIME_TYPES))</code>) , imported straight from <code>text-extractor.service.ts</code> , <code>size</code> (bounded <code>1..MAX_FILE_BYTES</code>) — rejecting an oversized file <i>before</i> a signed URL is even minted, not after 20MB has already been uploaded

Group 7 — AI Module

Why these belong together: every LLM and embedding call in the system, the RAG pipeline, and the chunking algorithm that feeds it — the largest and most architecturally significant module in the codebase.

Order to understand them: `interfaces/*.ts` → `providers/*.ts` → `chunking.util.ts` → `indexing.service.ts` → `prompts.ts` → `ai.service.ts` → `ai.controller.ts` → `dto/ai.dto.ts` → `ai.module.ts`.

File: `backend/src/ai/interfaces/ai-provider.interface.ts`

Type: Interface + DI token **Purpose:** Defines the contract every text-generation provider must implement — the seam that makes swapping Grok for another LLM a one-class change. **Key Concepts:** `AIPProvider` interface (`generateText` , `summarize` , `chat` , `rewrite` , `chatStream`); `AI_PROVIDER` — a `Symbol` -based DI token, since TypeScript interfaces don't exist at runtime and Nest needs a real token to bind a provider to. **Detailed Explanation:** The interface deliberately has no embedding method — Grok has no embeddings endpoint, and folding embeddings in here would produce a method that throws at runtime the instant a provider without embedding support was plugged in. This exact problem is what motivated splitting `EmbeddingProvider` into its own interface entirely. **How To Use:** Every consumer of AI text generation (`AIService`) depends only on this interface, injected via `@Inject(AI_PROVIDER)` , never on a concrete class like `GrokProvider` directly.

File: `backend/src/ai/interfaces/embedding-provider.interface.ts`

Type: Interface + DI token **Purpose:** Defines the contract for turning text into vectors, entirely separately from text generation. **Key Concepts:** `EmbeddingProvider` (`embed` , `embedMany` , `dimensions` , `name`); `EMBEDDING_PROVIDER` DI token. **Detailed Explanation:** This is the interface `IndexingService` depends on — it never knows or cares whether embeddings come from a local in-process model or a hosted API, only that whatever it's given returns 384-dimension vectors.

File: `backend/src/ai/providers/grok.provider.ts`

Type: Injectable, implements `AIPProvider` **Purpose:** The concrete Grok/xAI implementation — currently the only `AIPProvider` bound in the application. **Dependencies:** `openai` SDK (pointed at `GROK_BASE_URL`), `ConfigService` . **Key Concepts:** A single private `complete()` helper every public method routes through; `parseSummary()` ; `chatStream()` ; `truncate()` . **Detailed Explanation:** The client is constructed once with `maxRetries: 2` and `timeout: 120_000` , and every public method (`generateText` , `summarize` , `chat` , `rewrite`) funnels through `complete()` , which wraps any SDK or network failure into a `ServiceUnavailableException` — centralizing error handling in one place rather than repeating a try/catch in every method. `summarize()` requests `response_format: {type: 'json_object'}` and hands the result to `parseSummary()` , which strips markdown code fences the model sometimes wraps its JSON in, and falls back to treating the raw text as the summary if parsing still fails outright — a defensive measure against an LLM not perfectly honoring a requested output format, so a malformed response degrades gracefully instead of crashing the whole job. `chatStream()` is an async generator, yielding `chunk.choices[0]?.delta?.content` as the SDK's native stream produces it — this is what `AIService.answerStream()` iterates over to produce SSE frames. `truncate()` caps prompt input at roughly 100,000 characters using a `chars/4` heuristic (no real tokenizer dependency), applied before every call to avoid sending an oversized prompt that would fail or be needlessly expensive. **Key Code Sections Explained:** The centralized `complete()` wrapper is the detail that makes `GrokProvider` a template for any future provider — a `ClaudeProvider` implementing the same interface would need the same "wrap every SDK error consistently" pattern to preserve the app's uniform error contract. **How To Use:** Bound to the `AI_PROVIDER` token via `useClass:` `GrokProvider` in `ai.module.ts` — never referenced by name anywhere else in the codebase. **Important Notes:** The system prompt used for the JSON-mode summarize call must literally contain the word "JSON" — both Grok's and Groq's OpenAI-compatible APIs 400 on a `json_object` response-format request otherwise, per a code comment.

File: backend/src/ai/providers/local-embedding.provider.ts

Type: Injectable, implements `EmbeddingProvider` **Purpose:** Runs `Xenova/all-MiniLM-L6-v2` in-process via `transformers.js` — the default, free, API-key-free embedding provider. **Dependencies:** `@xenova/transformers` (dynamically imported). **Key Concepts:** Lazy dynamic `import()`, a single-flight loading promise (`pipelinePromise`), `pooling: 'mean'`, `normalize: true`. **Detailed Explanation:** The model is not loaded at application boot — it's imported and initialized only on the first actual embedding call, and a shared promise (`pipelinePromise`) ensures that if several embedding jobs start concurrently before the model finishes loading, they all await the same single load rather than racing to load duplicate copies of a 25MB model into memory. `pooling: 'mean'` reduces the model's per-token output into one fixed-size sentence vector; `normalize: true` L2-normalizes that vector — a specifically important detail, because it means cosine similarity (what pgvector's HNSW index is configured for via `vector_cosine_ops`) reduces to a plain dot product between two already-unit-length vectors, which is exactly what the comparison query in `indexing.service.ts` computes. **How To Use:** Bound to `EMBEDDING_PROVIDER` when `EMBEDDING_PROVIDER=local` (the default) in `.env`. **Important Notes:** The first embedding call after a cold start pays a real 2-4 second latency cost for model loading — this is precisely why embedding only ever happens inside a BullMQ worker, never inline during an HTTP request.

File: backend/src/ai/providers/openai-embedding.provider.ts

Type: Injectable, implements `EmbeddingProvider` **Purpose:** The alternative, hosted embedding provider using OpenAI's `text-embedding-3-small`. **Dependencies:** `openai` SDK. **Key Concepts:** `dimensions: 384` request parameter — truncating the model's native 1536-dimension output. **Detailed Explanation:** Explicitly requesting 384 dimensions (rather than the model's native 1536) is what keeps this provider's output shape identical to the local MiniLM provider's — both emit exactly 384 numbers, so the `pgvector` column's fixed width never needs to change when an operator switches `EMBEDDING_PROVIDER` in `.env`. The response is re-sorted by `item.index` before use, since OpenAI's batch embeddings API doesn't guarantee the returned order matches request order. **How To Use:** Bound to `EMBEDDING_PROVIDER` when `EMBEDDING_PROVIDER=openai` is set, alongside a valid `OPENAI_API_KEY`.

File: backend/src/ai/ai.module.ts

Type: Module **Purpose:** The single place where an abstract interface becomes a concrete implementation — the literal "one line" the README refers to for swapping providers. **Detailed Explanation:** `AI_PROVIDER` is bound with a static `useClass: GrokProvider`. `EMBEDDING_PROVIDER` is bound with a `useFactory` that reads `embedding.provider` from config and returns either `LocalEmbeddingProvider` or `OpenAIEmbeddingProvider` — a runtime, config-driven choice rather than a compile-time static binding, since which embedding provider to use is meant to be an operator decision (an env var), not a code change. **Important Notes:** Adding a new text-generation provider requires exactly one new class implementing `AIProvider` and one changed line here (`useClass: NewProvider`); no other file in the codebase needs to change.

File: backend/src/ai/chunking.util.ts

Type: Pure utility function **Purpose:** Splits a document's plain text into overlapping chunks suitable for embedding and retrieval — the algorithm that determines how well RAG retrieval actually works. **Key Concepts:** Paragraph-aware sliding window, `maxChars = 1200` (~300 tokens), `overlapChars = 200`. **Detailed Explanation:** The algorithm works in three stages. First, text is normalized (line-ending unification, trimming) and split on blank-line boundaries (`\n{2,}`) into paragraphs — respecting the text's natural structure rather than cutting at arbitrary character counts. Second, any single paragraph larger than `maxChars` goes through `splitOversized()`: a sentence-boundary regex (`/[^\.\!]+\.[\.\!]+|\s*[^\.\!]+\$/g`) splits it into sentences, which are then packed into chunks up to `maxChars`; if even a single "sentence" is still too long (e.g. a giant table or minified text with no punctuation), it's hard-cut at fixed character boundaries as a last resort — the function never fails to produce output regardless of how

pathological the input is. Third, the main loop accumulates paragraph-sized blocks into a buffer; when the next block would push the buffer past `maxChars`, the current buffer is flushed as a finished chunk, and the *last* `overlapChars` characters of that just-flushed chunk seed the start of the next buffer. This overlap is the detail that keeps RAG retrieval working across chunk boundaries — without it, a piece of information that happens to straddle where one chunk ends and the next begins could be split such that neither chunk alone contains a complete, retrievable answer to a question about it. Each finished chunk gets a `tokenCount` estimate (`Math.ceil(length / 4)`). **Key Code Sections Explained:** The overlap-seeding step is the single most important few lines in this file — it's a direct, working answer to "what happens if the answer to a user's question is split across two chunks," which is one of the most common practical failure modes in naive RAG implementations. **How To Use:** Called once per document by `IndexingService.indexDocument()`; takes a document's `plainText` and returns an array of chunk strings ready for embedding. **Important Notes:** `chunking.util.ts` is covered by tests in `ai.service.spec.ts` rather than its own spec file — verifying empty input produces zero chunks, short text produces exactly one chunk, long text produces multiple chunks each within the size bound, that the overlap actually appears (the tail of chunk N reappears at the head of chunk N+1), and that a 2500-character single "sentence" with no punctuation still hard-splits into at least five pieces.

File: `backend/src/ai/indexing.service.ts`

Type: Injectable service **Purpose:** The bridge between plain text and the vector database — turns a document into embedded, searchable chunks, and answers "which chunks are most similar to this query." **Dependencies:** `PrismaService`, `EMBEDDING_PROVIDER`, `chunking.util.ts`. **Key Concepts:** `indexDocument()` (full re-index, not incremental), `EMBED_BATCH_SIZE = 32`, raw parameterized SQL inserts with a `toVectorLiteral()` helper, `searchSimilar()`. **Detailed Explanation:** `indexDocument(documentId, onProgress)` always deletes every existing `DocumentChunk` row for that document before re-chunking and re-embedding from scratch — a deliberate simplification rather than trying to diff old and new chunk boundaries, justified by the fact that any edit can shift where chunk boundaries fall throughout the entire document, making incremental updates error-prone for comparatively little savings. Chunking happens once via `chunkText`, then embeddings are generated in batches of 32 via `embedMany` (reducing per-call overhead versus embedding one chunk at a time), reporting percentage progress back through a callback after each batch — this progress callback is what `FileProcessor / IndexingProcessor` forward on to the Socket.IO gateway as `embedding` stage percentages. Because Prisma has no native vector column type, each chunk is inserted via raw parameterized SQL (`Prisma.sql / $executeRaw`), with the embedding array converted to a Postgres vector literal string (`"[1,2,3]"`) via `toVectorLiteral()` and explicitly cast with `::vector` in the query — parameterized the whole way through, never string-concatenated, so there's no injection risk despite being hand-written SQL.

`searchSimilar(userId, query, topK = 6, documentId?)` embeds the incoming query text with the same provider, then runs a raw SQL query computing `1 - (c.embedding <=> $vector::vector) AS similarity` (pgvector's cosine-distance operator, inverted into a similarity score), joined against `documents` and filtered `WHERE d.user_id = userId` — tenant isolation is enforced directly in the SQL, not as an afterthought in application code — ordered by the same distance expression and limited to `topK`. An optional `documentId` filter is conditionally appended via `Prisma.sql / Prisma.empty`, letting the same query serve both "search across all my documents" (global semantic search) and "search within this one document" (in-document RAG for the AI chat panel scoped to a specific document). **Key Code Sections Explained:** `toVectorLiteral()` plus the `::vector` cast is the exact mechanism that lets a JavaScript number array become a value pgvector's index can use — getting this string format wrong (e.g. spaces after commas) would silently fail to match the column type. **How To Use:** `indexDocument` is called from `FileProcessor` and `IndexingProcessor`; `searchSimilar` is called from `AIService` (building RAG context) and `SearchService` (the semantic search endpoint) — both consumers depend on this one service rather than duplicating the raw SQL. **Important Notes:** Full re-indexing rather than incremental updates means editing even a single character anywhere in a long document re-embeds the entire document — an intentional simplicity-over-efficiency trade-off, made tolerable by the fact that indexing already runs asynchronously and debounced.

File: `backend/src/ai/prompts.ts`

Type: Constants / template functions **Purpose:** Every system prompt and prompt-construction function used across the four AI features. **Key Concepts:** `SUMMARY_SYSTEM_PROMPT`, `REWRITE_INSTRUCTIONS` + `rewritePrompt()`, `GENERATE_SYSTEM_PROMPT`, `ragSystemPrompt(context)`. **Detailed Explanation:** `SUMMARY_SYSTEM_PROMPT` forces a strict JSON shape (`{summary, keyPoints[], actionItems[]}`) — a code comment notes the literal word "JSON" must remain somewhere in

the prompt text, or Grok/Groq's `json_object` response-format mode rejects the request with a 400 error, an easy-to-miss API quirk. `REWRITE_INSTRUCTIONS` holds one instruction string per supported mode (`improve` , `professional` , `shorter` , `longer` , `simplify`); `rewritePrompt()` wraps the user's content with the selected mode's instruction plus an explicit "no preamble, no markdown fences" directive, so the model's raw output can be used directly without post-processing. `ragSystemPrompt(context)` is the most carefully worded prompt in the file: alongside the retrieved chunk context and numbered `[1]` , `[2]` citation instructions, it includes an explicit "say you don't know if the answer isn't in the provided context" clause — a comment in the code calls this instruction *more important than the retrieved text itself*, because without it the model will happily answer from its own general training knowledge instead of the user's actual documents the moment retrieval comes up short, silently breaking the "grounded in your own content" guarantee the whole RAG feature exists to provide, while still producing citation markers that look legitimate. **How To Use:** Imported directly by `AIService` — every call to Grok in the summarize/rewrite/generate/chat code paths constructs its system prompt from a function or constant in this file, so prompt wording changes happen in exactly one place. **Important Notes:** This file is a strong candidate for anyone iterating on AI output quality — prompt wording changes here have no other code to touch.

File: backend/src/ai/ai.service.ts

Type: Injectable service **Purpose:** Orchestrates every AI feature — the enqueue side (HTTP handlers), the execution side (called only from `AiProcessor`), and the one synchronous exception, streaming chat. **Dependencies:** `PrismaService` , `QueueService` , `AI_PROVIDER` , `IndexingService` , `prompts.ts` . **Key Concepts:** The enqueue/execute split, `run()` 's dispatch-by-type switch, `buildRagMessages()` , `answer()` vs. `answerStream()` . **Detailed Explanation:** HTTP-facing methods (`requestSummarize` , `requestRewrite` , `requestGenerate`) never call the AI provider directly — each creates an `AIRequest` row (`status: PENDING`), calls `QueueService.enqueueAi()` , and stores the returned job ID back onto the row, then returns immediately. The actual work happens in `execute(job, onProgress)` , called exclusively by `AiProcessor` from inside a BullMQ worker: it flips the row to `PROCESSING` , dispatches to a private `run()` method that switches on `request.type` (summarize/rewrite/generate/chat), and on success writes `COMPLETED` plus the output — for summarize specifically, the parsed result is *also* written onto the parent `Document.summary` column, so the document itself carries its own latest summary without a separate lookup. On any thrown error, the row is marked `FAILED` with the error message, and the error is re-thrown — deliberately, so BullMQ's own retry/backoff logic still gets to run; if this method swallowed the error instead, a transient failure would never be retried.

RAG assembly is shared between the streaming and non-streaming chat paths through `buildRagMessages()` : it calls `IndexingService.searchSimilar()` , formats each retrieved chunk into a numbered `[i] (document title)\n<content>` block, and constructs the full message array using `ragSystemPrompt()` plus conversation history plus the current question. `answer()` is the non-streaming version (used by `/ai/ask` and exercised in the e2e test). `answerStream()` is an async generator yielding three frame types in sequence: first `{type: 'sources'}` (the retrieved citations, sent before any generated text so the frontend can render "answering from these documents" immediately), then repeated `{type: 'token'}` frames as `provider.chatStream()` yields text, and finally `{type: 'done', requestId}` . Notably, this streaming path still writes an `AIRequest` row (type `CHAT`) for history purposes, even though — unlike every other AI feature — it runs entirely synchronously inside the HTTP request rather than through the queue; the code comment justifies this directly: the SSE response *is* the answer, so queuing it would only add latency for no benefit, but the history record is still worth keeping. **Key Code Sections Explained:** The rethrow in `execute()` 's catch block is easy to miss but critical — removing it would silently disable BullMQ's retry mechanism for every AI job type. The "write sources before any token" ordering in `answerStream()` is a deliberate UX choice: citations render before the answer starts appearing, rather than only becoming visible after the full response completes. **How To Use:** Called from `AiController` (enqueue-side methods, plus `answer` / `answerStream` directly) and from `AiProcessor` (`execute` , inside the worker). **Important Notes:** `ai.service.spec.ts` exercises the chunking algorithm's edge cases (described above) in addition to service-level orchestration logic.

File: backend/src/ai/ai.controller.ts

Type: Controller **Purpose:** Exposes every AI HTTP endpoint, and hand-implements the SSE streaming response for chat. **Detailed Explanation:** The entire controller is throttled at 30 requests/minute (`@Throttle`), tighter than the global 120/min limit, reflecting that every route here either costs an LLM call or enqueues one. `POST /ai/ask/stream` is written using the raw Express `Response` object rather than Nest's `@Sse()` decorator, specifically because `@Sse()` only supports GET requests with no body, while this

endpoint needs to accept a JSON body (the question, optional document scope, and conversation history) — so the handler manually sets `X-Accel-Buffering: no` (telling Render's reverse proxy not to buffer the response, which would otherwise defeat streaming entirely), writes hand-formatted `event: X\ndata: JSON\n\n` frames as `answerStream()` yields them, and catches any mid-stream error into a final `error` SSE event rather than letting the connection die silently. **How To Use:** Called by the frontend's `services/index.ts` (queued endpoints) and `streamAsk()` (the streaming endpoint) respectively. **Important Notes:** Any change to the SSE frame format here must be mirrored exactly in the frontend's hand-rolled SSE parser (`services/index.ts` 's `streamAsk`), since there's no shared schema enforcing the two stay in sync.

Remaining file:

FILE	TYPE	PURPOSE
<code>dto/ai.dto.ts</code>	DTO	<code>SummarizedDto</code> (content XOR documentId), <code>RewriteDto</code> (mode restricted to the same <code>REWRITE_MODES</code> set <code>prompts.ts</code> defines), <code>GenerateDto</code> (prompt ≤4000 chars), <code>AskDto</code> (question ≤2000 chars, history ≤20 messages, <code>topK</code> 1-20 default 6)

Group 8 — Queue Module (BullMQ)

Why these belong together: the entire async job infrastructure — producer and consumer sides deliberately live in two separate Nest modules to avoid a circular dependency, but conceptually form one system.

Order to understand them: `queue.constants.ts` → `queue.types.ts` → `redis.config.ts` → `queue.module.ts` → `queue.service.ts` → `worker.module.ts` → `workers/*.ts` .

File: `backend/src/queue/queue.constants.ts`

Type: Constants **Purpose:** Names the three queues (`files` , `ai` , `indexing`) and the three job names (`process-file` , `index-document` , `run-ai`) exactly once. **Why Created:** A single source of truth for queue/job names prevents a typo'd string in a producer from silently never matching any consumer.

File: `backend/src/queue/queue.types.ts`

Type: Type definitions **Purpose:** Shared payload shapes for each job type, and the `JobStage` union used in progress events. **Key Concepts:** `JobStage = 'queued' | 'uploading' | 'extracting' | 'chunking' | 'embedding' | 'generating' | 'completed' | 'failed'` — the vocabulary every processor uses to describe where a job currently is.

File: `backend/src/queue/redis.config.ts`

Type: Configuration helper **Purpose:** Builds `ioredis` -compatible connection options from a single `REDIS_URL` string, correctly for both local Redis and Upstash's production endpoint. **Detailed Explanation:** `redisOptionsFromUrl()` parses the URL and detects a `redis://` scheme to enable TLS (`tls: {servername}`). It also always sets `maxRetriesPerRequest: null` and `enableReadyCheck: false` — both required by BullMQ's use of Redis's blocking commands, and the latter specifically needed because Upstash restricts the `INFO` command BullMQ would otherwise call as part of its default ready-check. A comment explains why a *fresh* options object is constructed per queue rather than one shared `ioredis` client instance being reused: BullMQ internally mutates connection options per blocking connection it opens, so sharing one client's options object across multiple queues could cause queues to interfere with each other's connection state. **How To Use:** Called once per queue registration in `queue.module.ts` .

File: backend/src/queue/queue.module.ts

Type: Module (@Global() , producer-only) **Purpose:** Registers the three BullMQ queues and their shared default job options. **Key Concepts:** defaultJobOptions : attempts: 3 , exponential backoff starting at 2 seconds, removeOnComplete: {age: 3600, count: 500} , removeOnFail: {age: 24h} — completed jobs are cleaned up quickly (they're transient work records, not permanent history — the permanent record is the AIRequest / File database row), while failed jobs are kept longer for debugging. **Why Created:** Being global and producer-only lets any feature module inject QueueService and enqueue work without needing to import (or even be aware of) the processor classes that will eventually execute it — this is the specific design choice that avoids a circular-import problem between feature modules and the worker module.

File: backend/src/queue/queue.service.ts

Type: Injectable service **Purpose:** The only way anything in the codebase enqueues a background job — three producer methods, one per queue. **Detailed Explanation:** enqueueFileProcessing and enqueueAi both use fixed, deterministic job IDs (file-\${fileId} , ai-\${aiRequestId}) — since BullMQ treats a job ID as unique within a queue, re-enqueuing the same logical operation (e.g. accidentally calling complete-upload twice) can never create duplicate work; the second enqueue attempt is simply a no-op against the already-existing job.

enqueueIndexing(data, delayMs = 15_000) is the most interesting method in the file — the actual mechanism behind the README's "editing a document re-embeds it, debounced by a fixed BullMQ job id so a burst of autosaves collapses into one indexing run." It works by explicitly removing any existing job with the fixed ID index-\${documentId} before adding a new one with the same ID and a fresh 15-second delay. If a user types continuously and triggers five autosaves in ten seconds, each autosave call cancels the previous pending delayed job and reschedules a new one — only the very last autosave in that burst ever actually survives long enough to fire, collapsing five potential re-indexing runs into exactly one. Because this debounce state lives in Redis (as an actual delayed BullMQ job), rather than a setTimeout in Node process memory, it survives a backend restart mid-burst and would continue to work correctly even if the backend ever ran as multiple replicas — a meaningfully more robust debounce mechanism than an in-memory timer. **How To Use:** Injected into DocumentsService (enqueueIndexing), FilesService (enqueueFileProcessing), and AiService (enqueueAi) — every feature module that needs background work done depends only on this service, never on a processor class directly.

File: backend/src/queue/worker.module.ts

Type: Module (consumer-only) **Purpose:** Hosts the three @Processor classes and imports exactly the feature modules they need (AiModule , FilesModule) to do real work. **Why Created:** Kept as a separate module from QueueModule specifically to avoid the circular dependency that would result if feature modules needed to import processor-containing code just to enqueue work. A code comment notes this separation also means the worker portion of the application could, in principle, be extracted into its own deployment later, since the queue (Redis) is already the only coupling between "the part that enqueues" and "the part that processes" — currently both run in the same Node process for simplicity.

File: backend/src/queue/workers/ai.processor.ts

Type: BullMQ processor (@Processor , WorkerHost) **Purpose:** Consumes run-ai jobs — the execution side of every summarize/rewrite/generate request. **Key Concepts:** concurrency: 4 ; @OnWorkerEvent('failed') failure handling. **Detailed Explanation:** Each job calls AiService.execute() , emitting generating then completed progress stages over Socket.IO, followed by a resource:updated event so the frontend refetches the affected AIRequest / Document . The failure handler specifically checks job.attemptsMade >= job.opts.attempts before marking the underlying AIRequest row FAILED — ensuring a DB row (and the Socket.IO event derived from showing it as failed) only reflects a truly final outcome, not an intermediate retry attempt that BullMQ is about to automatically retry anyway. **How To Use:** Never called directly — BullMQ invokes it automatically whenever a job lands in the ai queue.

File: `backend/src/queue/workers/file.processor.ts`

Type: BullMQ processor **Purpose:** Consumes `process-file` jobs — the entire pipeline from "uploaded bytes exist in Supabase Storage" to "searchable, embedded document chunks exist in Postgres." **Key Concepts:** `concurrency: 2` (deliberately lower than the AI processor — a code comment explains PDF parsing and local embedding are both CPU-bound, and the target deployment (Render's free/starter tier) has only a single core, so running too many of these concurrently would just cause them to contend for the same CPU rather than genuinely parallelize). **Detailed Explanation:** The pipeline is: download bytes from Supabase Storage → extract text via `TextExtractorService` → create or update the associated `Document` row (handling the "reprocessing an already-processed file" case) → convert extracted plain text into Tiptap JSON via `plainTextToTiptap` (so an uploaded file becomes a fully editable document, not a read-only text dump) → call `IndexingService.indexDocument()`. Progress is reported in granular stages matching the `JobStage` vocabulary: `extracting` (10%, then 35% once extraction actually completes) → `chunking` (50%) → `embedding` (scaled from 50% to 95%, proportional to `IndexingService`'s own internal batch-completion percentage) → `completed` (100%) — giving the frontend's per-file progress bar meaningfully granular feedback rather than a single "processing..." spinner for the whole operation. The same `attemptsMade` -exhaustion guard as `AIProcessor` applies before marking a file `FAILED`. **How To Use:** Never called directly — triggered by `FilesService.completeUpload()` enqueueing a job.

File: `backend/src/queue/workers/indexing.processor.ts`

Type: BullMQ processor **Purpose:** Consumes the debounced `index-document` jobs produced whenever a document's content is edited. **Key Concepts:** `concurrency: 2`. **Detailed Explanation:** Functionally simple — it just calls `IndexingService.indexDocument()` again (the same full-reindex operation `FileProcessor` uses) and emits `embedding` → `completed` progress. Unlike the other two processors, it has no `@OnWorkerEvent('failed')` handler of its own, relying on BullMQ's default retry behavior without a corresponding DB-status side effect — a minor asymmetry with the other two processors' explicit failure handling. **How To Use:** Never called directly — triggered by `QueueService.enqueueIndexing()`'s debounce mechanism, ultimately originating from `DocumentsService.update()` whenever a document's content changes.

Group 9 — Events Module (Socket.IO Gateway)

Why this stands alone: it's the one piece of realtime infrastructure every worker and no HTTP controller talks to directly — a genuinely separate concern from both the REST API and the queue.

File: `backend/src/events/events.gateway.ts`

Type: WebSocket gateway (`@WebSocketGateway`) **Purpose:** Authenticates Socket.IO connections and pushes job-progress/resource-update notifications to the specific user they belong to. **Dependencies:** `JwtService`, `ConfigService`. **Key Concepts:** JWT handshake auth, `userRoom(userId)` (`user:${userId}`), `job:progress` / `resource:updated` events, CORS-origin-as-a-function. **Detailed Explanation:** `handleConnection` reads a token from either `handshake.auth.token` (the frontend's chosen method) or an `Authorization` header, verifies it with the same JWT secret and `JwtService` used by REST auth, and on success joins the socket to a room named `user:${userId}` — every subsequent emit to that user targets this room specifically, `to(userRoom(userId))`, and the gateway never broadcasts to all connected clients. A missing or invalid token results in `client.disconnect(true)` with no further error detail surfaced — a deliberately terse failure mode, since a socket connection attempt isn't a user-facing form submission needing detailed error messaging. The CORS origin is configured as a **function**, evaluated fresh on every handshake, that reads `process.env.FRONTEND_URL` directly rather than through `ConfigService` — a documented necessity, since Nest evaluates gateway decorator arguments before `ConfigModule` has finished loading environment variables from `.env`, so a `ConfigService`-based origin at decorator-evaluation time would see an undefined value. **Key Code Sections Explained:** The per-user room strategy is the single detail that makes this gateway safe from a tenant-isolation perspective — even if two users are connected simultaneously, there is no code path by which one user's job-progress event could ever reach the other's socket. **How To Use:** Injected into every BullMQ processor (`AIProcessor`, `FileProcessor`, `IndexingProcessor`) to emit progress; the frontend

connects via `lib/socket.ts` and listens via the `RealtimeBridge` component in `providers.tsx`. **Important Notes:** `events.module.ts` is `@Global()` specifically so any processor, in any module, can inject the gateway singleton without `WorkerModule` needing to explicitly import `EventsModule` — one more instance of this codebase's consistent "global module for genuinely cross-cutting infrastructure" pattern (alongside `PrismaModule` and `QueueModule`).

Group 10 — Search Module

Why these belong together: the two independent retrieval paths (keyword and semantic) users actually interact with, plus the deliberate choice not to merge their results.

File: `backend/src/search/search.service.ts`

Type: Injectable service **Purpose:** Runs full-text and semantic search, and combines them (side by side, not fused) for the "search everything" experience. **Dependencies:** `PrismaService`, `IndexingService` (imported via `AiModule`). **Key Concepts:** `keyword()`, `semantic()`, `combined()`, `websearch_to_tsquery`, `ts_rank`, `ts_headline`. **Detailed Explanation:** `keyword(userId, query, limit = 20)` runs a raw parameterized SQL query using `websearch_to_tsquery('english', ...)` — chosen deliberately over the stricter `to_tsquery` specifically because `websearch_to_tsquery` accepts natural human search syntax (quoted phrases, `OR`, a leading `-` for exclusion) and, critically, never throws a syntax error on malformed input, unlike `to_tsquery`. The query also computes `ts_rank` for ordering and `ts_headline` for a server-generated `<mark>`-wrapped snippet — meaning the frontend never needs any client-side text-highlighting logic; it just renders the returned HTML fragment. An empty or whitespace-only query short-circuits before any database call at all.

```
SELECT d.id, d.title, d.icon,
       ts_headline('english', d.plain_text,
                  websearch_to_tsquery('english', ${trimmed}),
                  'MaxFragments=2, MinWords=8, MaxWords=24, StartSel=<mark>, StopSel=</mark>') AS snippet,
       ts_rank(d.search_vector, websearch_to_tsquery('english', ${trimmed})) AS rank,
       d.updated_at
FROM documents d
WHERE d.user_id = ${userId} AND d.is_archived = false
      AND d.search_vector @@ websearch_to_tsquery('english', ${trimmed})
ORDER BY rank DESC, d.updated_at DESC
LIMIT ${limit}
```

`semantic(userId, query, limit = 10)` delegates entirely to `IndexingService.searchSimilar()`, truncating each result's excerpt to 280 characters for display. `combined(userId, query)` runs both paths via `Promise.all`, but — notably — wraps only the semantic call in `.catch(() => [])`. This is a deliberate asymmetry: if the embedding provider is unavailable or slow, keyword search still returns results and the semantic column simply degrades to empty, whereas keyword search (considered the "always available" core path) is not similarly protected. The two result sets are never merged or reranked together into one list — a code comment explains why: full-text `ts_rank` and cosine similarity are scores on "incomparable scales," so attempting to fuse them into a single ranking would produce a meaningless combined score; showing them side by side, letting the user judge relevance from each source themselves, was chosen as the more honest approach. **Key Code Sections Explained:** The asymmetric error handling in `combined()` (semantic guarded, keyword not) is the single detail most worth internalizing about this file's design philosophy — it treats the two retrieval paths as having different reliability guarantees, not interchangeable equals. **How To Use:** Called from `SearchController`, branching on the requested `mode` query parameter. **Important Notes:** `search.service.spec.ts` specifically verifies `combined()` still returns keyword hits when the semantic call rejects — proving the graceful-degradation contract actually holds, not just that it's intended.

Remaining files (table):

FILE	TYPE	PURPOSE
<code>search.controller.ts</code>	Controller	GET <code>/search?q=&mode=keyword\ semantic\ all</code> (default <code>all</code>); branches on <code>mode</code> and shapes the response as <code>{query, keyword, semantic}</code> — no dispatch logic lives here beyond that branch, it's kept in the service
<code>search.module.ts</code>	Module	Imports <code> AiModule </code> specifically because semantic search needs <code> IndexingService </code> , which <code> AiModule </code> owns — documented directly in a code comment as the reason for this cross-module dependency
<code>search.service.spec.ts</code>	Unit test	Verifies: empty query short-circuits without a DB call; raw FTS rows are mapped correctly (string rank like <code>'0.42'</code> converts to a real number); semantic excerpts truncate to exactly 280 characters; <code>combined()</code> degrades gracefully when semantic search rejects

Group 11 — Backend Tests (End-to-End)

File: `backend/test/app.e2e-spec.ts`

Type: End-to-end test **Purpose:** Proves the entire async pipeline — queue, worker, pgvector write, semantic read-back — genuinely works together, using real Postgres and Redis. **Dependencies:** The full `AppModule` , `@nestjs/testing` , `supertest` . **Key Concepts:** `Test.createTestingModule({imports: [AppModule]}).overrideProvider(...)` — swapping only three providers (`AI_PROVIDER` , `EMBEDDING_PROVIDER` , `StorageService`) for deterministic fakes, while everything else runs for real. **Detailed Explanation:** The fake `EMBEDDING_PROVIDER` generates deterministic, `Math.sin` -based, L2-normalized 384-dimension vectors — a specific choice ensuring pgvector receives genuinely valid, internally-comparable vectors (not random noise), so the semantic-search assertion later in the test is actually meaningful rather than trivially passing regardless of correctness. A session is minted by calling `AuthService.issueTokens()` directly, bypassing the actual GitHub OAuth redirect (which can't be automated in a test). The test then exercises the full flow in order: reject an unauthenticated request → GET `/auth/me` succeeds with a valid token → create a document (asserting Tiptap-to-plain-text flattening happened correctly) → request an upload URL → call "complete" → poll (via a hand-written `waitFor()` helper, 500ms interval, 60-second timeout) until the real `FileProcessor` worker finishes processing → assert `DocumentChunk` rows actually exist in the database → assert semantic search returns hits → POST `/ai/summarize` → poll until the resulting `AIRequest` reaches `COMPLETED` → assert `document.summary` was actually written. **How To Use:** Run via `npm --filter @kw/backend test:e2e` ; requires real Postgres and Redis to be reachable (via `docker compose up` locally, or CI service containers). **Important Notes:** `test/jest-e2e.json` sets a generous 120-second `testTimeout` specifically to accommodate the real, non-mocked async worker completion this test waits on.

Group 12 — Backend Tooling & Configuration

FILE	TYPE	PURPOSE
<code>backend/package.json</code>	Config	Scripts (<code> build </code> , <code> dev </code> , <code> start:prod </code> runs <code> prisma migrate deploy </code> before starting, <code> test </code> , <code> test:e2e </code> , <code> postinstall: prisma generate </code>), full dependency list, embedded Jest unit-test config
<code>backend/tsconfig.json</code> / <code>tsconfig.build.json</code>	Config	TypeScript compiler options — strict mode, ES2023 target, decorators enabled, <code> strictPropertyInitialization: false </code> (a Nest-DI-specific carve-out)
<code>backend/nest-cli.json</code>	Config	Nest CLI project configuration (source root, compiler options passthrough)
<code>backend/eslint.config.mjs</code>	Config	Flat ESLint config — <code> typescript-eslint </code> recommended rules plus <code> eslint-config-prettier </code> to disable any stylistic rule that would conflict with Prettier's own formatting
<code>backend/.prettierrc</code>	Config	Single quotes, trailing commas, 100-character line width

FILE	TYPE	PURPOSE
backend/Dockerfile	Build	Multi-stage (<code>deps</code> → <code>build</code> → <code>runtime</code>) image build; copies <code>backend/prisma</code> before <code>pnpm install</code> (since <code>postinstall</code> needs the schema present); runtime entrypoint runs <code>prisma migrate deploy && node dist/main.js</code>

Group 13 — Frontend App Router: Layouts & Pages

Why these belong together: the entire routing/navigation skeleton of the frontend, and the client-side auth guard every protected page relies on.

Order to understand them: `app/layout.tsx` → `components/providers.tsx` (Group 14) → `app/login/page.tsx` → `app/auth/callback/page.tsx` → `app/(workspace)/layout.tsx` → the six workspace pages.

File: `frontend/app/layout.tsx`

Type: Root layout (Server Component) **Purpose:** The outermost wrapper for every page — sets the font and wraps everything in `<Providers>`. **Detailed Explanation:** Configures the `Inter` font as a CSS variable and renders `<Providers>{children}</Providers>` — deliberately minimal, since almost all real setup logic lives in `providers.tsx` rather than here.

File: `frontend/app/page.tsx`

Type: Server Component **Purpose:** The root `/` route — immediately redirects to `/dashboard`. **Detailed Explanation:** A one-line `redirect('/dashboard')` — there's no real "home page" distinct from the dashboard; this file exists purely because Next.js requires something at the root route.

File: `frontend/app/login/page.tsx`

Type: Client Component **Purpose:** The public login screen — a "Sign in with GitHub" call to action. **Detailed Explanation:** If `useAuthStore().status === 'authenticated'` already (e.g. a returning user whose session is still valid), it redirects straight to `/dashboard` rather than showing the login screen again. The GitHub sign-in control is a plain `<a href="...">` element, not a `fetch / onClick` handler — a deliberate necessity, since starting an OAuth flow requires an actual full-page browser navigation to GitHub's consent screen, which a JavaScript `fetch` call cannot initiate.

File: `frontend/app/auth/callback/page.tsx`

Type: Client Component **Purpose:** The landing page the backend redirects to after GitHub OAuth completes and the httpOnly refresh cookie has been set. **Detailed Explanation:** Immediately calls `refreshAccessToken()` (trading the now-present httpOnly cookie for a real access token and user object), then routes to `/dashboard` on success or back to `/login` on failure. This page's entire purpose is bridging "the backend just set a cookie" to "the frontend now has an access token in memory."

File: `frontend/app/(workspace)/layout.tsx`

Type: Client Component **Purpose:** The shared shell (sidebar + topbar + command palette) for all six authenticated pages, and the sole client-side route guard in the app. **Key Concepts:** A pure client-side auth guard driven by `useAuthStore().status`. **Detailed Explanation:** While `status === 'loading'`, the layout renders only a spinner — no child page ever mounts before authentication status is definitively known one way or the other. If `status === 'anonymous'`, it redirects to `/login`. Only once `status === 'authenticated'` does it render `Sidebar` + `Topbar` + a scrollable `main` region containing the actual page + the global

CommandPalette . A code comment explicitly justifies the complete absence of any *server-side* route protection here: the session lives in an httpOnly cookie the Next.js server has no cheap way to validate on its own, and every real API call is independently re-authorized by the NestJS backend regardless — so the client guard's only actual job is controlling what UI shell renders, not enforcing real security (the backend does that). **How To Use:** Applies automatically to every page under `app/(workspace)/` due to Next.js's layout-nesting convention — no page needs to opt in individually. **Important Notes:** This means an unauthenticated visitor's browser briefly downloads the workspace shell's JavaScript bundle before the client-side redirect fires — a minor, acknowledged UX/security-theater gap, not a real data-exposure issue, since no actual API data is ever fetched or rendered before the redirect completes.

The six workspace pages (table):

FILE	PURPOSE
<code>(workspace)/dashboard/page.tsx</code>	<code>useDashboard()</code> query drives four animated stat cards (documents/AI requests/files/storage used), a recent-documents list, a recent-files list with status badges, and a recent-activity feed
<code>(workspace)/documents/page.tsx</code>	Document list with client-side title filtering, inline rename (local state + form), create-and-navigate, delete via a dropdown menu
<code>(workspace)/documents/[id]/page.tsx</code>	Loads one document and renders the <code>DocumentEditor</code> and <code>AIPanel</code> side by side; the editor's slash-menu AI actions bubble up via an <code>onAiAction</code> callback that opens the AI panel to a specific tab
<code>(workspace)/files/page.tsx</code>	<code>UploadDropzone</code> plus a list of files with live status badges and per-file progress bars driven by the Socket.IO-fed <code>useUiStore().jobs</code> map
<code>(workspace)/search/page.tsx</code>	A single query box with two-column keyword-vs-semantic results; keyword snippets are rendered via <code>dangerouslySetInnerHTML</code> specifically because the <code><mark></code> tags come from the backend's own <code>ts_headline</code> SQL function, not from unsanitized user input
<code>(workspace)/assistant/page.tsx</code>	Three tabs: Chat (the full RAG-grounded <code><Chat/></code> component), Generate (a freeform prompt that polls a queued job to completion), History (a list of past <code>AIRequest</code> rows)
<code>(workspace)/settings/page.tsx</code>	Display-name editing (updates both the backend and the local auth store, then invalidates the dashboard cache) and a theme picker via <code>next-themes</code>

Group 14 — Global Providers & Realtime Bridge

File: `frontend/components/providers.tsx`

Type: Client Component — the composition root of the entire frontend **Purpose:** Wires together every cross-cutting concern the app needs: server-state caching, theming, session bootstrapping, and the realtime socket bridge. **Dependencies:** `@tanstack/react-query`, `next-themes`, `lib/socket.ts`, `store/auth-store.ts`, `store/ui-store.ts`, `sonner`. **Key Concepts:** A nested provider tree: `QueryClientProvider` → `ThemeProvider` → `SessionBootstrap` → `RealtimeBridge` → `{children}` → `Toaster`. **Detailed Explanation:** `QueryClientProvider` is configured once here with the app's global React Query defaults: 30-second `staleTime`, no refetch-on-window-focus, and retries explicitly skipped for 401/403/404 error responses (matched via a regex on the error message) — those status codes represent auth/not-found states that retrying can never fix, so retrying them would only waste time and requests. `ThemeProvider` (from `next-themes`) enables class-based dark/light/system theming for every descendant. `SessionBootstrap` is the component responsible for the app's entire "silent re-authentication on page load" behavior: while `useAuthStore().status === 'loading'`, it fires `refreshAccessToken()` exactly once, relying on the httpOnly refresh cookie the browser automatically sends — this single component is what makes a hard page refresh not immediately bounce an already-logged-in user to the login screen. `RealtimeBridge` opens the Socket.IO connection only once a valid `accessToken` exists in the auth store, and wires up the two events the gateway ever emits: `job:progress` (calls `useUiStore().trackJob(...)`, updating the progress-bar map any component can read) and `resource:updated` (calls `router.refresh()`, invalidating cached/rendered state for whatever resource changed). **Key Code Sections Explained:** The provider nesting order matters — `SessionBootstrap` must resolve auth status before `RealtimeBridge` can know whether there's a valid token to connect the

socket with, which is why bootstrapping is a separate, earlier component in the tree rather than logic folded directly into the socket bridge. **How To Use:** Rendered once, in `app/layout.tsx`, wrapping the entire application — never instantiated more than once.

Group 15 — Client State (Zustand Stores)

File: `frontend/store/auth-store.ts`

Type: Zustand store **Purpose:** Holds the current access token, user object, and auth status — deliberately in memory only. **Key Concepts:** `status: 'loading' | 'authenticated' | 'anonymous'`, no persistence middleware. **Detailed Explanation:** The absence of any `persist` middleware is a deliberate security decision, explained directly in a code comment: the `httpOnly` refresh cookie is the one durable credential, invisible to JavaScript and therefore immune to XSS-based exfiltration; keeping the access token purely in memory means there is no `localStorage` entry an XSS payload could steal to impersonate the user for up to 15 minutes. The trade-off is that every hard page refresh starts from `status: 'loading'` and requires `SessionBootstrap` to silently re-authenticate via the cookie before the app is usable — a small, deliberate latency cost paid for a meaningfully smaller credential-theft attack surface. **How To Use:** `setSession(accessToken, user)` called after login/refresh succeeds; `clear()` called on logout or refresh failure; `status` is read by every route guard in the app (`(workspace)/layout.tsx`, `login/page.tsx`).

File: `frontend/store/ui-store.ts`

Type: Zustand store **Purpose:** Purely client-local UI state that has no server-side counterpart to fetch or cache. **Key Concepts:** `commandPaletteOpen`, `aiPanelOpen` (default `true`), `jobs: Record<resourceId, JobProgressEvent>`. **Detailed Explanation:** `jobs` is the live progress map fed exclusively by `RealtimeBridge`'s `Socket.IO job:progress` listener — any component (the files list's per-file progress bar, the AI panel) reads its own entry out of this map by resource ID to render a progress indicator, without needing to independently manage its own socket subscription. **How To Use:** `trackJob(event)` called from `RealtimeBridge`; `jobs[resourceId]` read anywhere a progress bar needs live status.

Group 16 — API Client, Sockets & Shared Types (`lib/` + `services/`)

Why these belong together: every network call the frontend ever makes flows through this small set of files — nothing else in the app talks to `fetch`, `Socket.IO`, or Supabase directly.

Order to understand them: `lib/types.ts` → `lib/utils.ts` → `lib/api.ts` → `lib/socket.ts` → `lib/supabase.ts` → `services/index.ts`.

File: `frontend/lib/types.ts`

Type: Type definitions **Purpose:** Hand-written TypeScript types mirroring every backend DTO/model the frontend consumes. **Detailed Explanation:** Defines `User`, `DocumentSummary` / `ListItem` / `Detail`, `FileStatus` / `StoredFile`, `AIRequestType` / `Status` / `AIRequest`, `Paginated<T>`, `DashboardStats`, `KeywordHit` / `SemanticHit` / `SearchResults`, `AskSource`, `RewriteMode`, and `JobStage` / `JobProgressEvent` (the exact shape of the `Socket.IO` progress payload). This is a genuinely hand-maintained, independent copy of the backend's contract — there is no code generation step and no shared types package in this monorepo linking the two sides. A field renamed on a backend DTO will not produce a frontend compile error; it will only surface as a runtime mismatch, which is the real cost of this simplicity trade-off. **How To Use:** Imported by every hook, service, and component that touches API data.

File: frontend/lib/api.ts

Type: Fetch wrapper + auth-refresh logic **Purpose:** The single function every REST call in the app goes through — attaches auth, handles 401-triggered token refresh, and normalizes errors. **Key Concepts:** `apiFetch<T>()`, `ApiError`, the single-in-flight-refresh-promise pattern. **Detailed Explanation:** `apiFetch` attaches `Authorization: Bearer <accessToken>` from the auth store and always sends `credentials: 'include'` (so the httpOnly refresh cookie rides along automatically). On a `401` response, it calls `refreshAccessToken()` and retries the original request exactly once (guarded by a `_retried` flag on the request options, preventing an infinite retry loop if the refreshed token is *also* somehow rejected).

The single-in-flight-promise pattern is the most subtle and important piece of logic in this file:

```
let refreshInFlight: Promise<string | null> | null = null;
async function refreshAccessToken() {
  refreshInFlight ??= (async () => {
    try {
      const response = await fetch(`${API_URL}/auth/refresh`, { method: 'POST', credentials: 'include'
    });
    // ...
    useAuthStore.getState().setSession(data.accessToken, data.user);
  } catch { useAuthStore.getState().clear(); }
  finally { setTimeout(() => { refreshInFlight = null; }, 0); }
})();
return refreshInFlight;
}
```

Without this pattern, if several React Query hooks all received a `401` at roughly the same moment (a common occurrence right when an access token expires), each would independently `POST /auth/refresh`, and because the backend's refresh-token rotation treats a *second* use of the same refresh token as a theft signal (see `AuthService.rotateRefreshToken`), the second concurrent refresh call would trip the reuse-detection logic and kill every session for that user. By sharing exactly one in-flight promise across all simultaneous callers, every concurrent 401 in the same batch awaits the same single refresh call rather than each independently rotating the token. The promise is deliberately cleared on the *next* JavaScript tick (`setTimeout(..., 0)`), not immediately — ensuring every synchronous caller in the same batch has a chance to observe and await the same in-flight promise before it's reset for the next, later refresh cycle.

`ApiError` carries the backend's normalized `status` and `details` fields (matching `AllExceptionsFilter`'s response shape exactly); `api.get/post/patch/delete` are thin, typed convenience wrappers around `apiFetch`. **Key Code Sections Explained:** The `??=` single-flight assignment plus the deferred-clear `setTimeout` together are the two lines that prevent the single most dangerous frontend/backend interaction bug this app's auth design could otherwise produce. **How To Use:** Every hook in `hooks/*.ts` and every function in `services/index.ts` ultimately calls through `api.get/post/patch/delete`, never raw `fetch`.

File: frontend/lib/socket.ts

Type: Socket.IO client wrapper **Purpose:** Manages a single, app-wide Socket.IO connection. **Detailed Explanation:** `connectSocket(token)` passes the JWT via the connection's `auth` handshake payload (not a query string or header) and forces `transports: ['websocket']` only — skipping Socket.IO's default HTTP long-polling fallback entirely, since this app doesn't need to support environments where WebSocket connections are blocked. The connection is a module-level singleton (`let socket`), reused across the whole app rather than opened per-component; `disconnectSocket()` tears it down explicitly on logout. **How To Use:** Called exclusively from `RealtimeBridge` in `providers.tsx`.

File: frontend/lib/supabase.ts

Type: Supabase client wrapper **Purpose:** Provides the one Supabase client call the frontend makes directly — uploading bytes to a signed URL. **Detailed Explanation:** Created with `persistSession: false`, since there's no real Supabase Auth session in play here at all — this app doesn't use Supabase's own authentication system. The anon key this client uses grants no meaningful access on its own; every actual upload requires a one-time signed token minted by the backend, so exposing this key to the browser (via `NEXT_PUBLIC_SUPABASE_ANON_KEY`) carries no real risk. **How To Use:** Its `uploadToSignedUrl` method is called from `useUploadFile` in `hooks/use-files.ts`.

File: frontend/lib/utils.ts

Type: Utility functions **Purpose:** `cn()` (the standard `twMerge(clsx(...))` className-merging helper used throughout `components/ui`), `formatBytes()`, `initials()` (for avatar fallback text).

File: frontend/services/index.ts

Type: Typed API surface **Purpose:** Groups every backend endpoint into per-domain objects (`userService`, `documentsService`, `filesService`, `aiService`, `searchService`), and hosts the frontend's hand-rolled SSE parser. **Detailed Explanation:** Most of the file is thin, typed wrappers around `api.get/post/patch/delete` — e.g. `filesService.createUploadTicket()` returns exactly the `{fileId, path, signedUrl, token}` shape the direct-to-Supabase upload flow needs. The one substantial piece of logic is `streamAsk()`, which hand-parses the backend's Server-Sent Events response — a genuinely necessary piece of custom code, since the browser's built-in `EventSource` API only supports GET requests with no body, while this endpoint needs to accept a JSON body (the question, optional document scope, and conversation history):

```
const reader = response.body.getReader();
let buffer = '';
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
  const frames = buffer.split('\n\n'); // SSE events are separated by a blank line
  buffer = frames.pop() ?? ''; // the last (possibly incomplete) frame is kept for the
  next read
  for (const frame of frames) {
    const event = frame.match(/^event: (.*)$/m)?.[1];
    const data = frame.match(/^data: (.*)$/m)?.[1];
    // ...yield the typed event...
  }
}
```

This yields a discriminated union matching exactly what `AiController`'s streaming endpoint writes: `sources` (citations, sent first) → repeated `token` (the answer, streamed piece by piece) → a final `done` (carrying the persisted `AIRequest ID`) or `error`. **Key Code Sections Explained:** The buffer-and-split-on-double-newline logic is the correct, general way to parse SSE frames from a raw byte stream where a single `read()` call is not guaranteed to align with frame boundaries — a partial frame at the end of a chunk must be held back and prepended to the next chunk rather than parsed prematurely, which is exactly what `buffer = frames.pop() ?? ''` accomplishes. **How To Use:** Every hook in `hooks/*.ts` imports from this file rather than calling `lib/api.ts` directly — this is the layer that gives each API call its final, feature-specific typed shape. **Important Notes:** Any change to the backend's SSE frame format (in `ai.controller.ts`) must be mirrored exactly here — there is no schema shared between the two, only convention.

Group 17 — Editor Feature (Tiptap)

Why these belong together: the block editor and its slash-command menu are one cohesive interaction, built on the same Tiptap instance.

File: `frontend/features/editor/editor.tsx`

Type: Client Component **Purpose:** The document body editor — configures Tiptap, wires up autosave, and exposes keyboard shortcuts. **Dependencies:** `@tiptap/react`, `@tiptap/starter-kit`, `@tiptap/extension-placeholder`, `@tiptap/extension-task-list` / `task-item`, `@tiptap/extension-image`, `@tiptap/extension-link`, the local `SlashCommand` extension, `hooks/use-documents.ts`. **Key Concepts:** `immediatelyRender: false`, a 900ms debounced autosave, `Ctrl/Cmd+S` force-save, `editor.commands.setContent()` on document switch. **Detailed Explanation:** The extension list is `StarterKit` (headings restricted to levels 1–3), `Placeholder`, nested `TaskList + TaskItem`, `Image`, `Link` (autolink enabled, click-to-navigate disabled so links don't accidentally navigate away while editing), and the custom `SlashCommand` extension. `immediatelyRender: false` is a Next.js-15-specific requirement — Tiptap's editor view is inherently DOM-dependent and cannot meaningfully render during server-side rendering, so this flag defers actual editor construction to the client. Autosave works by clearing and resetting a 900ms `setTimeout` on every `onUpdate` event, calling the update mutation only once typing pauses — this means one network write per pause in typing, not one per keystroke, keeping both network traffic and (per the backend's rename-vs-content-update logic) reindexing job frequency reasonable. `Ctrl/Cmd+S` force-flushes any pending debounce timer and saves the title and content immediately, showing a confirmation toast — giving users an explicit "I know this is saved" affordance beyond trusting the invisible debounce. `Ctrl/Cmd+/` inserts a literal `/` character to open the slash menu without requiring the user to type it themselves. When the active document changes (navigating from one document to another), the component calls `editor.commands.setContent(...)` on the existing editor instance rather than unmounting and remounting a new one — preserving editor state/undo-history semantics correctly across navigation rather than treating each document as a fresh, disconnected editor instance. **Key Code Sections Explained:** The debounce-then-force-flush pattern (autosave timer + `Ctrl/Cmd+S` explicitly clearing it) is worth internalizing — it's the same general pattern the backend uses for reindexing debounce, applied one layer up in the stack. **How To Use:** Rendered by `(workspace)/documents/[id]/page.tsx`, receiving the current document and an `onAiAction` callback for the AI slash-menu items.

File: `frontend/features/editor/slash-command.tsx`

Type: Tiptap extension + React component **Purpose:** Implements the `/` -triggered block-insertion and AI-action menu. **Dependencies:** `@tiptap/suggestion`, `tippy.js`. **Key Concepts:** `@tiptap/suggestion` (Tiptap's official primitive for trigger-character-plus-filtered-popup interactions), a static `ITEMS` array (13 block types plus two AI actions), `tippy.js` popup lifecycle, `ReactRenderer`, `useImperativeHandle` for keyboard forwarding. **Detailed Explanation:** Rather than hand-building caret-position tracking and popup show/hide logic, this extension is built on `@tiptap/suggestion`, the library's own maintained solution for exactly this UX pattern. `ITEMS` includes ordinary block types (heading, list, task, code block, etc.) alongside two AI-specific entries (`AI: summarize`, `AI: rewrite`) — selecting one of the AI items doesn't insert a block at all; instead it calls an `onAi` callback that bubbles up to open the `AiPanel` on a specific tab, repurposing the same slash-menu UI as a launcher for a different feature entirely. Filtering as the user types after the `/` is a simple substring match against each item's `title` plus its `keywords` array. Popup positioning uses `tippy.js`, manually created in the extension's `onStart` handler and destroyed in `onExit` — not a persistent always-mounted component — while the dropdown's actual content is a genuine React component rendered via Tiptap's `ReactRenderer`. The trickiest piece of code in this file is keyboard handling: because the popup isn't a naturally-focused DOM element that would receive `keydown` events on its own, the dropdown component exposes an imperative handle via `useImperativeHandle`, and Tiptap's own `onKeyDown` hook forwards raw arrow-key/Enter events into that handle manually — an intentionally non-declarative pattern, necessary because the interaction model (typing continues in the editor while arrow keys should control a separate popup) doesn't fit React's normal event-flow assumptions cleanly. **Key Code Sections Explained:** The manual `tippy.js` lifecycle plus the `useImperativeHandle` keyboard-forwarding bridge together are the most "un-React-idiomatic" code in the entire frontend — worth flagging explicitly to anyone reading this file for the first time so the imperative style doesn't read as a mistake. **How To Use:** Registered as one of `editor.tsx`'s Tiptap extensions; not used or imported anywhere else.

Group 18 — AI Feature (Panel & Chat)

Why these belong together: the two distinct AI interaction patterns in the product — queued jobs with polling, and synchronous streaming chat — live side by side in this feature folder specifically to make that contrast visible.

File: `frontend/features/ai/ai-panel.tsx`

Type: Client Component **Purpose:** The collapsible sidebar hosting Summarize, Rewrite, and Ask (Chat) as three tabs. **Dependencies:** `hooks/use-ai.ts`, `features/ai/chat.tsx`, `framer-motion`. **Detailed Explanation:** Summarize renders the structured `DocumentSummary` shape (`summary`, `keyPoints[]`, `actionItems[]`) once a queued job completes. Rewrite offers a mode picker (improve/professional/shorter/longer/simplify — matching the backend's `REWRITE_MODES`) and displays the rewritten output with a copy button. Ask embeds the `<Chat/>` component scoped to the current document (`documentId` prop), so RAG retrieval for that conversation is limited to the document currently open. The panel's collapse/expand transition is animated via Framer Motion's width interpolation. **How To Use:** Rendered by `(workspace)/documents/[id]/page.tsx`; its open/closed state and active tab are driven by `useUiStore().aiPanelOpen` and local component state respectively, with the editor's slash-menu able to open it directly to a specific tab via the `onAiAction` callback.

File: `frontend/features/ai/chat.tsx`

Type: Client Component **Purpose:** The streaming RAG chat interface — the one AI feature that doesn't go through the queued-job pattern. **Dependencies:** `services/index.ts`'s `streamAsk()`, `AbortController`, `react-markdown` + `remark-gfm`. **Detailed Explanation:** Calls `streamAsk()` directly rather than a queued mutation — the SSE response *is* the answer, appended token-by-token into the last message in local `messages` state as each `token` frame arrives. An `AbortController` is wired to a stop button, letting the user cancel a stream mid-flight (aborting the underlying `fetch`). A subtle React StrictMode bug is explicitly avoided and documented in a code comment: naively appending a token to the last message by mutating it in place, even inside an otherwise-immutable state update, still shares object identity with the previous render's state — under StrictMode's deliberate double-invocation of state updaters (a development-only behavior meant to surface exactly this class of bug), this would silently double-append every streamed token; the fix is to always create a genuinely new message object (a fresh shallow copy) on every token, not just a new array wrapping the same mutated object. **Key Code Sections Explained:** The "always shallow-copy the last message object, not just the array" detail is the one piece of code in this file most likely to look unnecessary at a glance but is actually load-bearing — removing it would only manifest as a bug under React StrictMode (development) or React's future concurrent-rendering behaviors, not in a simple manual test. **How To Use:** Rendered by `ai-panel.tsx`'s Ask tab (scoped to a document) and by `(workspace)/assistant/page.tsx`'s Chat tab (unscoped, searching across all documents).

Two distinct AI UX patterns, side by side:

	SUMMARIZE / REWRITE / GENERATE	ASK (CHAT)
Backend endpoint pattern	Enqueue → returns <code>PENDING</code> row immediately	Direct SSE stream — the response <i>is</i> the answer
Frontend hook	<code>useAiJob()</code> — posts, then polls <code>refetchInterval</code> every 1.5s until status leaves <code>PENDING</code> / <code>PROCESSING</code>	<code>streamAsk()</code> called directly, no queue involved
Why polling here	The hook's own comment: polling "stays correct across reconnects" — the socket's <code>job:progress</code> also updates a progress bar, but the <i>final</i> state always comes from the polled row, not the socket alone	N/A — there's no queued job to poll; the stream itself is the completion signal
Cancellable	No (once enqueued, it runs to completion or failure)	Yes — <code>AbortController</code> lets the user stop mid-stream

Group 19 — Files Feature & React Query Hooks

Why these belong together: the upload dropzone and its supporting hook implement the exact three-step signed-upload flow the backend's `files` module was designed around; the other hooks share the same React-Query conventions.

File: `frontend/features/files/upload-dropzone.tsx`

Type: Client Component **Purpose:** Drag-and-drop (or click-to-browse) file upload UI. **Detailed Explanation:** Accepts `.pdf, .docx, .txt, .md, .markdown` and enforces a client-side 20MB size gate mirroring the backend DTO's limit exactly — explicitly so that an obviously-oversized file is rejected instantly in the browser rather than round-tripping to the backend just to be told no. **How To Use:** Rendered on `(workspace)/files/page.tsx` ; delegates the actual upload mechanics entirely to `useUploadFile()` .

File: `frontend/hooks/use-files.ts`

Type: React Query hooks **Purpose:** `useFiles()` (list, with a resilience poll) and `useUploadFile()` (the three-step upload mutation). **Detailed Explanation:** `useUploadFile()` implements the exact sequence the backend expects: (1) `filesService.createUploadTicket()` returns `{fileId, path, signedUrl, token}` ; (2) `supabase.storage.from(bucket).uploadToSignedUrl(path, token, body)` — bytes travel browser-to-Supabase directly, never through the NestJS backend; (3) `filesService.complete(fileId)` tells the backend the upload actually finished, which is what triggers `FileProcessor` to run. A specific, well-documented bug fix lives in this file: browsers report an empty `File.type` string for `.md` files, and Supabase's storage-js client reads the MIME type off the `Blob` object itself rather than accepting an explicit `contentType` override — so uploading a `.md` file with its natively-empty type would otherwise fail with a MIME-type-not-supported error. The fix re-wraps the file in `new File([file], name, {type: guessedType})` whenever `file.type` is falsy, using the filename extension to guess the correct MIME type before handing it to Supabase.

`useFiles()` also polls every 4 seconds while any listed file is still `PENDING` or `PROCESSING` — a fallback described in a code comment as "belt-and-braces alongside socket events," ensuring the file list eventually reflects true status even if the Socket.IO connection drops or reconnects mid-job. **Key Code Sections Explained:** The `.md` MIME-type re-wrapping is the single most "hard-won" piece of frontend code in the project — the kind of fix that's invisible unless you know exactly which upload format triggers it. **How To Use:** `useUploadFile()` is called from `upload-dropzone.tsx` ; `useFiles()` from `(workspace)/files/page.tsx` .

Remaining hooks (table):

FILE	PURPOSE
<code>hooks/use-documents.ts</code>	Query keys namespaced under <code>documents</code> (<code>list(q)</code> , <code>detail(id)</code> , <code>recent</code>). <code>useUpdateDocument</code> and <code>useDeleteDocument</code> both perform optimistic updates (<code>onMutate</code> snapshots and patches the cache immediately, <code>onError</code> rolls back) — this is what makes the editor's autosave and document deletion feel instant
<code>hooks/use-ai.ts</code>	<code>aiKeys.history / request(id)</code> ; <code>useAiJob()</code> composes the three mutation-plus-polling patterns described above for <code>summarize/rewrite/generate</code>
<code>hooks/use-dashboard.ts</code>	<code>useDashboard()</code> (a simple query against the backend's one-transaction aggregate) and <code>useSearch(query, mode)</code> (gated with <code>enabled: query.trim().length > 1</code> , 15-second <code>staleTime</code>)

All mutation hooks across this feature surface use `sonner` 's `toast.error` / `toast.success` for user feedback and invalidate the relevant React Query cache keys on success, so a mutation's effects are visible everywhere that data is displayed without a manual refetch call at each call site.

Group 20 — UI Components & Layout Shell

FILE	TYPE	PURPOSE
<code>components/command-palette.tsx</code>	Client Component	Built on <code>cmdk</code> 's <code>Command.Dialog</code> ; global <code>Ctrl/Cmd+K</code> listener toggles <code>useUiStore().commandPaletteOpen</code> . Groups: quick actions (new document), static page links, a live document list (<code>useDocuments()</code>), and semantic search results (<code>useSearch(query, 'semantic')</code>), itself gated on query length inside the hook
<code>components/layout/sidebar.tsx</code>	Client Component	Static navigation list with active-route highlighting via a shared Framer Motion <code>layoutId="sidebar-active"</code> element (animating one highlight pill between nav items rather than each item independently managing its own highlight state), a recent-documents scroll area, and a "new document" button
<code>components/layout/topbar.tsx</code>	Client Component	Search-trigger button (opens the command palette), a theme toggle (explicitly waits for client-side mount before rendering the sun/moon icon, avoiding an SSR/hydration mismatch since the correct icon depends on browser-only theme state), and a user menu (avatar, logout — which POSTs <code>/auth/logout</code> , disconnects the socket, clears the auth store, and redirects to <code>/login</code> , in that order)
<code>components/ui/avatar.tsx</code>	UI primitive	Radix <code>Avatar</code> wrapper with a fallback showing user initials
<code>components/ui/button.tsx</code>	UI primitive	<code>cva</code> -based variants (default/destructive/outline/secondary/ghost/link × sizes), <code>asChild</code> support via <code>@radix-ui/react-slot</code> for polymorphic rendering
<code>components/ui/card.tsx</code>	UI primitive	A single shared <code>make()</code> factory function generating <code>Card</code> / <code>CardHeader</code> / <code>CardTitle</code> / <code>CardDescription</code> / <code>CardContent</code> / <code>CardFooter</code> — one implementation, not six near-duplicate components
<code>components/ui/dialog.tsx</code>	UI primitive	Radix <code>Dialog</code> wrapper
<code>components/ui/dropdown-menu.tsx</code>	UI primitive	Radix <code>DropdownMenu</code> wrapper, extended with a <code>destructive</code> variant prop for delete-style menu items
<code>components/ui/input.tsx</code>	UI primitive	<code>Input</code> and <code>Textarea</code>
<code>components/ui/misc.tsx</code>	UI primitive	<code>Badge</code> (success/warning/destructive/secondary/outline variants), <code>Skeleton</code> , <code>Separator</code> , <code>ScrollArea</code> , and a hand-rolled <code>Progress</code> bar (no Radix progress primitive is used here, since Radix doesn't ship one this project needed)
<code>components/ui/tabs.tsx</code>	UI primitive	Radix <code>Tabs</code> wrapper

Group 21 — Frontend Tooling & Configuration

FILE	TYPE	PURPOSE
<code>instrumentation.ts</code>	Bootstrap	Next.js 15's server/edge instrumentation hook — <code>register()</code> , split by <code>NEXT_RUNTIME</code> (nodejs vs. edge), each independently calling <code>Sentry.init</code> since they run in genuinely different JS runtimes
<code>instrumentation-client.ts</code>	Bootstrap	The browser-side counterpart — its own <code>Sentry.init</code> call, with session/error-replay sampling explicitly disabled
<code>next.config.ts</code>	Config	<code>output: 'standalone'</code> only when <code>DOCKER_BUILD=true</code> (used by the Dockerfile to produce a minimal self-contained server bundle); whitelists two remote image hosts (GitHub avatars, Supabase) for <code>next/image</code> ; a scoped <code>webpack.ignoreWarnings</code> entry suppressing one specific benign warning from Sentry's OpenTelemetry dynamic <code>require()</code> (webpack can't statically trace it), without suppressing other, potentially real, warnings

FILE	TYPE	PURPOSE
<code>tailwind.config.ts</code>	Config	Standard shadcn/ui-style setup — every color mapped to a CSS custom property (<code>hsl(var(--x))</code>) for <code>next-themes</code> ' class-based dark/light strategy, plus <code>tailwindcss-animate</code> and <code>@tailwindcss/typography</code> plugins, content globs scoped to <code>app/</code> , <code>components/</code> , <code>features/</code>
<code>postcss.config.mjs</code>	Config	Standard Tailwind/Autoprefixer PostCSS pipeline
<code>frontend/package.json</code>	Config	Scripts (<code>dev</code> , <code>build</code> , <code>start</code> , <code>lint</code> , <code>typecheck</code>), full dependency list
<code>frontend/tsconfig.json</code>	Config	TypeScript compiler options for the Next.js app
<code>frontend/.eslintrc.json</code>	Config	<code>eslint-config-next</code> — Next.js's own recommended lint rule set
<code>frontend/Dockerfile</code>	Build	Multi-stage build: <code>deps</code> (pnpm install filtered to <code>@kw/frontend...</code> , <code>--frozen-lockfile</code>) → <code>build</code> (bakes <code>NEXT_PUBLIC_*</code> values as build arguments , since Next.js inlines them into the client bundle at build time — they cannot be changed later by setting an environment variable on the running container) → <code>runtime</code> (copies only <code>.next/standalone</code> , <code>.next/static</code> , and <code>public</code> , running <code>node frontend/server.js</code>)

Group 22 — Monorepo Root & Deployment Configuration

Why these belong together: everything that exists above the `backend/` / `frontend/` split — what ties the two packages together and what governs how the whole project ships.

FILE	TYPE	PURPOSE
<code>package.json</code> (root)	Config	Root scripts orchestrating both packages: <code>dev</code> (runs backend + frontend in parallel via <code>pnpm --parallel --filter</code>), <code>build</code> (backend then frontend, sequentially — the backend's Prisma client must exist before anything depends on it), <code>lint</code> / <code>test</code> / <code>test:e2e</code> (delegate into the relevant package), <code>db:generate</code> / <code>db:migrate</code> / <code>db:push</code> (Prisma commands, scoped to <code>backend</code>), <code>docker:up</code> / <code>docker:down</code> (starts/stops just Postgres and Redis from Compose, for local development without running the full Compose stack)
<code>pnpm-workspace.yaml</code>	Config	Declares <code>backend</code> and <code>frontend</code> as the two workspace packages; <code>onlyBuiltDependencies</code> allowlist for packages that genuinely need postinstall scripts (<code>@prisma/client</code> , <code>@prisma/engines</code> , <code>prisma</code> , <code>msgpackr-extract</code> , <code>protobufjs</code> , <code>sharp</code>)
<code>docker-compose.yml</code>	Config	Local development stack — <code>pgvector/pgvector:pg16</code> on host port 5433, Redis 7 on 6379, plus optional full <code>backend</code> / <code>frontend</code> service definitions for a closer-to-production local run; healthchecks gate service startup ordering
<code>render.yaml</code>	Deployment	Render's declarative blueprint for the backend — Docker-runtime web service, health check path, auto-generated JWT secrets, <code>sync: false</code> for every sensitive/environment-specific variable
<code>vercel.json</code>	Deployment	Vercel's monorepo-aware build configuration — scopes install/build commands to <code>@kw/frontend</code> only, points at <code>frontend/.next</code> as the output directory
<code>.env.example</code>	Config template	A fully-commented template of every environment variable either package needs, explicitly never containing real credentials (committed to the repository intentionally, unlike the actual <code>.env</code> / <code>.env.local</code> files it's copied into)
<code>.github/workflows/ci.yaml</code>	CI/CD	The full pipeline described in Part 1's GitHub Actions entry — backend lint/test/e2e against real service containers, frontend typecheck/build, a Docker-build-verification job, and a conditional Render deploy trigger on merges to <code>main</code>

FILE	TYPE	PURPOSE
<code>.gitignore</code> / <code>.dockerignore</code>	Config	Standard exclusion lists — <code>node_modules</code> , build output, <code>.env</code> / <code>.env.local</code> (the real, secret-containing files, as opposed to the committed <code>.env.example</code> template), etc.
<code>README.md</code>	Documentation	The project's own high-level overview, architecture diagram, and the explicit "decisions worth calling out" section this document draws directly from and expands upon throughout

Part 4 — Implementation Flow (Build Order)

This is the order in which building this project actually makes sense — each step depends only on what came before it, and matches the grouping used in Part 3.

Step 1: Monorepo scaffolding

What: Set up the pnpm workspace, root scripts, and the two empty packages. **Files created:** `package.json` (root), `pnpm-workspace.yaml` , `backend/package.json` , `frontend/package.json` (initial Next.js/NestJS scaffolds). **Why:** Every other step needs somewhere to put its code, and the workspace filtering (`--filter`) convention needs to exist before any per-package script is written. **Technologies introduced:** pnpm workspaces, TypeScript, Next.js, NestJS.

Step 2: Data model & local infrastructure

What: Design the Prisma schema, write the pgvector/tsvector raw migration, and get Postgres + Redis running locally. **Files created:** `backend/prisma/schema.prisma` , `backend/prisma/migrations/20250101000000_init/migration.sql` , `docker-compose.yml` , `.env.example` . **Why:** Nothing else — no service, no controller — can be written meaningfully without knowing what data it operates on. Getting a real pgvector-enabled Postgres running locally at this stage (rather than mocking it) is what allows the raw-SQL vector/FTS code to be developed and tested against real behavior from day one. **Technologies introduced:** PostgreSQL, pgvector, Prisma, Docker Compose.

Step 3: Backend bootstrap & global wiring

What: Get a minimal NestJS app booting with the cross-cutting pieces every feature will need. **Files created:** `main.ts` , `app.module.ts` , `config/configuration.ts` , `config/env.validation.ts` , `common/*` (guards, filters, interceptors, decorators, DTOs), `health.controller.ts` , `prisma/prisma.service.ts` , `prisma/prisma.module.ts` . **Why:** Building the global `ValidationPipe` , `AllExceptionsFilter` , `LoggingInterceptor` , and the `@Public()` / `JwtAuthGuard` default-deny pattern *before* any feature module exists means every feature module written afterward automatically inherits consistent auth, error handling, and validation — rather than each feature needing to remember to wire these up individually. **Technologies introduced:** NestJS module system, class-validator, Sentry (optional).

Step 4: Authentication

What: GitHub OAuth login, JWT issuance, and the refresh-rotation-with-theft-detection scheme. **Files created:** the entire `auth/` module, plus the `RefreshToken` / `User` Prisma models (already present in the schema from Step 2, but now actually used). **Why:** Every other feature module's routes are protected by the global `JwtAuthGuard` from Step 3 — auth has to exist and work before any other feature's HTTP endpoints are meaningfully testable end to end. **Technologies introduced:** Passport.js, `passport-github2` , `passport-jwt` , JWT.

Step 5: Core CRUD — users and documents

What: Profile management, the dashboard aggregate query, and full document CRUD including the Tiptap→plain-text conversion.

Files created: `users/*` , `documents/*` (including `tiptap.util.ts`). **Why:** Documents are the core content object of the entire product — files, search, and AI all ultimately operate on a `Document` 's `plainText` . Building this before file uploads or AI means the RAG pipeline has something real to index and retrieve from during its own development. **Technologies introduced:** Prisma `$transaction` , `class-transformer/class-validator` DTOs.

Step 6: File uploads

What: The signed-upload-to-Supabase flow, text extraction, and the two-phase-commit upload pattern. **Files created:** `files/*` (`storage.service.ts` , `text-extractor.service.ts` , `files.service.ts` , `files.controller.ts`). **Why:** This step needs Documents (Step 5) to already exist, since an uploaded file's extracted text ultimately becomes a `Document` row. At this stage, file processing can still be built and tested as a purely synchronous operation (no queue yet) — the async/BullMQ layer comes next.

Technologies introduced: Supabase Storage, `pdf-parse` , `mammoth` .

Step 7: Async infrastructure — BullMQ

What: Introduce the queue/worker split, move file processing (and eventually AI and indexing) behind it, and implement the debounced-reindexing mechanism. **Files created:** `queue/*` (constants, types, redis config, module, service), `worker.module.ts` , `queue/workers/file.processor.ts` (the first processor, converting Step 6's synchronous file processing into an async job). **Why:** Introducing BullMQ only after a synchronous version already works (Step 6) means the queue-related bugs (job serialization, retry behavior, Redis connectivity) can be debugged in isolation from "does the file-processing logic itself work" — those are two separable concerns. **Technologies introduced:** BullMQ, `ioredis/Upstash`.

Step 8: The AI module — providers, chunking, and RAG

What: The `AIProvider` / `EmbeddingProvider` interfaces, the two embedding implementations, `GrokProvider` , the chunking algorithm, `IndexingService` , prompts, and `AIService` 's orchestration. **Files created:** the entire `ai/` module, `queue/workers/ai.processor.ts` and `indexing.processor.ts` (extending the BullMQ pattern from Step 7). **Why:** This is the most architecturally significant module, and deliberately built after the queue infrastructure (Step 7) already exists and is proven — every AI operation except streaming chat is queue-based from the moment it's written, rather than starting synchronous and being migrated later the way file processing was in Steps 6–7. **Technologies introduced:** the `openai` SDK (for both Grok and OpenAI embeddings), `@xenova/transformers` .

Step 9: Search

What: Full-text search (the generated tsvector column, already defined in Step 2's migration) and semantic search (built on Step 8's `IndexingService`). **Files created:** `search/*` . **Why:** Semantic search has a hard dependency on the AI module's `IndexingService` (Step 8) — it literally imports `AIModule` to use it. Full-text search has no such dependency and could technically have been built earlier, but is grouped here since the two are presented to the user as one unified feature. **Technologies introduced:** PostgreSQL full-text search (`websearch_to_tsquery` , `ts_rank` , `ts_headline`).

Step 10: Realtime progress

What: The Socket.IO gateway, wired into every BullMQ processor built in Steps 7–8. **Files created:** `events/*` . **Why:** Realtime progress is a UX enhancement layered on top of already-functioning async infrastructure — it makes sense to build once there's real background work happening to report progress on, rather than building the notification channel before there's anything to notify about. **Technologies introduced:** Socket.IO.

Step 11: Backend test suites

What: Unit tests for each service (colocated `*.spec.ts` files) and the single, comprehensive e2e test exercising the full async pipeline. **Files created:** every `*.spec.ts` file, `test/app.e2e-spec.ts`, `test/jest-e2e.json`. **Why:** Placed after the full backend feature set exists specifically so the e2e test can exercise a genuinely complete flow (auth → document → upload → worker → embed → summarize) — writing it earlier would mean testing an incomplete system. **Technologies introduced:** Jest, ts-jest, Supertest.

Step 12: Frontend foundation

What: The App Router skeleton, global providers, auth store, and the API client (including the single-flight refresh pattern). **Files created:** `app/layout.tsx`, `app/page.tsx`, `login/page.tsx`, `auth/callback/page.tsx`, `(workspace)/layout.tsx`, `components/providers.tsx`, `store/auth-store.ts`, `store/ui-store.ts`, `lib/*`, `services/index.ts`. **Why:** Every subsequent frontend feature depends on being able to make an authenticated API call and render inside the workspace shell — this has to exist first. **Technologies introduced:** Next.js 15 App Router, Zustand, React Query, Socket.IO client.

Step 13: Documents UI & the editor

What: The documents list, the document detail page, and the full Tiptap editor with autosave and the slash-command menu. **Files created:** `(workspace)/documents/page.tsx`, `(workspace)/documents/[id]/page.tsx`, `features/editor/*`, `hooks/use-documents.ts`. **Why:** The editor is the app's primary interaction surface — building it early (once auth and API plumbing exist) means every subsequent feature (AI panel, file upload) can be developed and tested against a real, working document rather than a stub. **Technologies introduced:** Tiptap, `@tiptap/suggestion`, `tippy.js`, shadcn-style UI components (Radix + Tailwind + `cva`).

Step 14: Files UI

What: The upload dropzone, the three-step signed-upload flow, and the files list with live status. **Files created:** `(workspace)/files/page.tsx`, `features/files/upload-dropzone.tsx`, `hooks/use-files.ts`. **Why:** Requires both the backend's Step 6/7 upload pipeline and the frontend's Step 12 API client and Step 10 realtime infrastructure (for live progress) to already exist.

Step 15: AI panel, chat, search UI, dashboard, and settings

What: The remaining workspace pages and features — `AiPanel` / `Chat` (both interaction patterns), search results UI, the dashboard, and settings. **Files created:** `features/ai/*`, `hooks/use-ai.ts`, `hooks/use-dashboard.ts`, `(workspace)/search/page.tsx`, `(workspace)/assistant/page.tsx`, `(workspace)/dashboard/page.tsx`, `(workspace)/settings/page.tsx`, `components/command-palette.tsx`. **Why:** These are the final consumer-facing features, each depending on backend capability built in Steps 8–10 already being complete and stable — this is the layer where the hand-rolled SSE parser (`streamAsk`) and the queued-job polling pattern (`useAiJob`) both get built and connected to real, working endpoints.

Step 16: Deployment & CI/CD

What: Dockerfiles for both packages, Render/Vercel configuration, and the GitHub Actions pipeline. **Files created:** `backend/Dockerfile`, `frontend/Dockerfile`, `render.yaml`, `vercel.json`, `.github/workflows/ci.yml`. **Why:** Deployment configuration is written last because it packages an already-complete, already-tested application — writing it earlier would mean deploying and redeploying an incomplete system on every change, with no test suite (Step 11) yet in place to gate those deploys. **Technologies introduced:** Docker, GitHub Actions, Render, Vercel, Upstash.

Part 5 — Execution Summary

How to run the project

Prerequisites: Node 20+, pnpm 10, Docker.

```
pnpm install

# Starts Postgres (pgvector, host port 5433) and Redis (6379) via Docker Compose
pnpm docker:up

cp .env.example backend/.env      # then fill in real values (see below)
pnpm --filter @kw/backend prisma:deploy

pnpm dev                          # backend on :4000, frontend on :3000
```

The **minimum** set of environment variables that must be filled in for the app to be genuinely usable (rather than merely booting):

`GITHUB_CLIENT_ID` / `GITHUB_SECRET` (a GitHub OAuth App, callback URL `http://localhost:4000/api/auth/github/callback`), `JWT_SECRET` / `JWT_REFRESH_SECRET` (any two distinct 32+ character random strings), `GROK_API_KEY` (from console.x.ai), `SUPABASE_URL` / `SUPABASE_SERVICE_KEY` (a Supabase project with a **private** bucket named `knowledge-files`), and `REDIS_URL` (Upstash's `rediss://` **TCP** endpoint if not using local Docker Redis). The frontend additionally needs `NEXT_PUBLIC_SUPABASE_URL` / `NEXT_PUBLIC_SUPABASE_ANON_KEY` in `frontend/.env.local` for direct browser uploads. Embeddings default to `EMBEDDING_PROVIDER=local`, requiring no API key or additional cost — set it to `openai` (with `OPENAI_API_KEY`) for better retrieval quality at the cost of a per-call price.

What happens when you run it

- `docker compose up -d postgres redis` starts a `pgvector`-enabled Postgres 16 instance on host port 5433 and Redis on 6379, waiting for both healthchecks to pass.
- `prisma migrate deploy` applies every migration, including the raw-SQL HNSW vector index and the generated tsvector column that `schema.prisma` alone can't express.
- `pnpm dev` starts both the NestJS backend (`nest start --watch`, port 4000) and the Next.js frontend (`next dev`, port 3000) in parallel.
- On first load, the frontend redirects an unauthenticated visitor to `/login`; clicking "Sign in with GitHub" performs a real OAuth handshake, after which the backend sets an `httpOnly` refresh cookie and redirects to `/auth/callback`, which trades that cookie for an in-memory access token and lands the user on `/dashboard`.
- Creating a document opens the Tiptap editor; typing triggers a 900ms-debounced autosave; editing content (not just the title) enqueues a 15-second-debounced BullMQ reindexing job, which chunks, embeds (cold-starting the local MiniLM model on first use, ~2-4 seconds), and writes vectors to Postgres — with live progress visible via Socket.IO.
- Uploading a PDF/DOCX/TXT/MD file mints a signed Supabase Storage URL, uploads bytes directly from the browser, and — once confirmed complete — triggers `FileProcessor`: download → extract text → create a `Document` → chunk → embed, with granular per-stage progress shown live.
- Search queries hit both the keyword (Postgres full-text) and semantic (pgvector cosine similarity) paths in parallel, shown side by side.
- Asking a question in the Assistant tab streams a token-by-token, citation-grounded answer over SSE, retrieved via the same semantic-search infrastructure that powers the Search page — this is the one AI operation that runs synchronously rather than through the job queue.
- Summarize/Rewrite/Generate all follow the queued pattern: an immediate `PENDING` response, background execution via a BullMQ worker calling Grok, and the frontend polling (backed up by Socket.IO progress) until the job resolves.

Expected inputs and outputs

FEATURE	INPUT	OUTPUT
Document editing	Tiptap rich-text edits	A saved <code>Document</code> row (JSON content + flattened plain text), eventually re-embedded chunks
File upload	A PDF/DOCX/TXT/MD file (≤20MB)	A new editable <code>Document</code> , extracted from the file's text, fully searchable and chat-able
Summarize	A document (or raw text)	<code>{summary, keyPoints[], actionItems[]}</code> , written onto the document
Rewrite	Content + a mode (improve/professional/shorter/longer/simplify)	Rewritten text
Generate	A freeform prompt	Generated markdown text
Ask (Chat)	A question, optional document scope, conversation history	A streamed, cited answer grounded in the user's own documents — or an explicit "I don't know" if nothing relevant is retrieved
Search	A query string + mode	Keyword hits (with <code><mark></code> -highlighted snippets) and/or semantic hits (with a similarity-ranked excerpt)

Common workflows

- **Write-and-search:** create a document, write notes, later find it again via keyword or semantic search using words that don't even appear verbatim in the text (semantic search matches meaning, not just terms).
- **Upload-and-ask:** upload a PDF report, wait for processing to complete (visible live in the Files page), then ask a question in the Assistant and receive an answer citing the specific uploaded document.
- **Draft-and-refine:** use Generate to produce a first draft from a prompt, then Rewrite it in a different tone, iterating entirely through the AI panel without leaving the editor.
- **Cross-document research:** ask a question in the unscoped Assistant chat (not tied to one document), retrieving and citing relevant chunks from across the user's entire document set.

Index of Technologies

TECHNOLOGY	CATEGORY	SECTION
TypeScript	Language	Languages
SQL / PostgreSQL / pgvector	Language	Languages
NestJS 11	Backend framework	Backend Framework & Libraries
Prisma 6	ORM	Backend Framework & Libraries
Passport.js	Auth	Backend Framework & Libraries
class-validator / class-transformer	Validation	Backend Framework & Libraries

TECHNOLOGY	CATEGORY	SECTION
BullMQ + ioredis	Job queue	Async Processing & Realtime
Socket.IO	Realtime	Async Processing & Realtime
xAI Grok (via <code>openai</code> SDK)	LLM API	APIs & External Services
<code>@xenova/transformers</code> / OpenAI embeddings	Embeddings	APIs & External Services
Supabase (Postgres & Storage)	Managed platform	APIs & External Services
<code>pdf-parse</code> / <code>mammoth</code>	File extraction	APIs & External Services
Sentry	Observability	APIs & External Services
Next.js 15	Frontend framework	Frontend Framework & Libraries
TanStack Query (React Query)	Server-state	Frontend Framework & Libraries
Zustand	Client-state	Frontend Framework & Libraries
Tiptap / ProseMirror	Rich-text editor	Frontend Framework & Libraries
Tailwind CSS / Radix UI / cmdk / Framer Motion	UI layer	Frontend Framework & Libraries
pnpm 10	Package manager	Build Tools & Package Management
Docker / Docker Compose	Containerization	Build Tools & Package Management
Jest / ts-jest / Supertest	Testing	Testing & Quality Assurance
Vercel	Frontend hosting	Deployment & DevOps
Render / Upstash	Backend hosting / Redis	Deployment & DevOps
GitHub Actions	CI/CD	Deployment & DevOps
helmet, cookie-parser, reflect-metadata, rxjs, date-fns, lucide-react, sonner, next-themes, react-markdown, ESLint, Prettier	Utilities & tooling	Other Utilities & Dev Tooling

Index of Files

Grouped by the module structure used in Part 3. Every backend and frontend source file is listed.

Backend — Bootstrap: `main.ts` · `instrument.ts` · `app.module.ts` · `health.controller.ts` · `config/configuration.ts` · `config/env.validation.ts` · `common/decorators/current-user.decorator.ts` · `common/decorators/public.decorator.ts` · `common/dto/pagination.dto.ts` · `common/filters/all-exceptions.filter.ts` · `common/interceptors/logging.interceptor.ts` · `common/types/jwt-payload.ts`

Backend — Data layer: prisma/schema.prisma · prisma/migrations/20250101000000_init/migration.sql · prisma/migrations/migration_lock.toml · src/prisma/prisma.service.ts · src/prisma/prisma.module.ts

Backend — Auth: auth/auth.service.ts · auth/auth.controller.ts · auth/auth.module.ts · auth/auth.service.spec.ts · auth/guards/github-auth.guard.ts · auth/guards/jwt-auth.guard.ts · auth/strategies/github.strategy.ts · auth/strategies/jwt.strategy.ts

Backend — Users: users/users.service.ts · users/users.controller.ts · users/users.module.ts · users/dto/update-user.dto.ts

Backend — Documents: documents/documents.service.ts · documents/documents.controller.ts · documents/documents.module.ts · documents/tiptap.util.ts · documents/dto/document.dto.ts · documents/documents.service.spec.ts

Backend — Files: files/files.service.ts · files/files.controller.ts · files/files.module.ts · files/storage.service.ts · files/text-extractor.service.ts · files/dto/file.dto.ts

Backend — AI: ai/ai.service.ts · ai/ai.controller.ts · ai/ai.module.ts · ai/chunking.util.ts · ai/indexing.service.ts · ai/prompts.ts · ai/ai.service.spec.ts · ai/dto/ai.dto.ts · ai/interfaces/ai-provider.interface.ts · ai/interfaces/embedding-provider.interface.ts · ai/providers/grok.provider.ts · ai/providers/local-embedding.provider.ts · ai/providers/openai-embedding.provider.ts

Backend — Queue: queue/queue.constants.ts · queue/queue.types.ts · queue/redis.config.ts · queue/queue.module.ts · queue/queue.service.ts · queue/worker.module.ts · queue/workers/ai.processor.ts · queue/workers/file.processor.ts · queue/workers/indexing.processor.ts

Backend — Events: events/events.gateway.ts · events/events.module.ts

Backend — Search: search/search.service.ts · search/search.controller.ts · search/search.module.ts · search/search.service.spec.ts

Backend — Tests & Tooling: test/app.e2e-spec.ts · test/jest-e2e.json · package.json · tsconfig.json · tsconfig.build.json · nest-cli.json · eslint.config.mjs · .prettierrc · Dockerfile

Frontend — App Router: app/layout.tsx · app/page.tsx · app/login/page.tsx · app/auth/callback/page.tsx · app/(workspace)/layout.tsx · app/(workspace)/dashboard/page.tsx · app/(workspace)/documents/page.tsx · app/(workspace)/documents/[id]/page.tsx · app/(workspace)/files/page.tsx · app/(workspace)/search/page.tsx · app/(workspace)/assistant/page.tsx · app/(workspace)/settings/page.tsx · app/globals.css

Frontend — Providers & State: components/providers.tsx · store/auth-store.ts · store/ui-store.ts

Frontend — API/lib/services: lib/api.ts · lib/socket.ts · lib/supabase.ts · lib/types.ts · lib/utils.ts · services/index.ts

Frontend — Editor feature: features/editor/editor.tsx · features/editor/slash-command.tsx

Frontend — AI feature: features/ai/ai-panel.tsx · features/ai/chat.tsx

Frontend — Files feature & hooks: features/files/upload-dropzone.tsx · hooks/use-files.ts · hooks/use-documents.ts · hooks/use-ai.ts · hooks/use-dashboard.ts

Frontend — Components: components/command-palette.tsx · components/layout/sidebar.tsx · components/layout/topbar.tsx · components/ui/avatar.tsx · components/ui/button.tsx · components/ui/card.tsx · components/ui/dialog.tsx · components/ui/dropdown-menu.tsx · components/ui/input.tsx · components/ui/misc.tsx · components/ui/tabs.tsx

Frontend — Tooling: instrumentation.ts · instrumentation-client.ts · next.config.ts · tailwind.config.ts · postcss.config.mjs · package.json · tsconfig.json · .eslintrc.json · Dockerfile

Root — Monorepo & Deployment: package.json · pnpm-workspace.yaml · docker-compose.yml · render.yaml · vercel.json · .env.example · .github/workflows/ci.yml · .gitignore · .dockerignore · README.md

End of document.